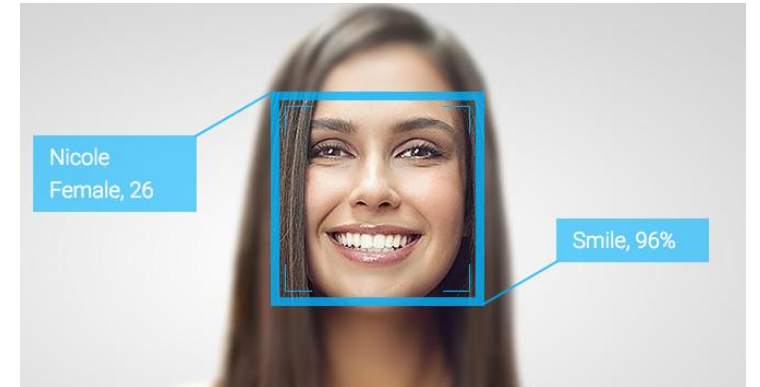
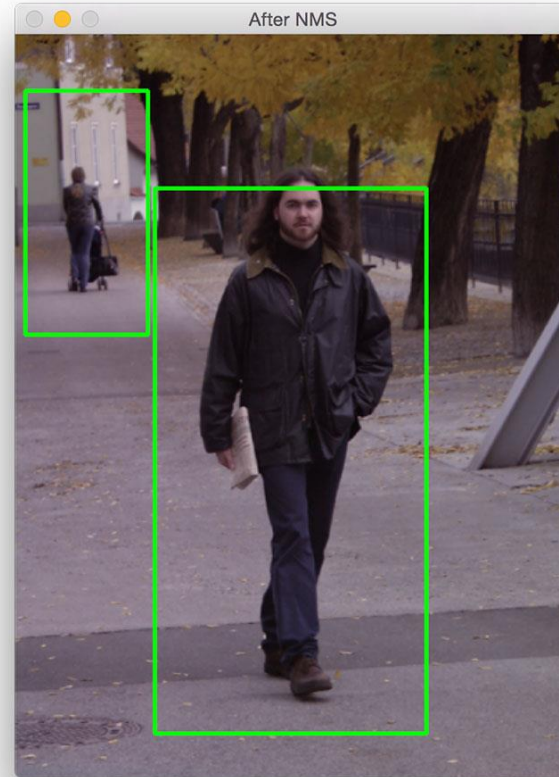
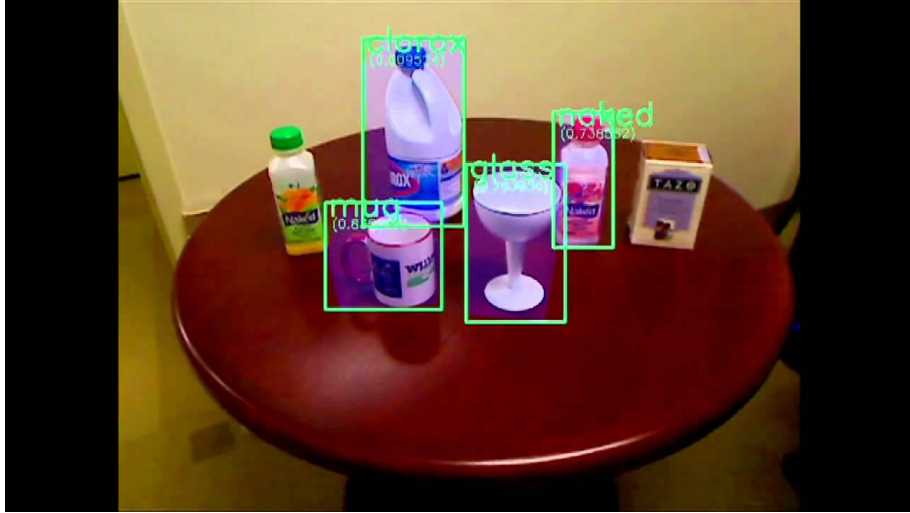


Retele neurale



Retele neurale



"little girl is eating piece of cake."



"baseball player is throwing ball
in game."



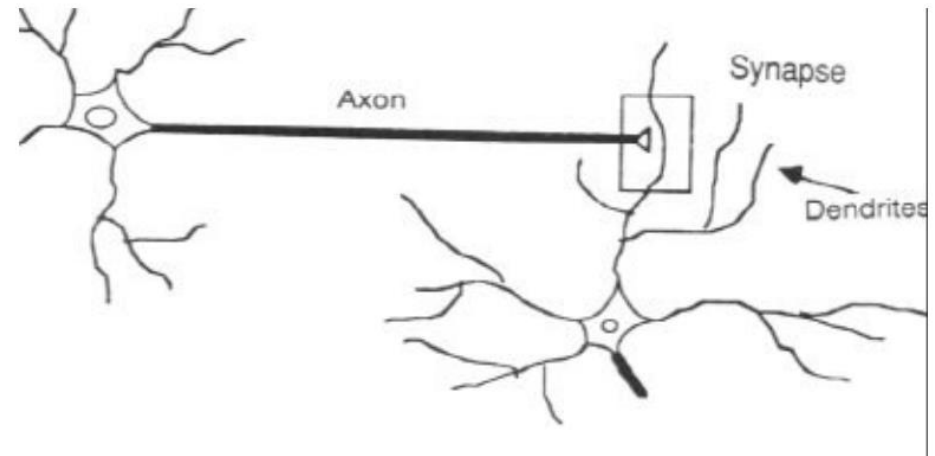
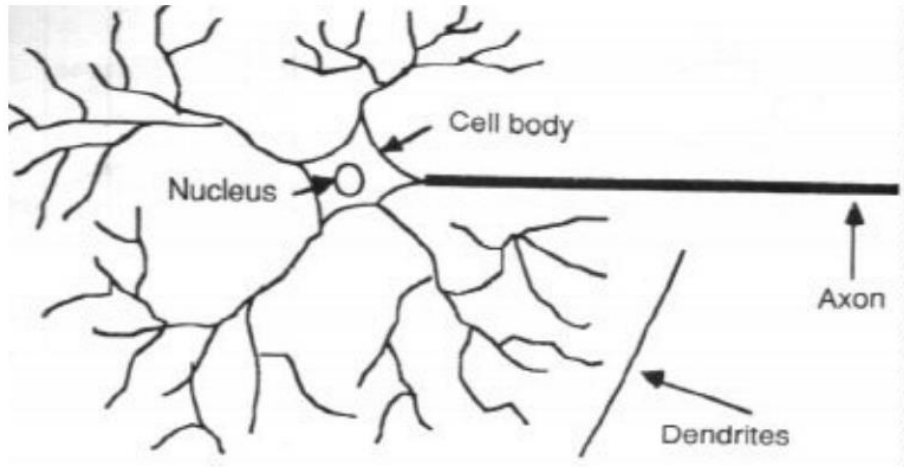
"woman is holding bunch of
bananas."



"black cat is sitting on top of
suitcase."

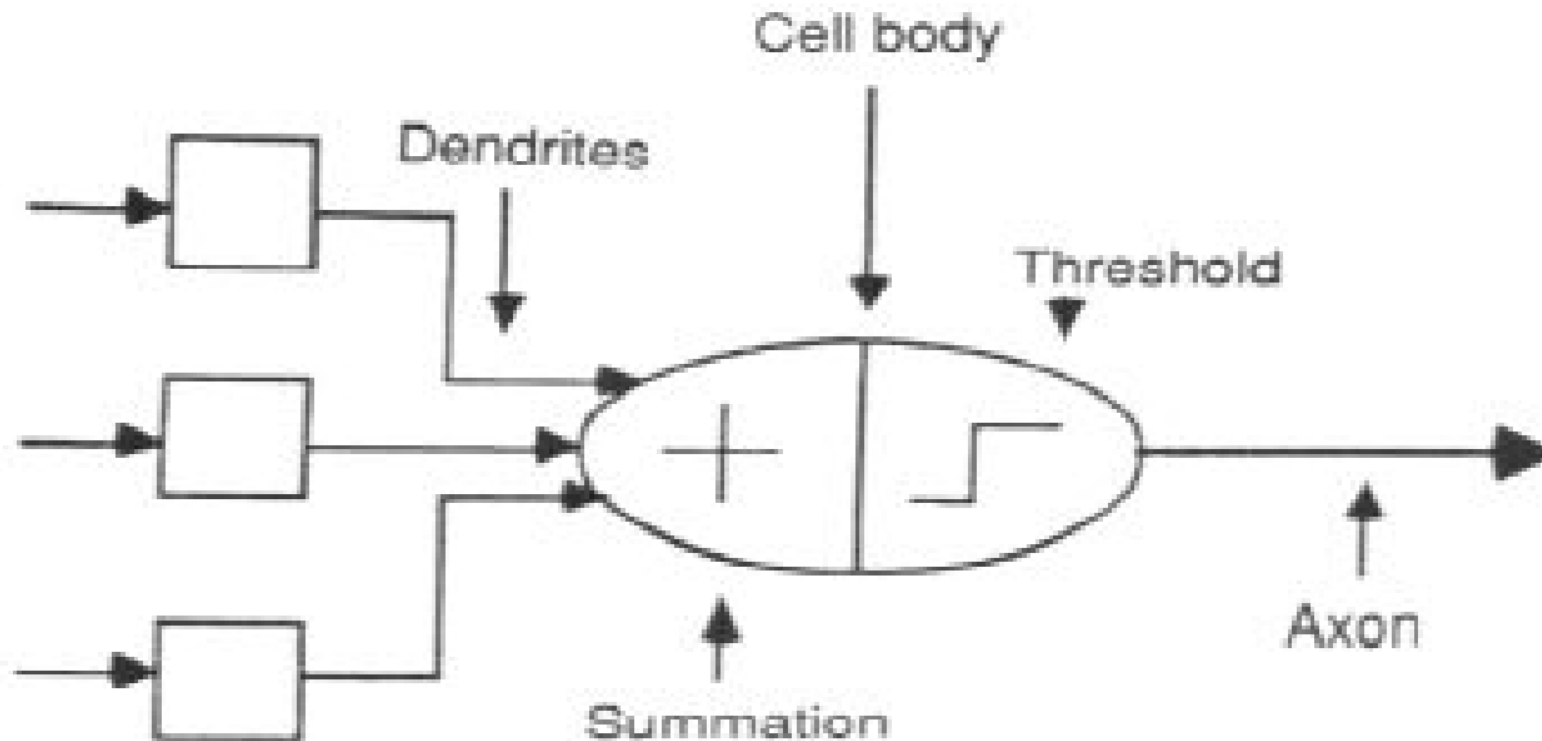
Retele neurale

- Retele neurale artificiale



Neuron artificial

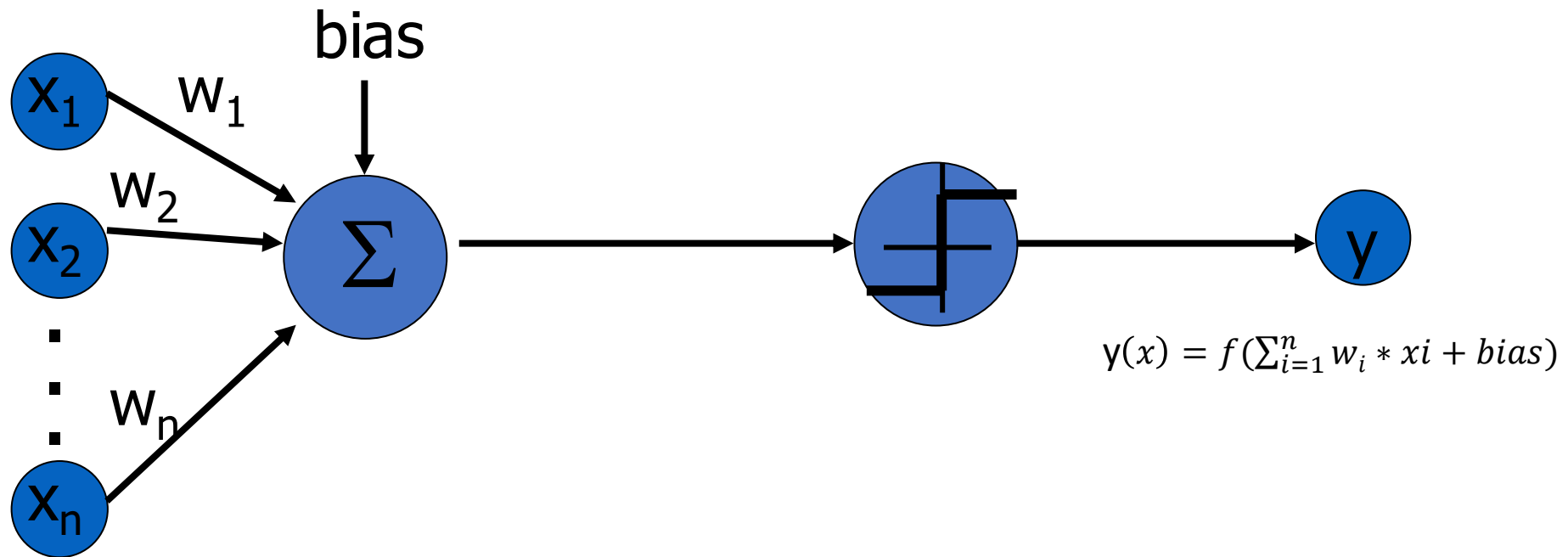
Warren S McCulloch and Walter Pitts, A logical calculus of the ideas immanent in nervous activity, The bulletin of mathematical biophysics 5 (1943), no. 4, 115–133



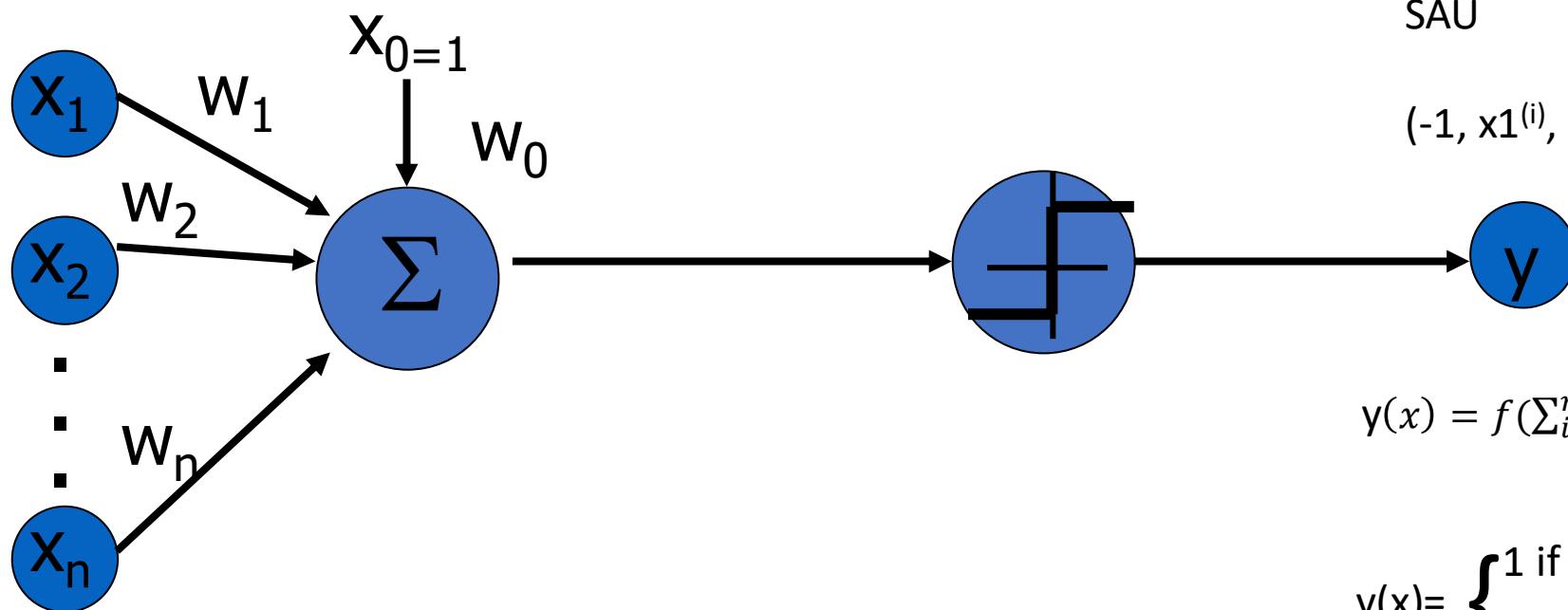
Modelul neuronului artificial

Antrenare:

- Setul de date de antrenare ($x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)} - y^{(i)}$)
- Invatare supervizata
- Antrenare -> bias, w_i



Modelul neuronului artificial



$w_0 = \text{bias}$

$(x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)} - y^{(i)}) \rightarrow$

$(1, x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)} - y^{(i)})$

SAU

$(-1, x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)} - y^{(i)})$

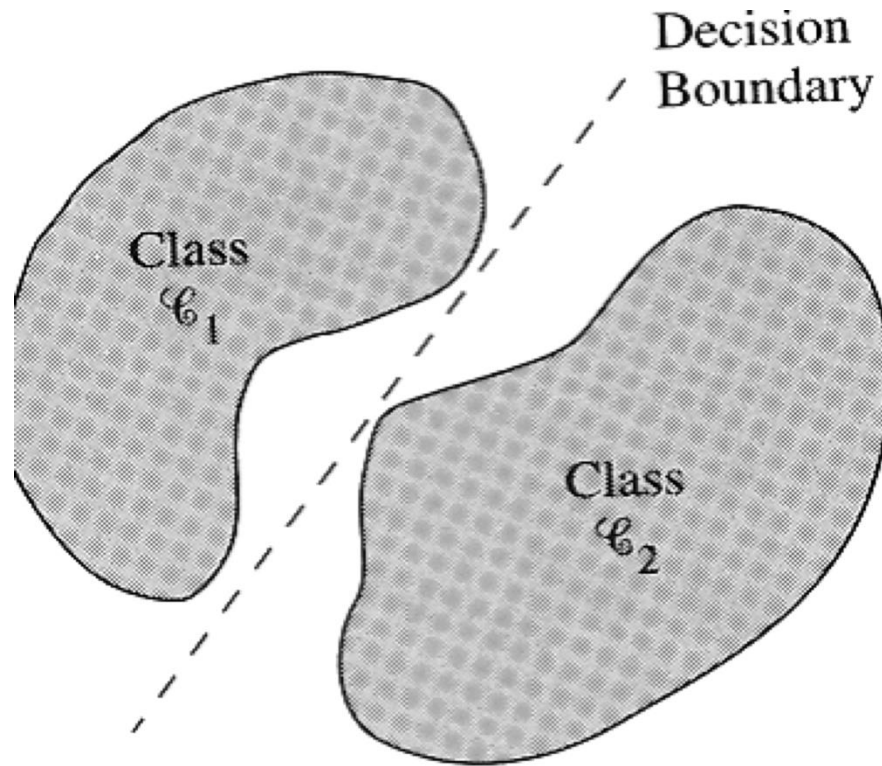
$$y(x) = f\left(\sum_{i=0}^n w_i * x_i\right)$$

$$y(x) = \begin{cases} 1 & \text{if } \sum_{i=0}^n w_i x_i > 0 \\ -1 & \text{altfel} \end{cases}$$

Perceptron

- Clasificare corecta $\mathbf{D}=\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$ in clasele C_1 si C_2
- Iesirea pentru lasa C_1 este $y=1$ si pentru C_2 este $y=-1$

• $n=2 \rightarrow$

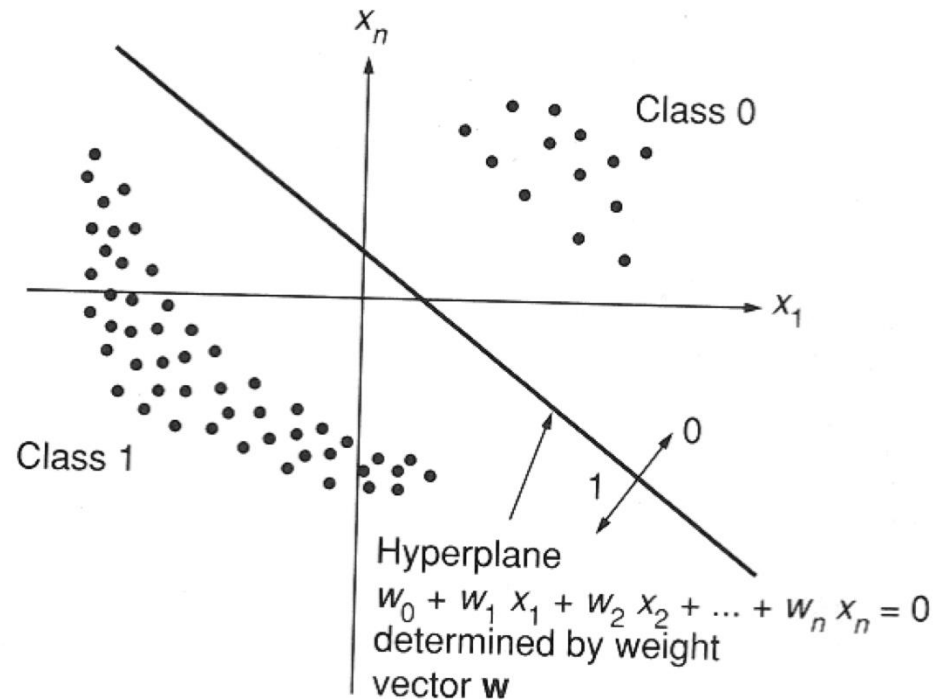


Exemple linéar separabile

$$o = \text{sgn}\left(\sum_{i=0}^n w_i x_i\right)$$

$$\sum_{i=0}^n w_i x_i > 0 \quad \text{for } C_0$$

$$\sum_{i=0}^n w_i x_i \leq 0 \quad \text{for } C_1$$



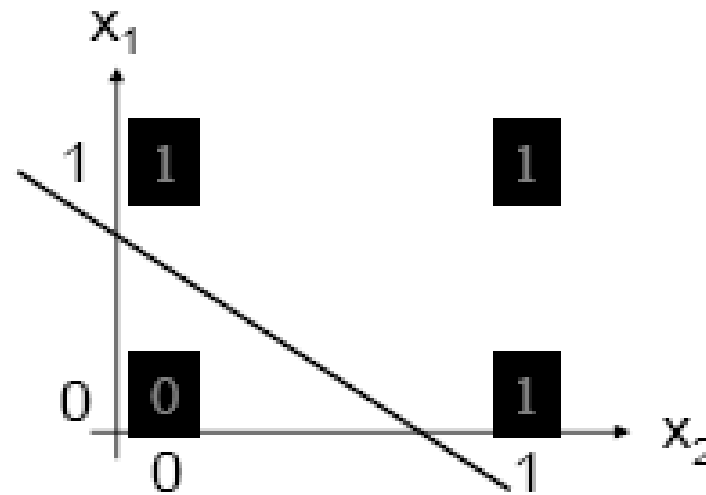
w_0 = Bias

Antrenare perceptron

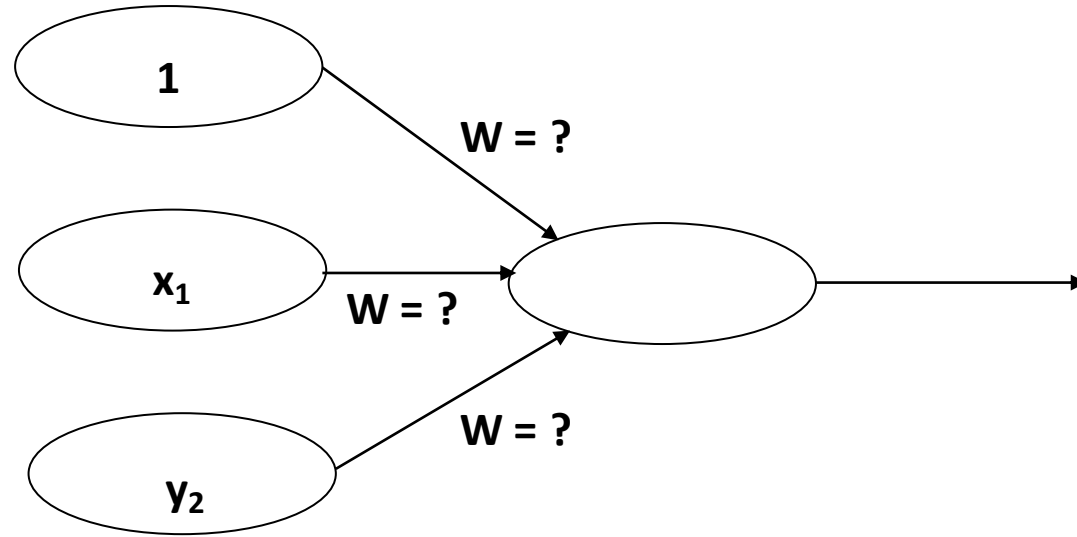
- Perceptron poate invata – ceea ce e separabil printr-un hiperplan

OR function

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

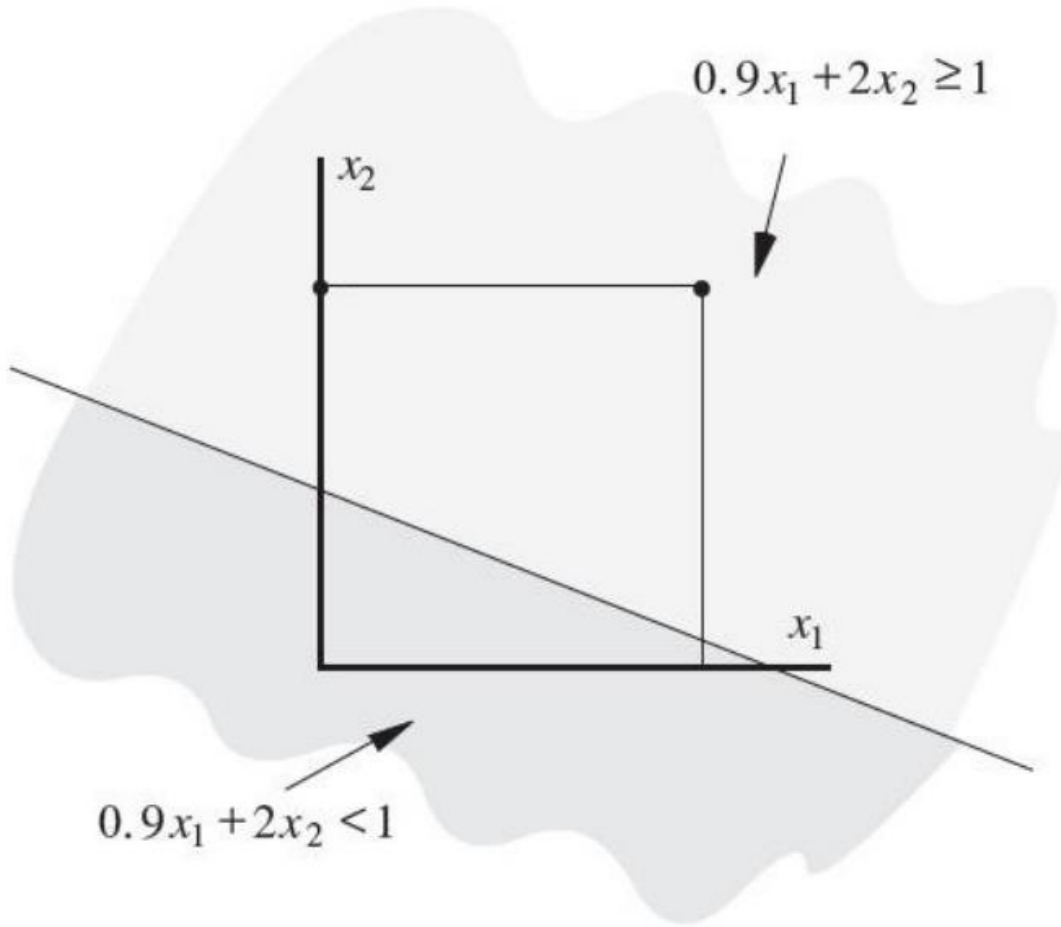


Antrenare Perceptron



•Ponderi ?

Exemplu



$$w_1=0.9$$

$$w_2=2$$

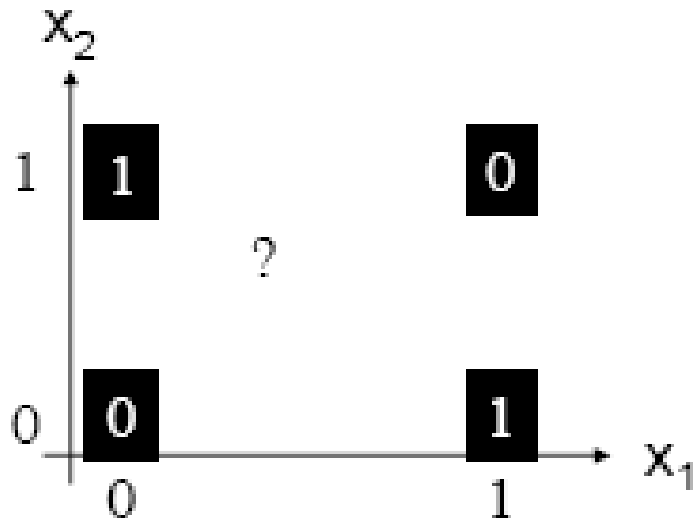
$$\text{Bias}=-1$$

Exclusive OR

Exclusive OR nu este liniar separabil -> perceptronul nu poate fi utilizat pentru antrenare

XOR function

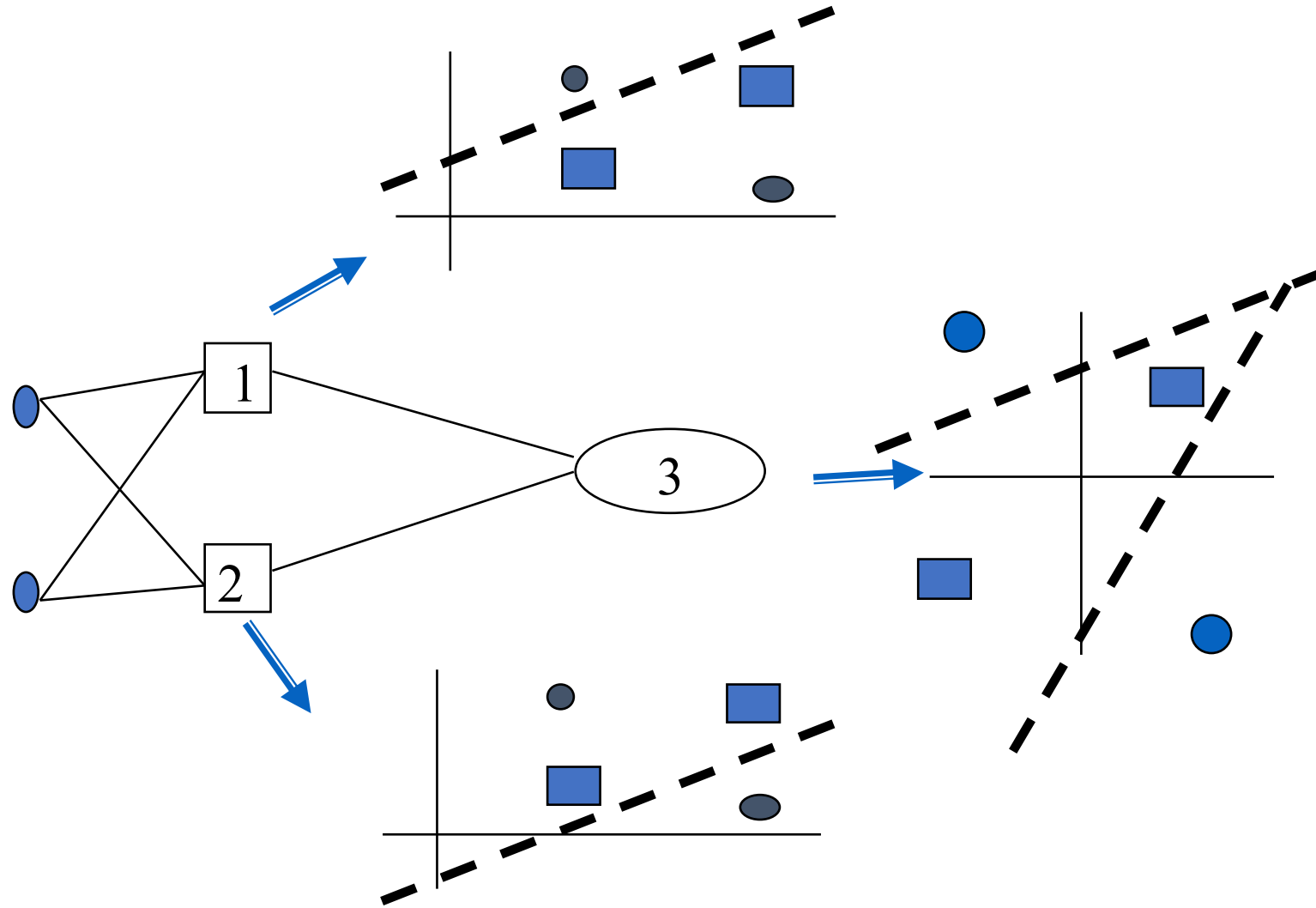
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



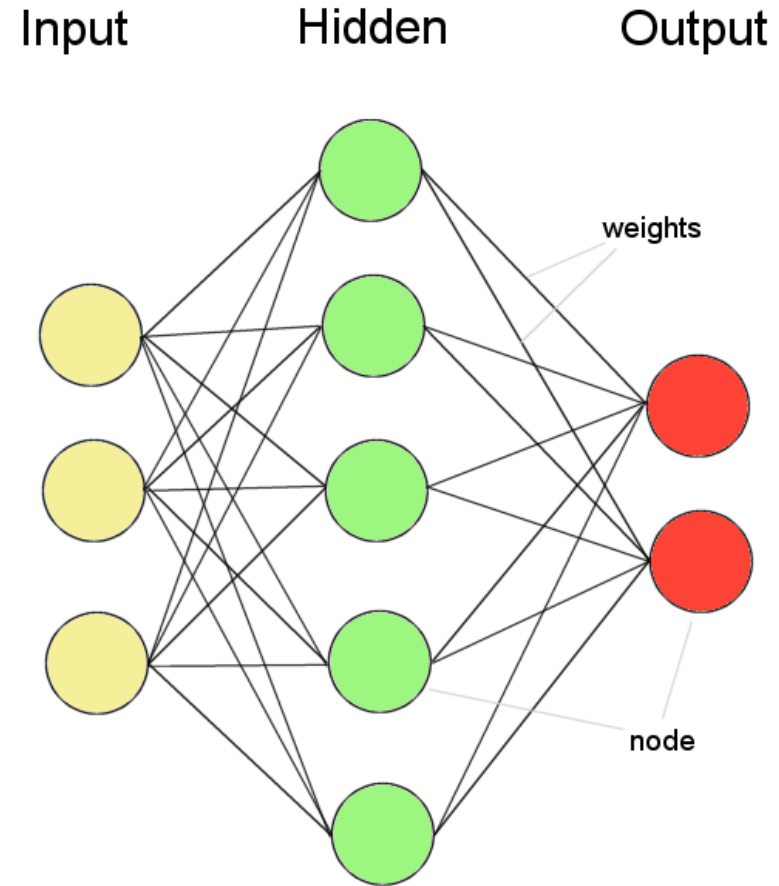
Multi Layer Perceptron (MLP)

- Minsky & Papert (1969) a rezolvat problema XOR prin combinarea perceptronilor
- Multi Layer Perceptron (MLP)

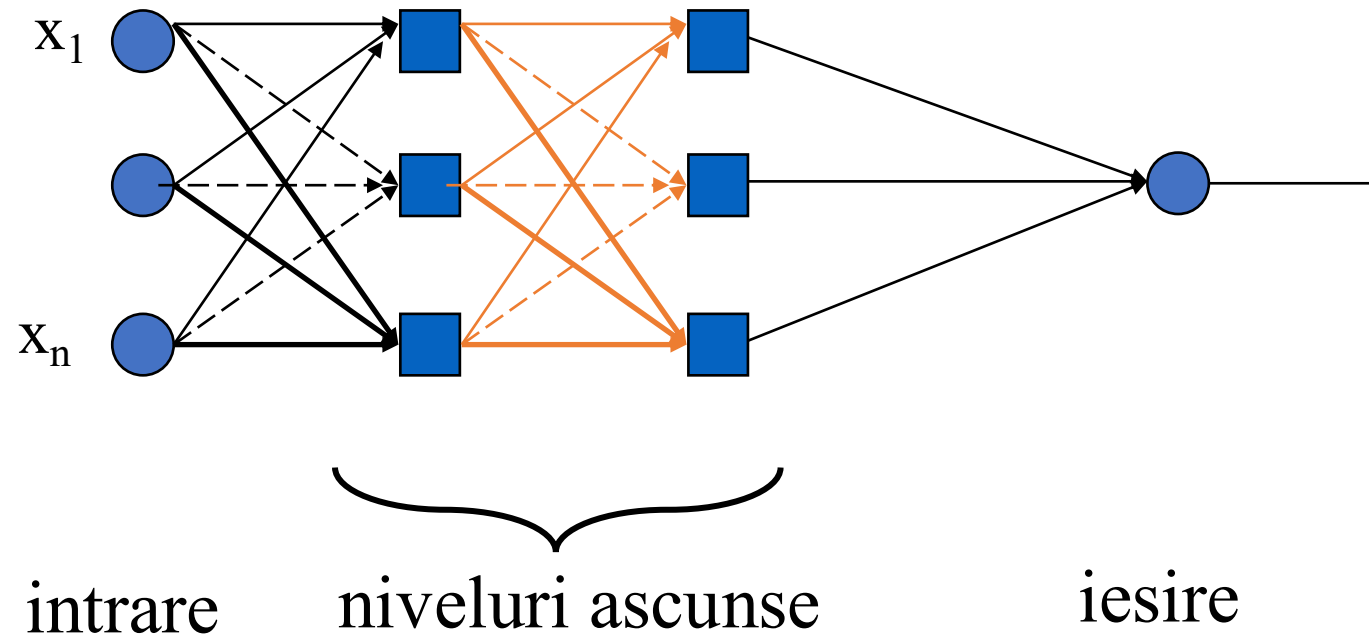
Multi Layer Perceptron (MLP)



Multi Layer Perceptron (MLP)

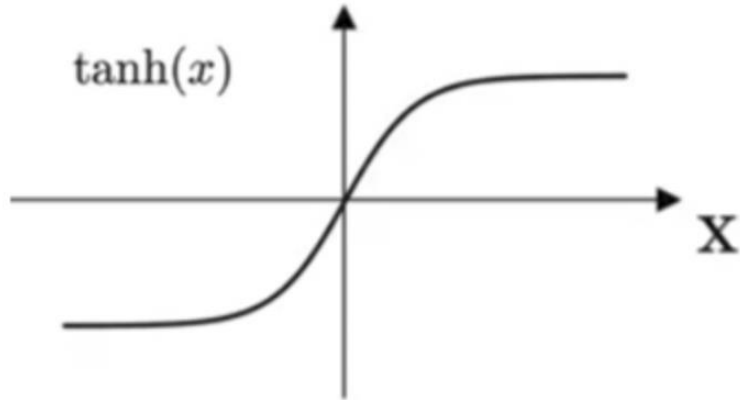


Multi Layer Perceptron (MLP) – fully connected

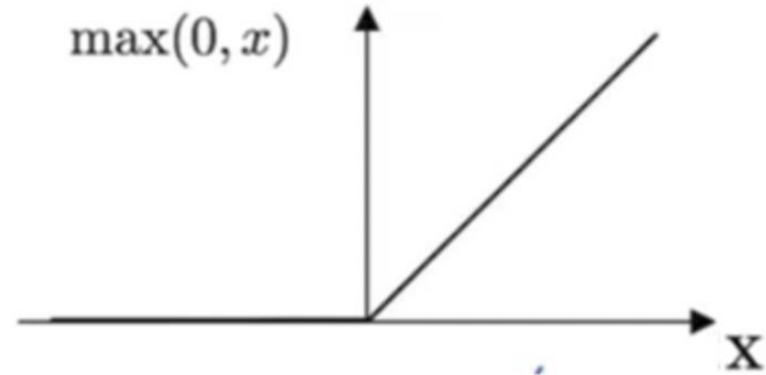


Functii de activare

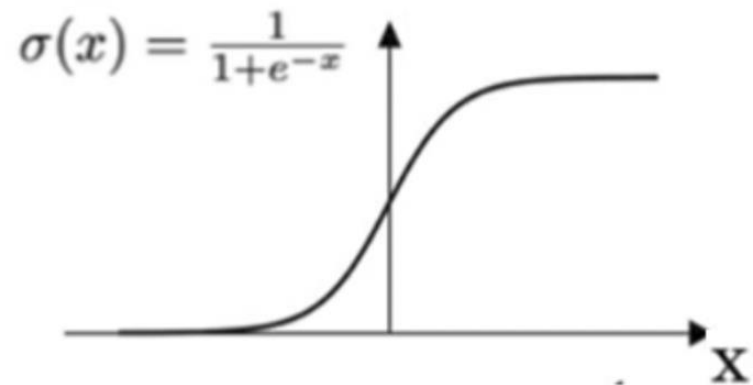
Hyper Tangent Function



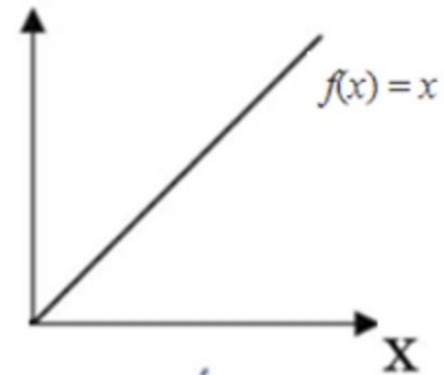
ReLU Function



Sigmoid Function

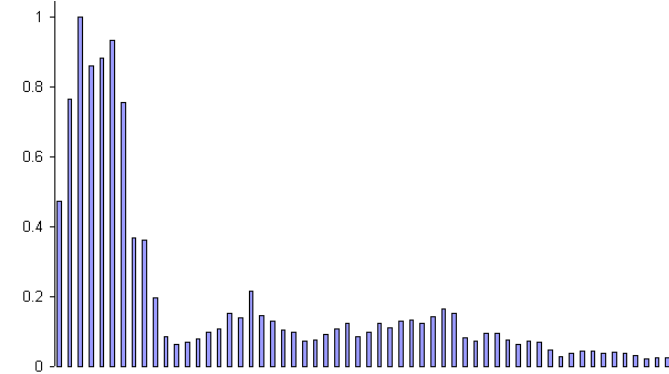


Identity Function



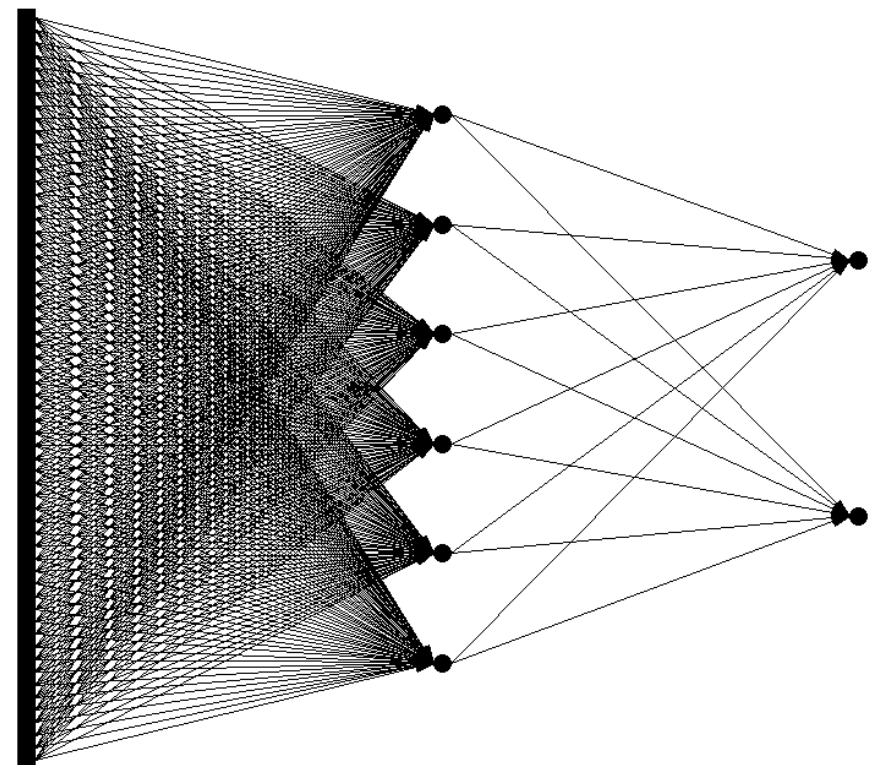
Exemplu: Recunoastere voce

- Diferentierea intre doua voci
- Data
 - Intrari
 - Steve
 - David
 - Format
 - Distributie de frecventa (60 bins)

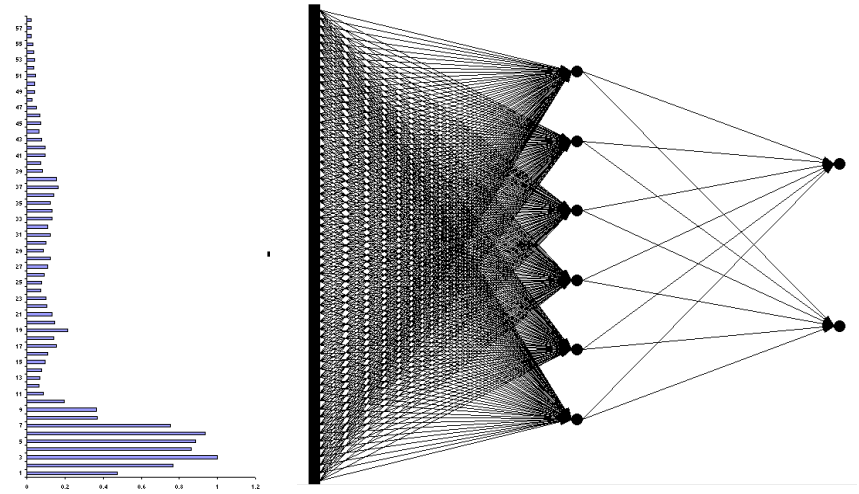


- Arhitectura retea

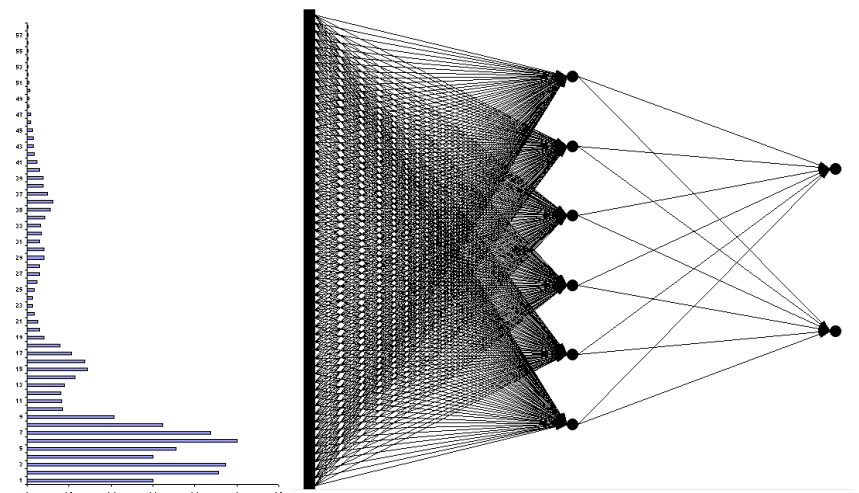
- 60 intrari (una pentru fiecare frecventa)
- 6 noduri ascunse
- 2 iesiri (0-1 - “Steve”, 1-0 - “David”)



Steve

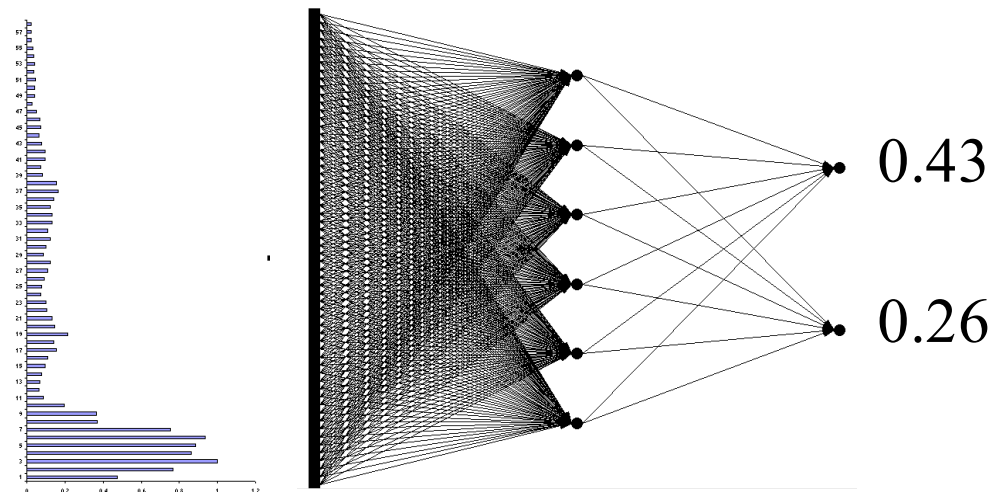


David

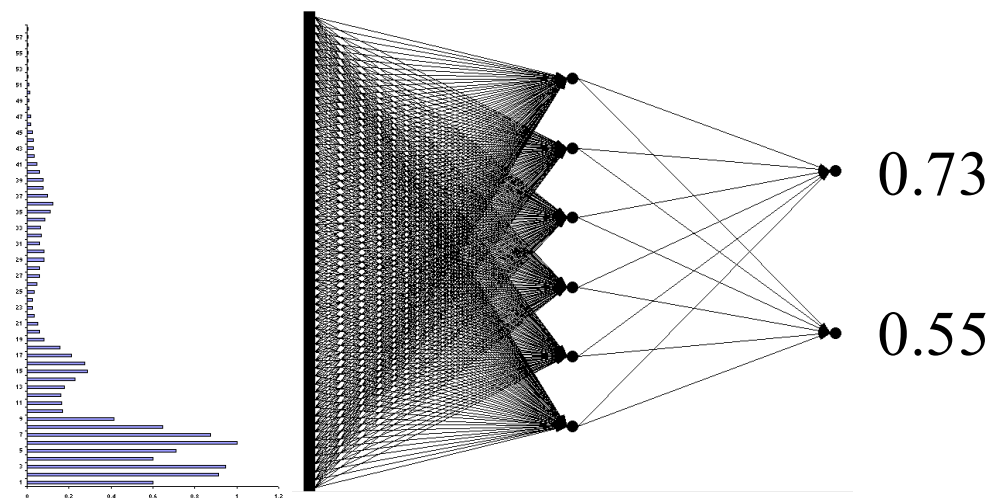


Retea neantrenata

Steve

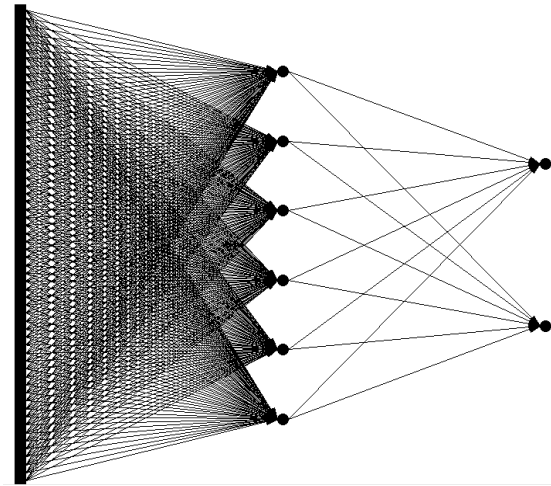
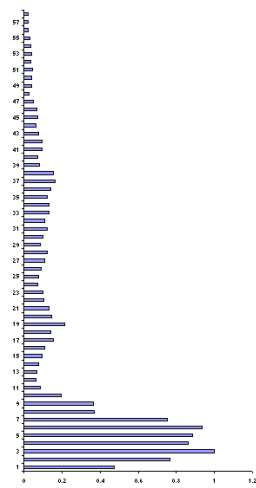


David



Se calculeaza eroarea

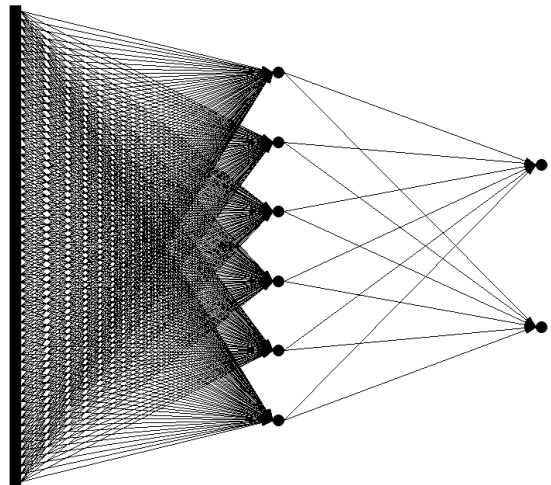
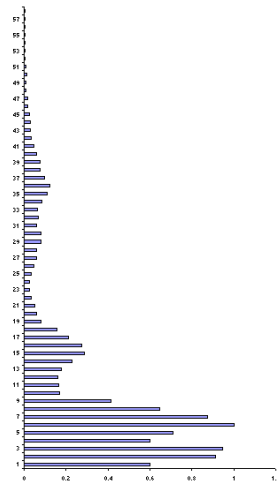
Steve



$$0.43 - 0 = 0.43$$

$$0.26 - 1 = 0.74$$

David

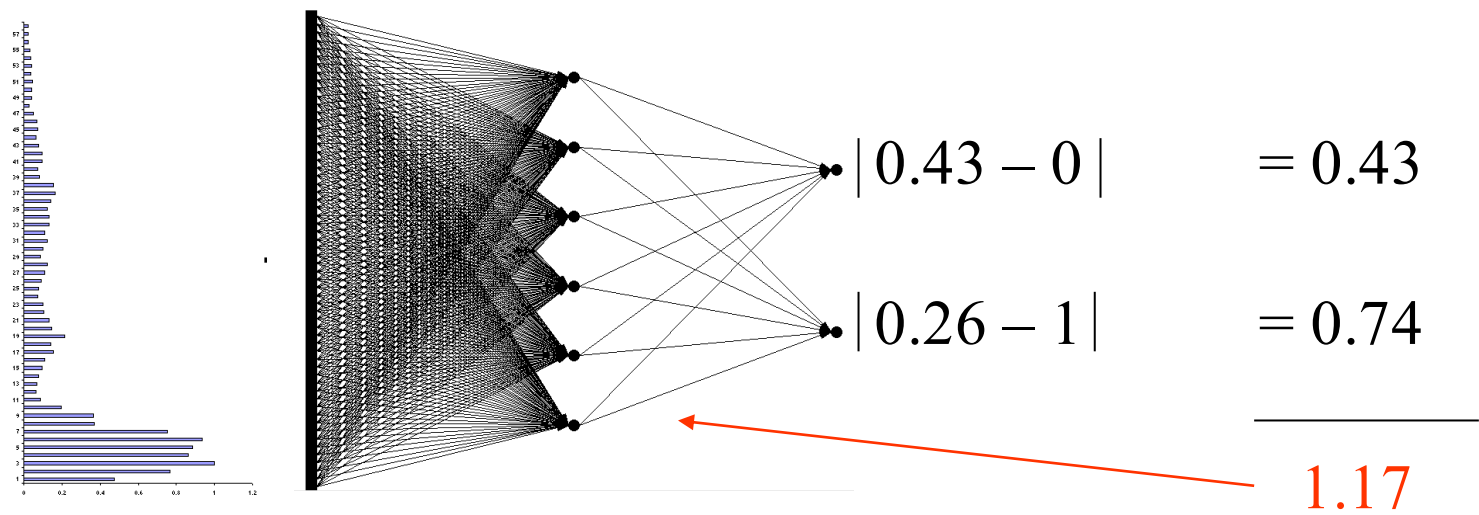


$$0.73 - 1 = 0.27$$

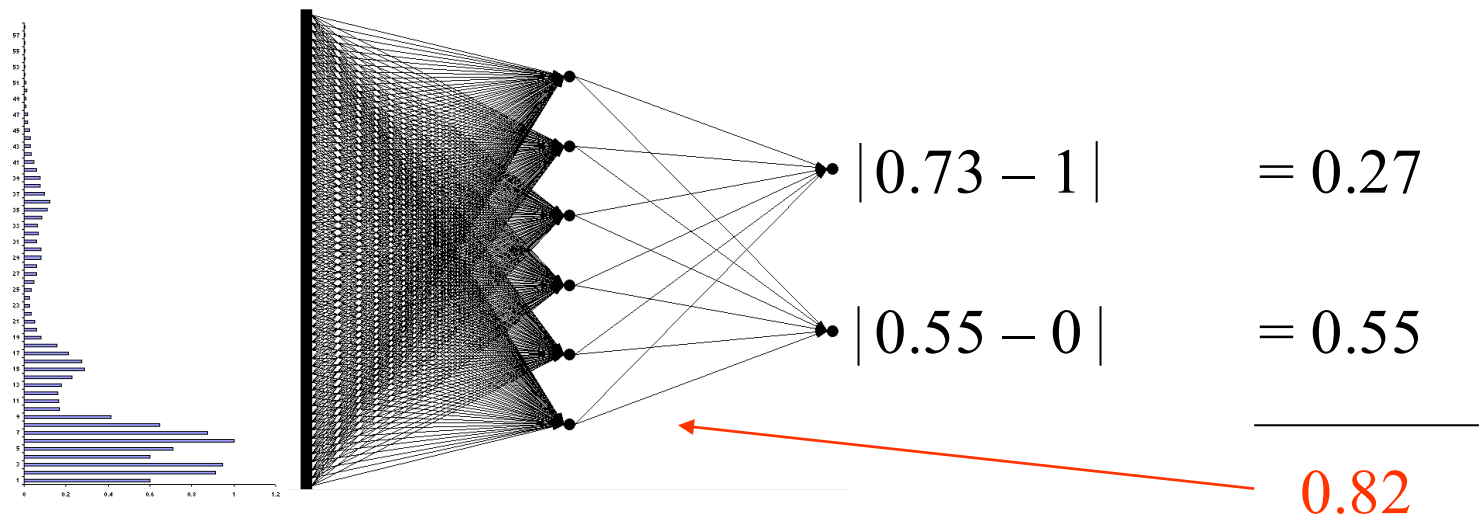
$$0.55 - 0 = 0.55$$

Propagare erori spre intrare si actualizare ponderi

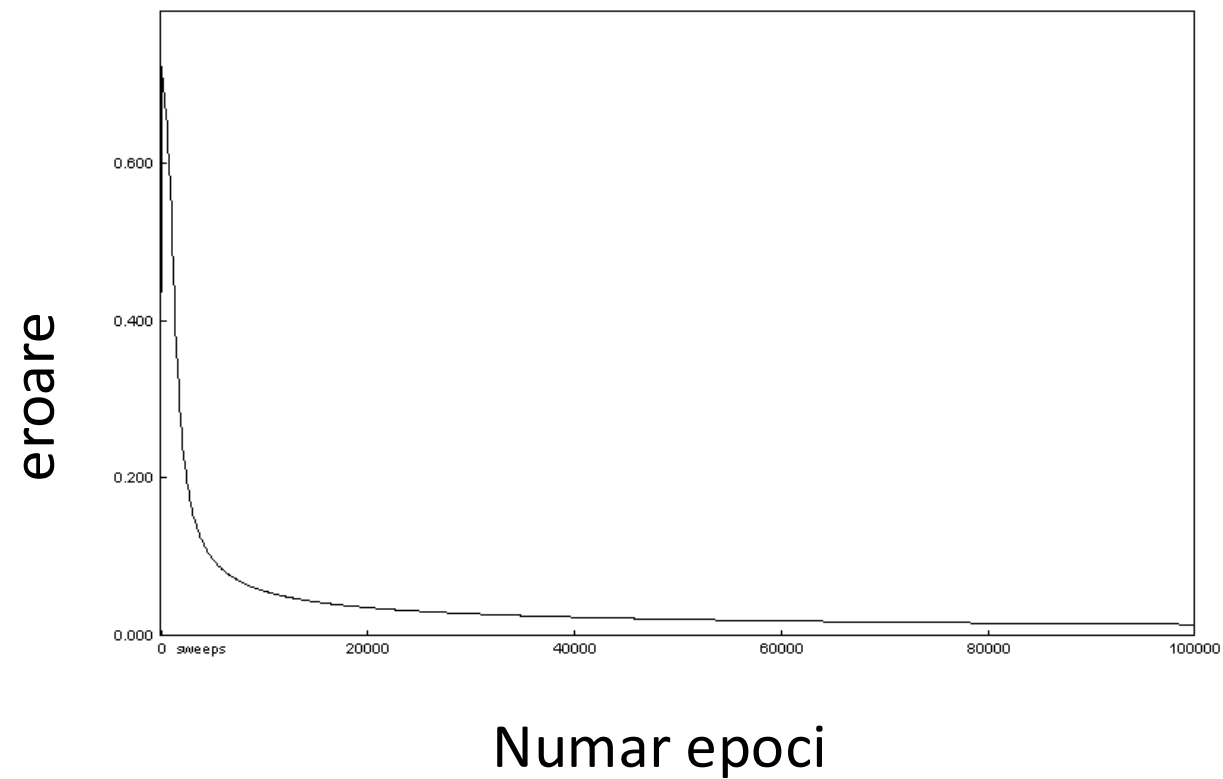
Steve



David

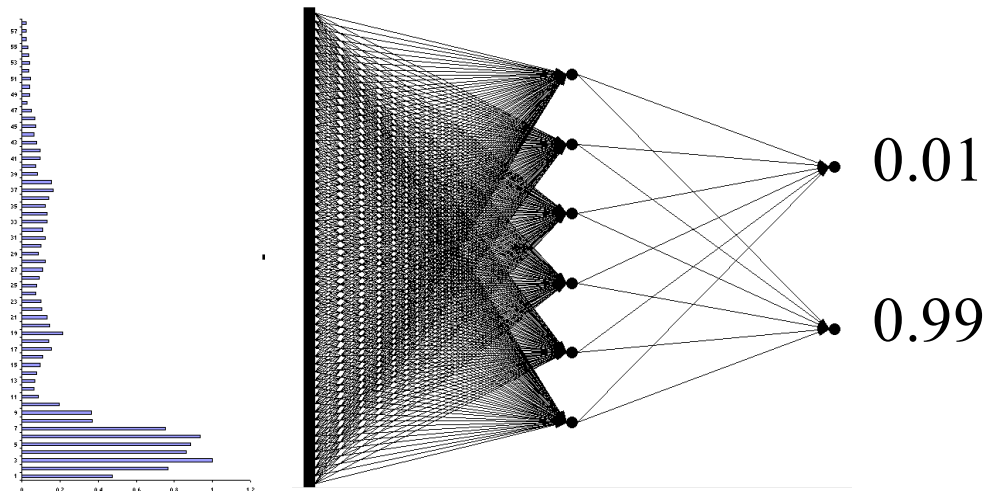


- Se repeta procesul pentru fiecare epoca – fiecare exemplu de antrenare

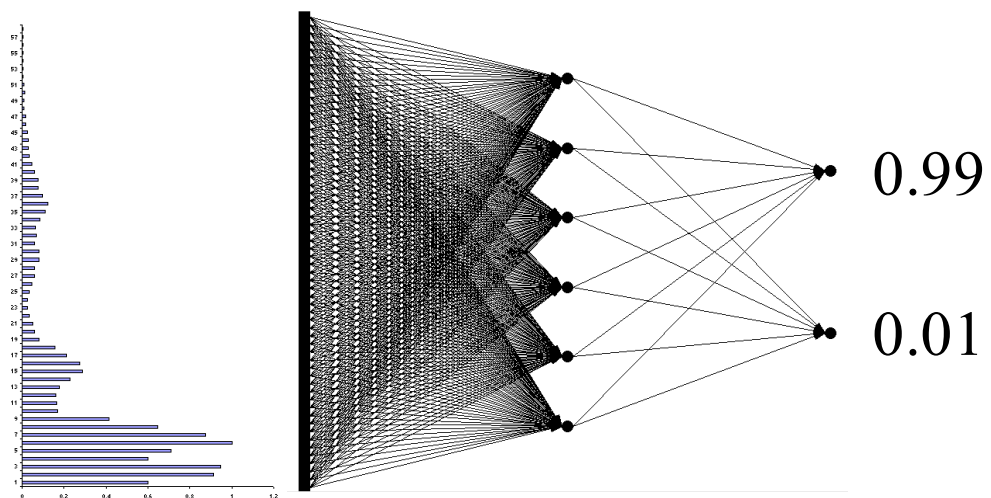


Retea antrenata

Steve

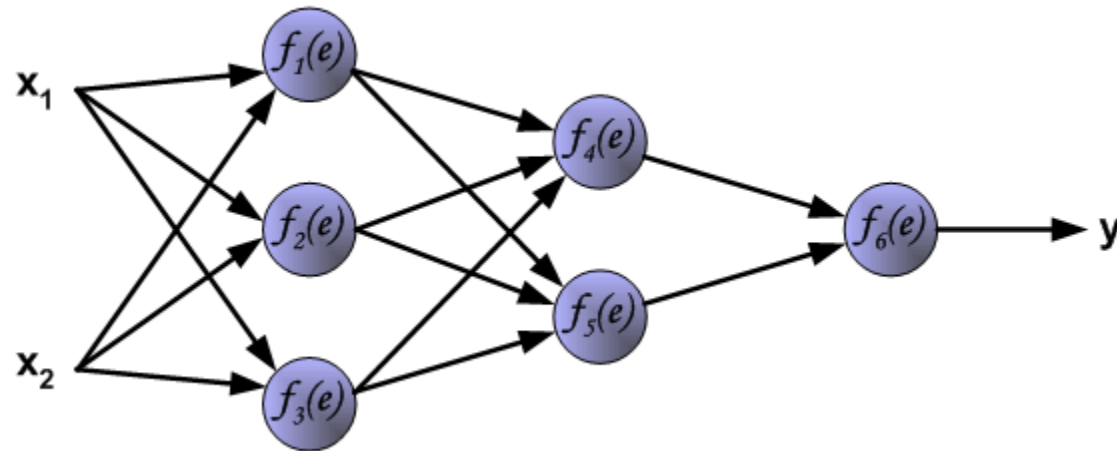


David

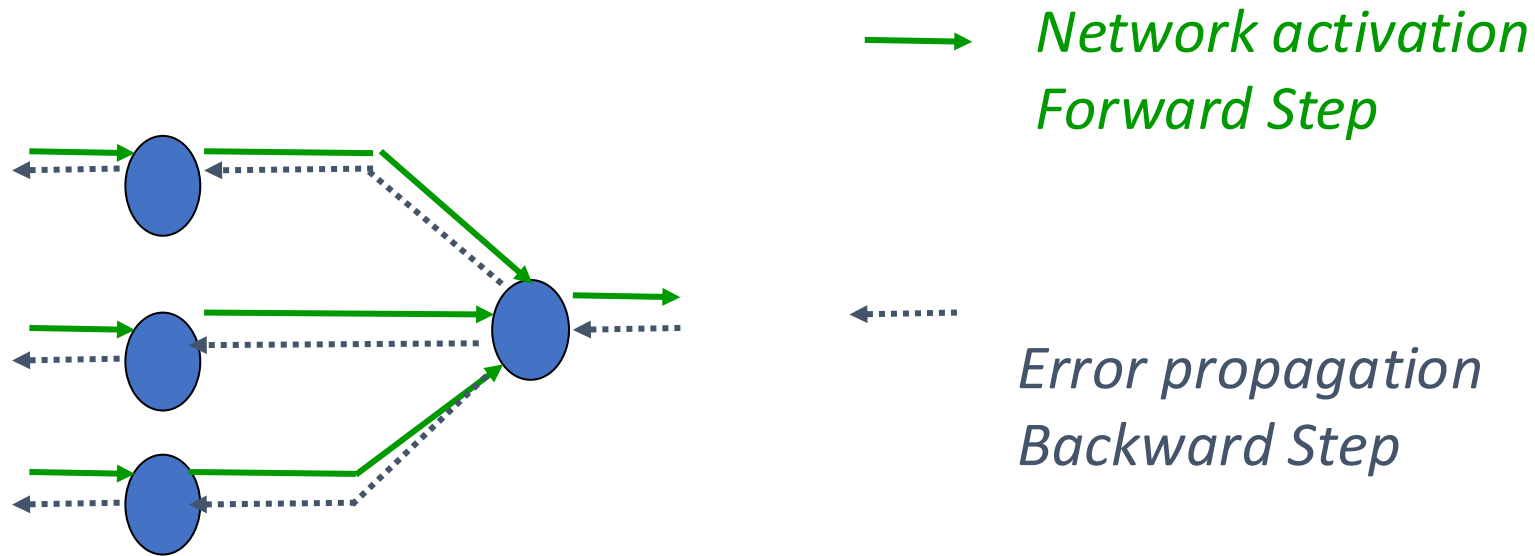


Antrenare MLP: Backpropagation

Exemplu: ilustrarea algoritmului backpropagation pe urmatoarea retea:



Antrenare: Backpropagation



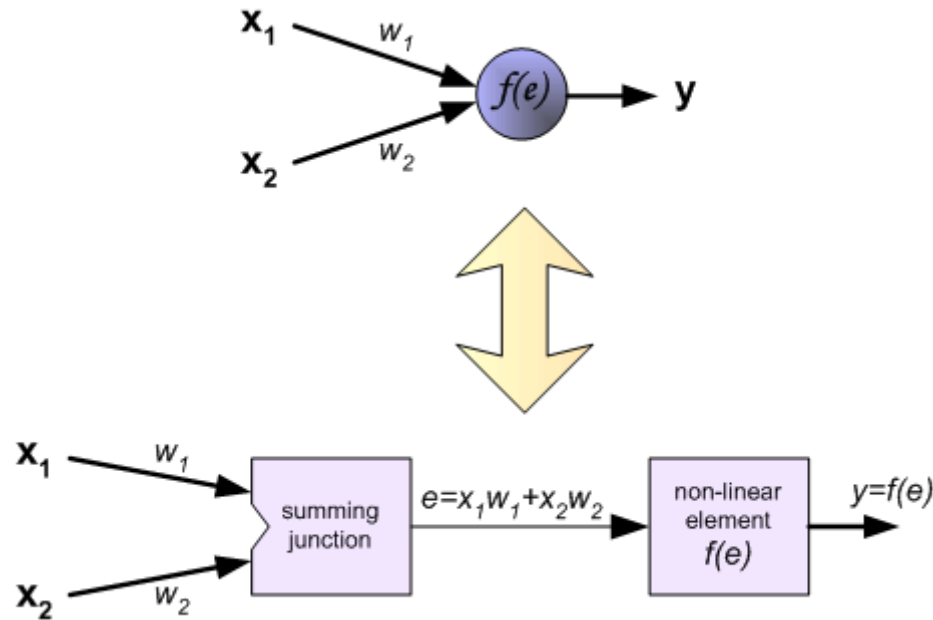
- Se actualizeaza ponderile pentru a minimiza eroarea.

Antrenare: Backpropagation

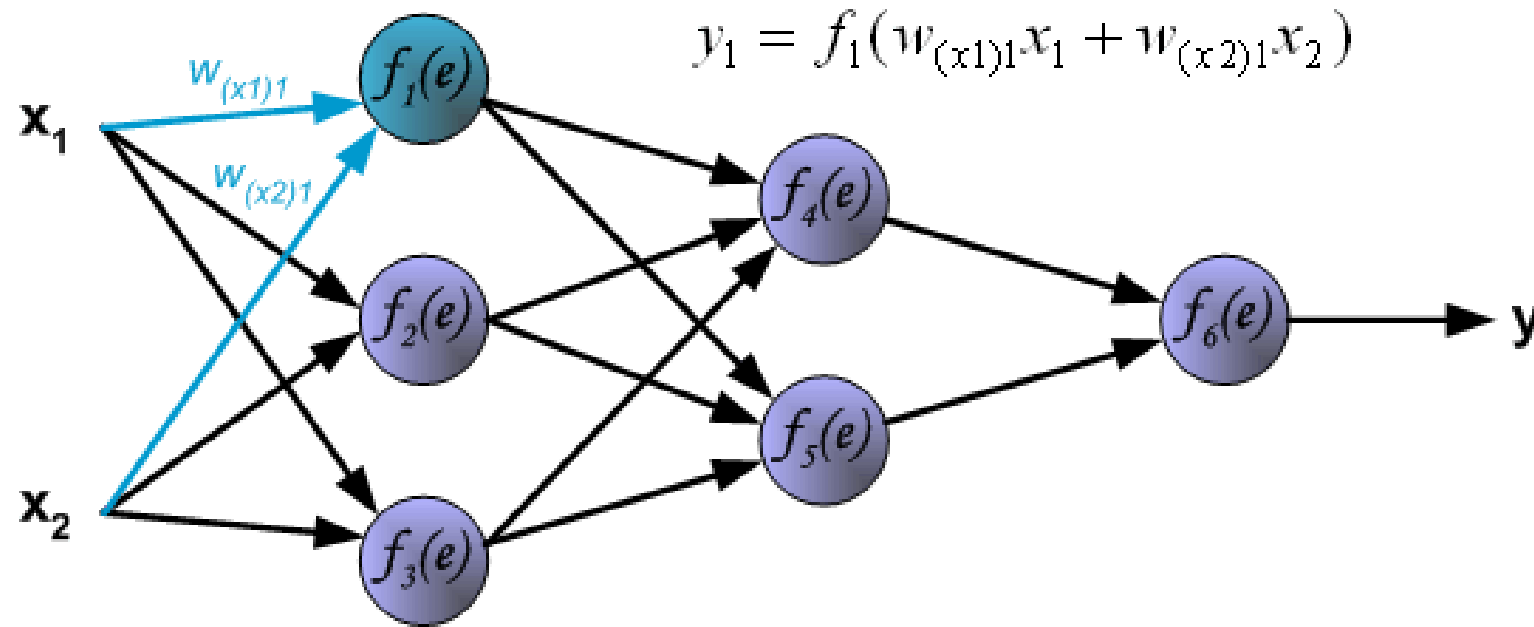
- 1: initializare aleatoare a ponderilor, setare viteza de invatare η
- pentru fiecare epoca
 - pentru fiecare exemplu de antrenare:
 - 2: forward: obtine iesirea retelei
 - 3: Backward: propaga eroarea (se calculeaza eroarea pentru fiecare neuron: δ_k)
 - 4: actualizeaza ponderi (stochastic gradient descent)

Antrenare MLP: Backpropagation

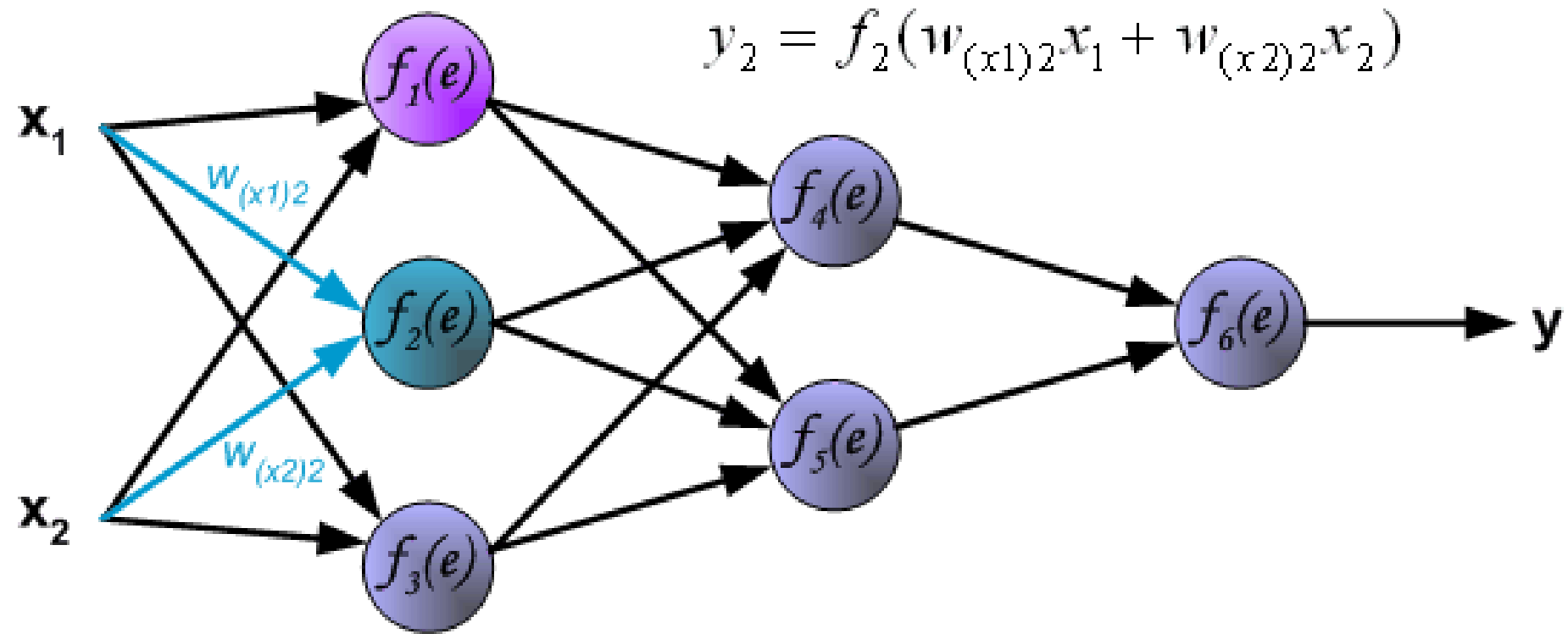
Fiecare neuron are doua intrari si functia de activare f (nu este exemplificata valoarea de bias).



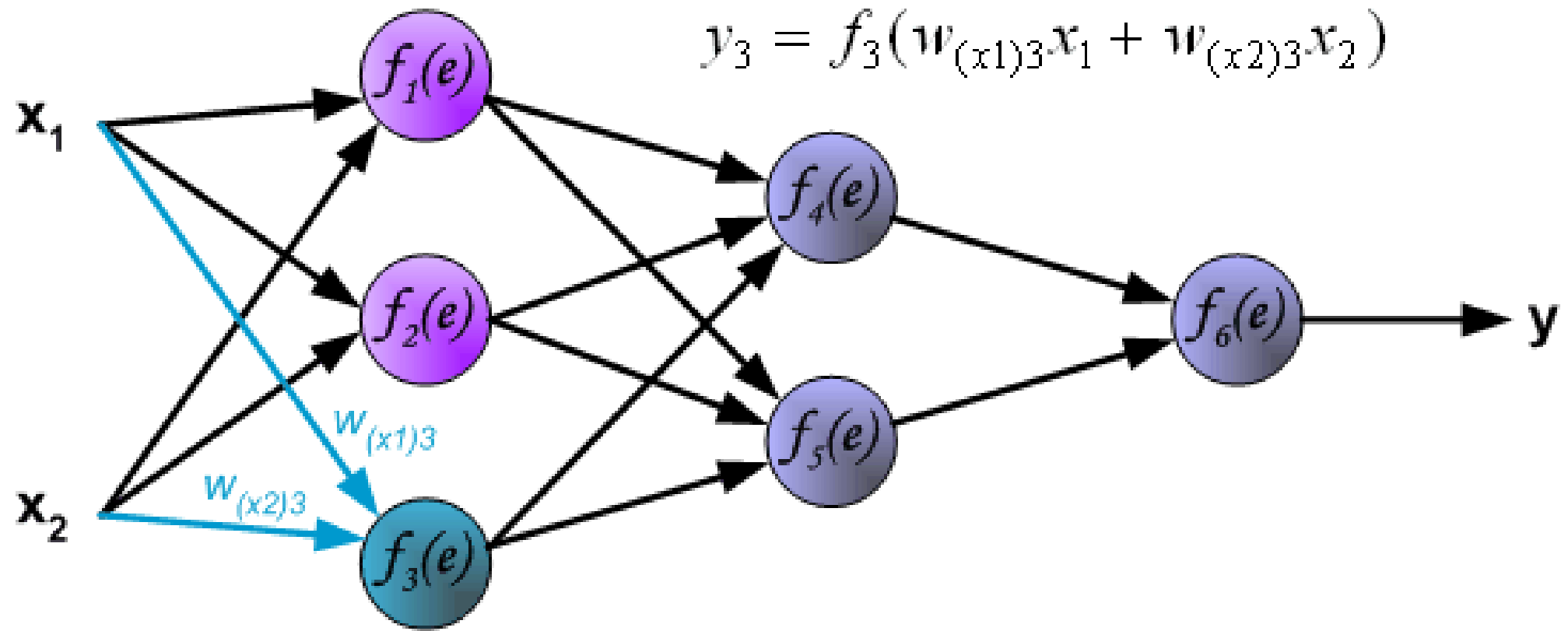
Exemplu: Backpropagation



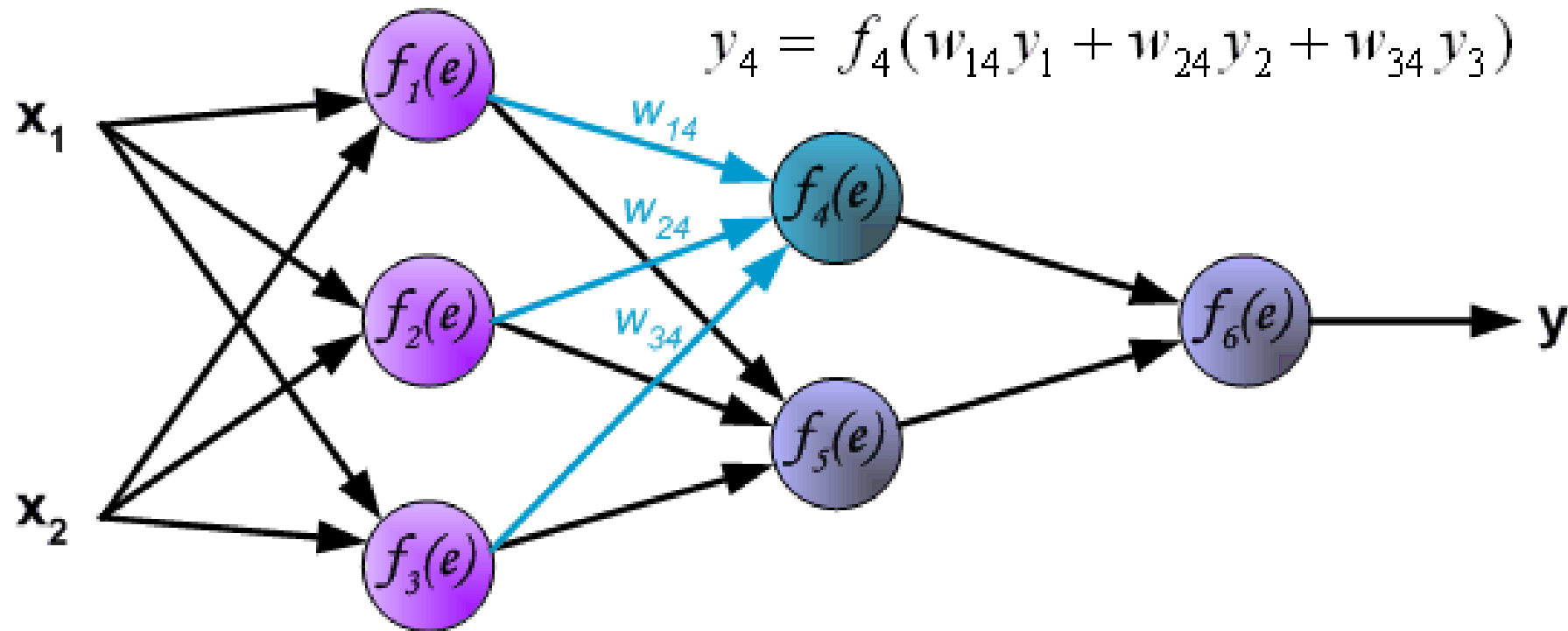
Exemplu: Backpropagation



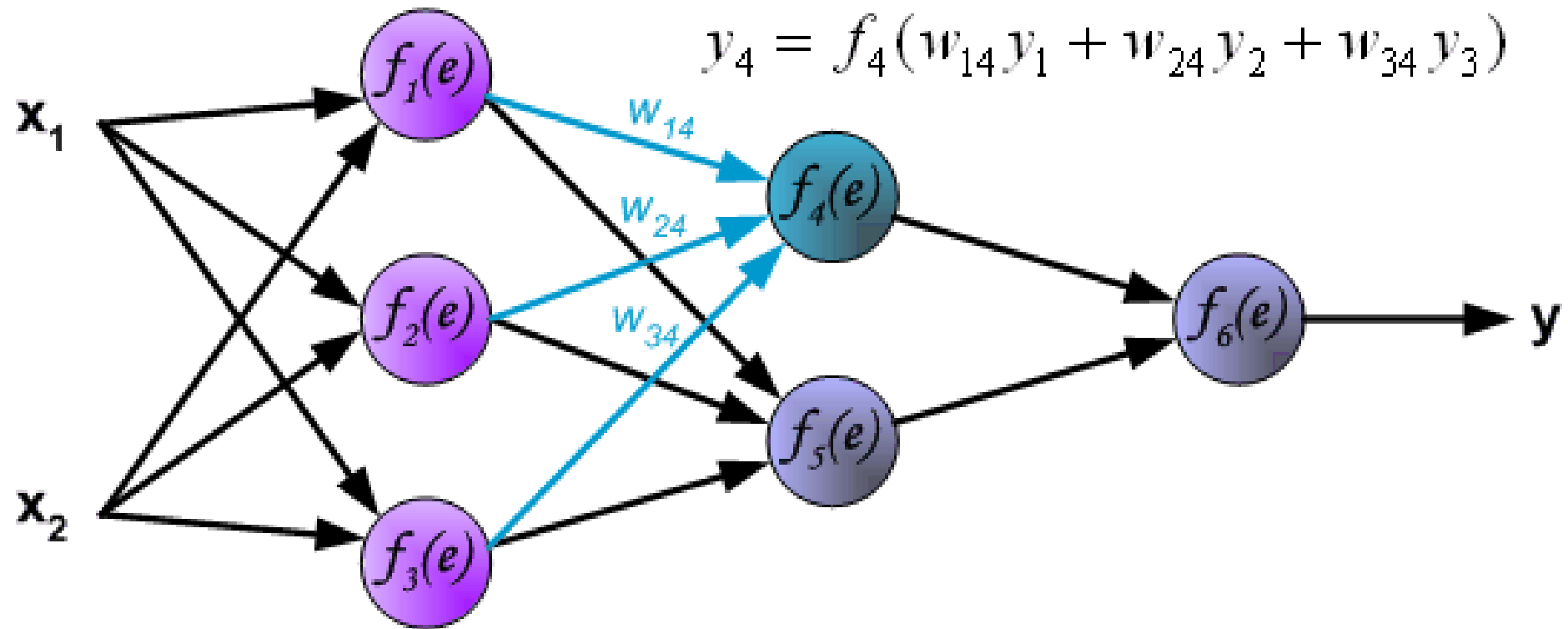
Exemplu: Backpropagation



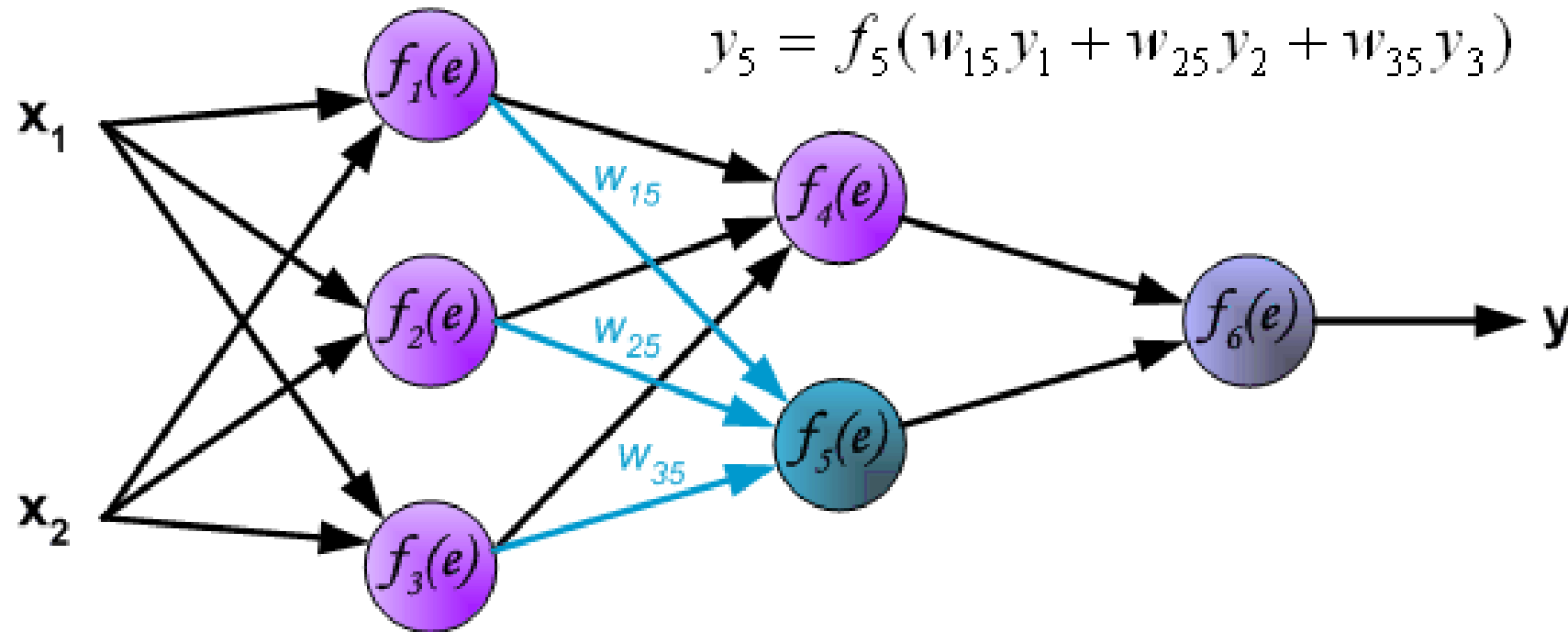
Exemplu: Backpropagation



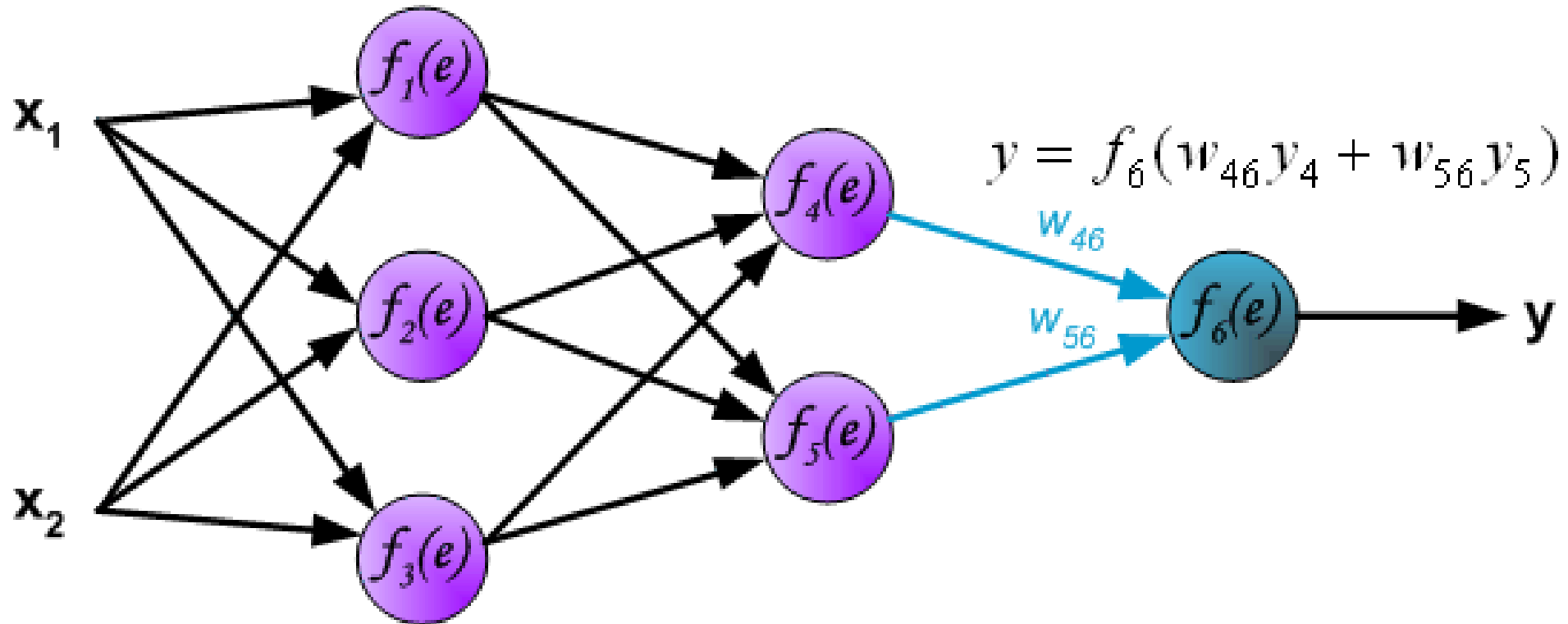
Exemplu: Backpropagation



Exemplu: Backpropagation

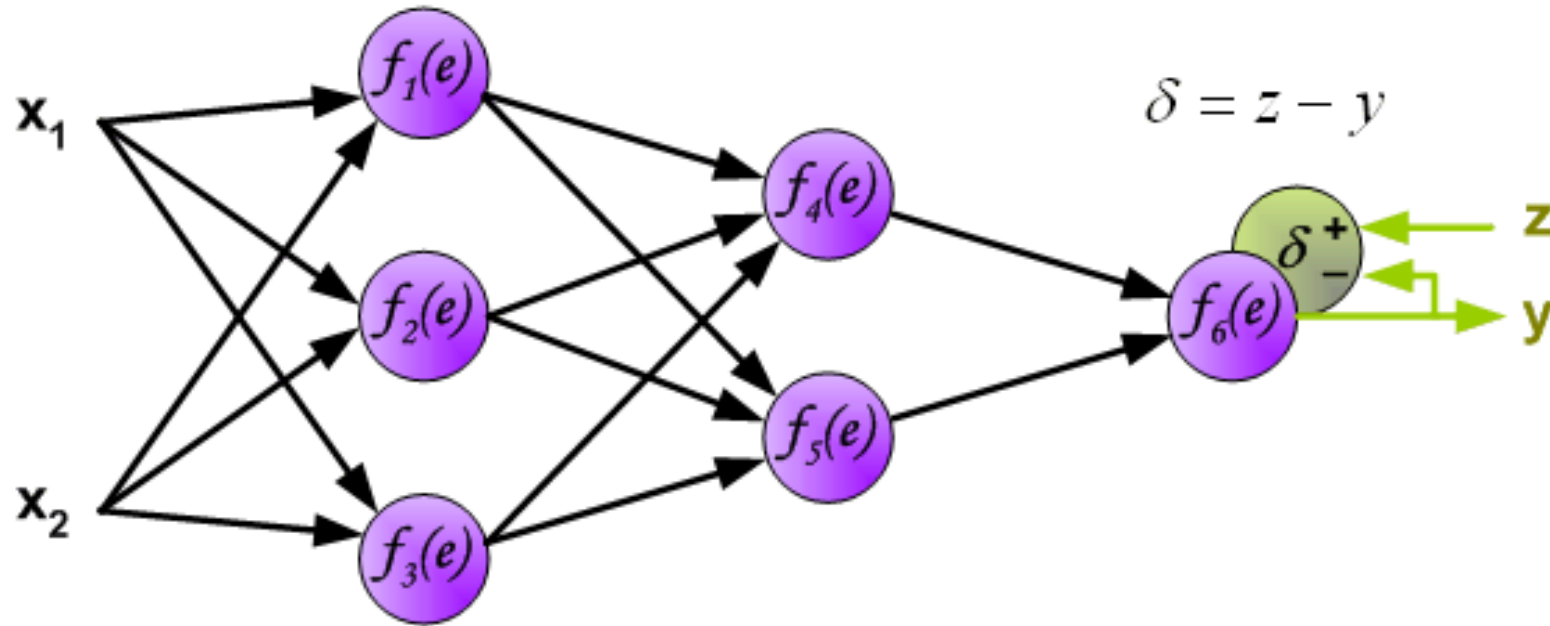


Exemplu: Backpropagation



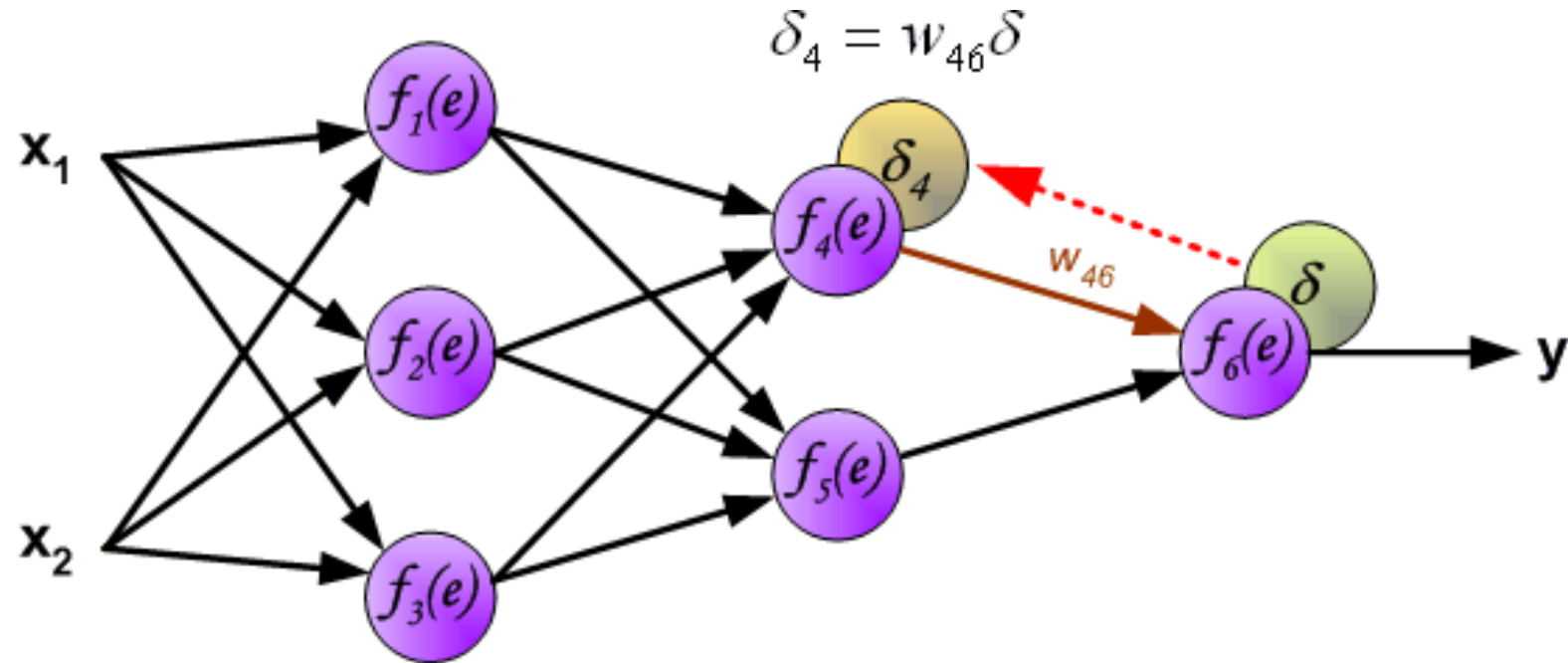
Exemplu: Backpropagation

Se calculeaza eroarea (δ) pe baza iesirii rețelei (y) si a valorii dorite (z)

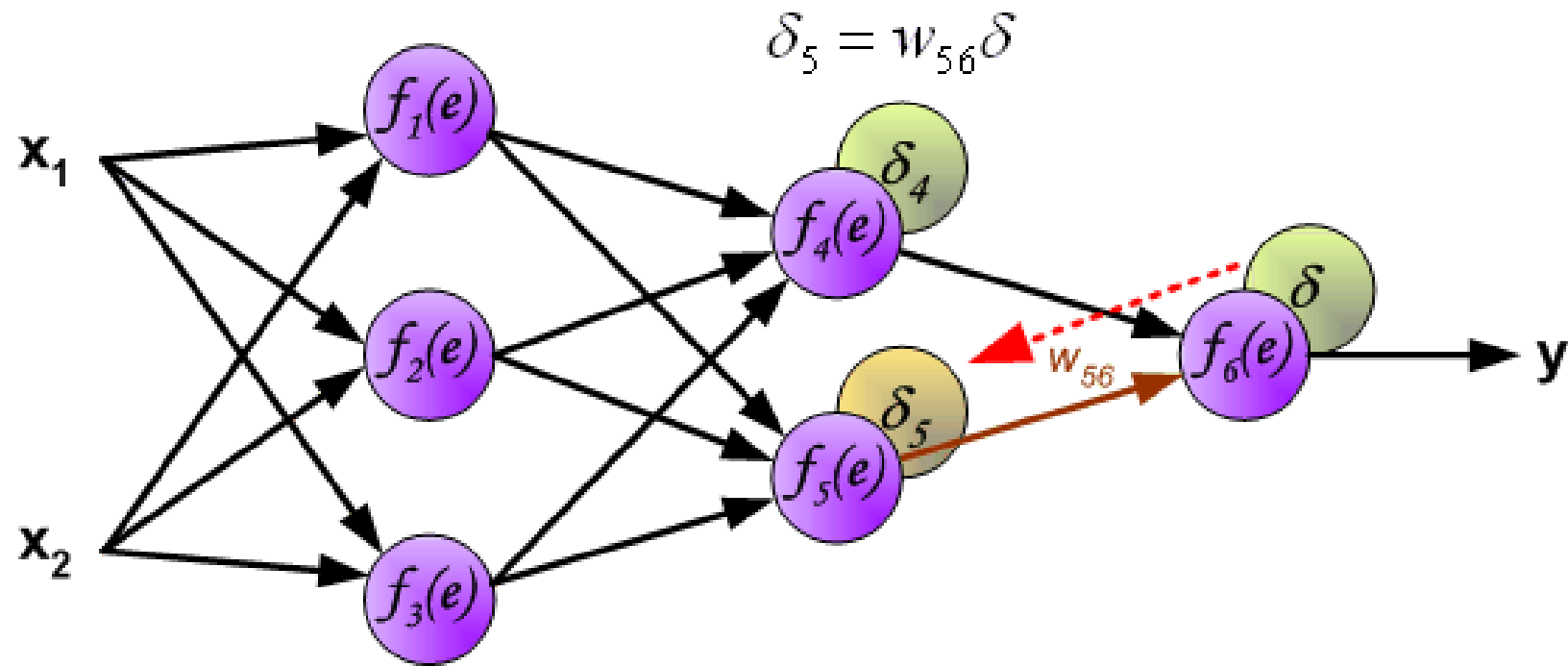


Exemplu: Backpropagation

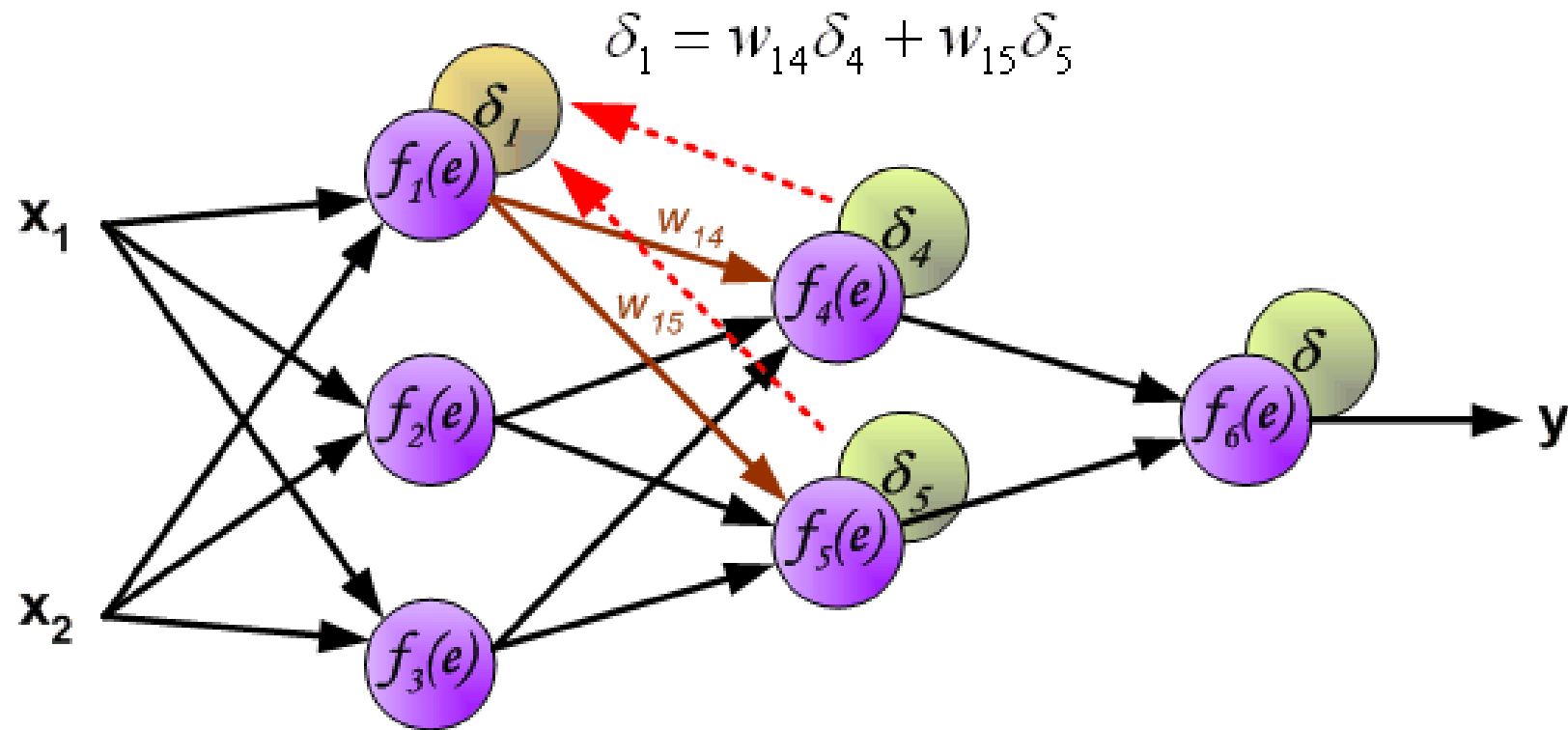
Propagare eroare δ spre intrarile rețelei



Exemplu: Backpropagation

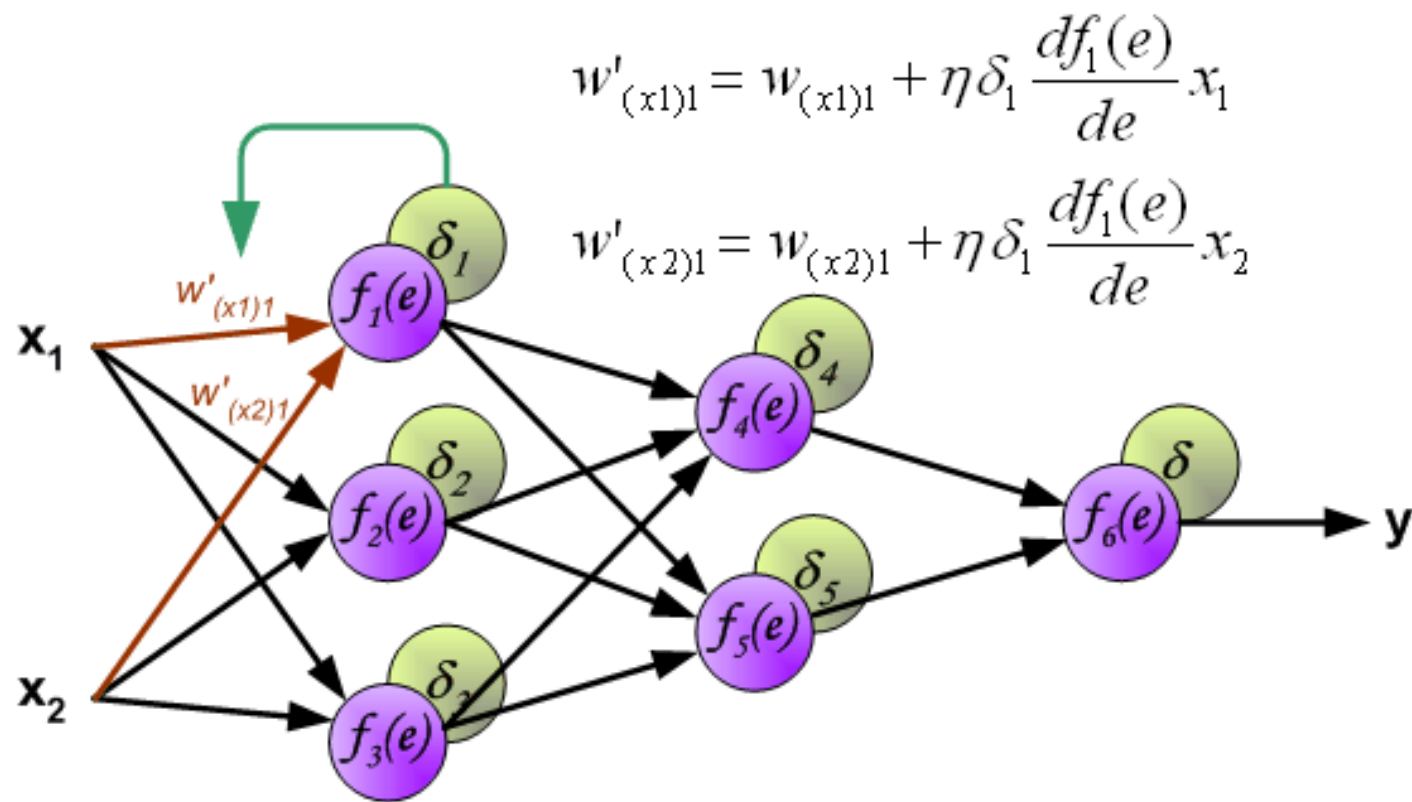


Exemplu Backpropagation



Exemplu: Backpropagation

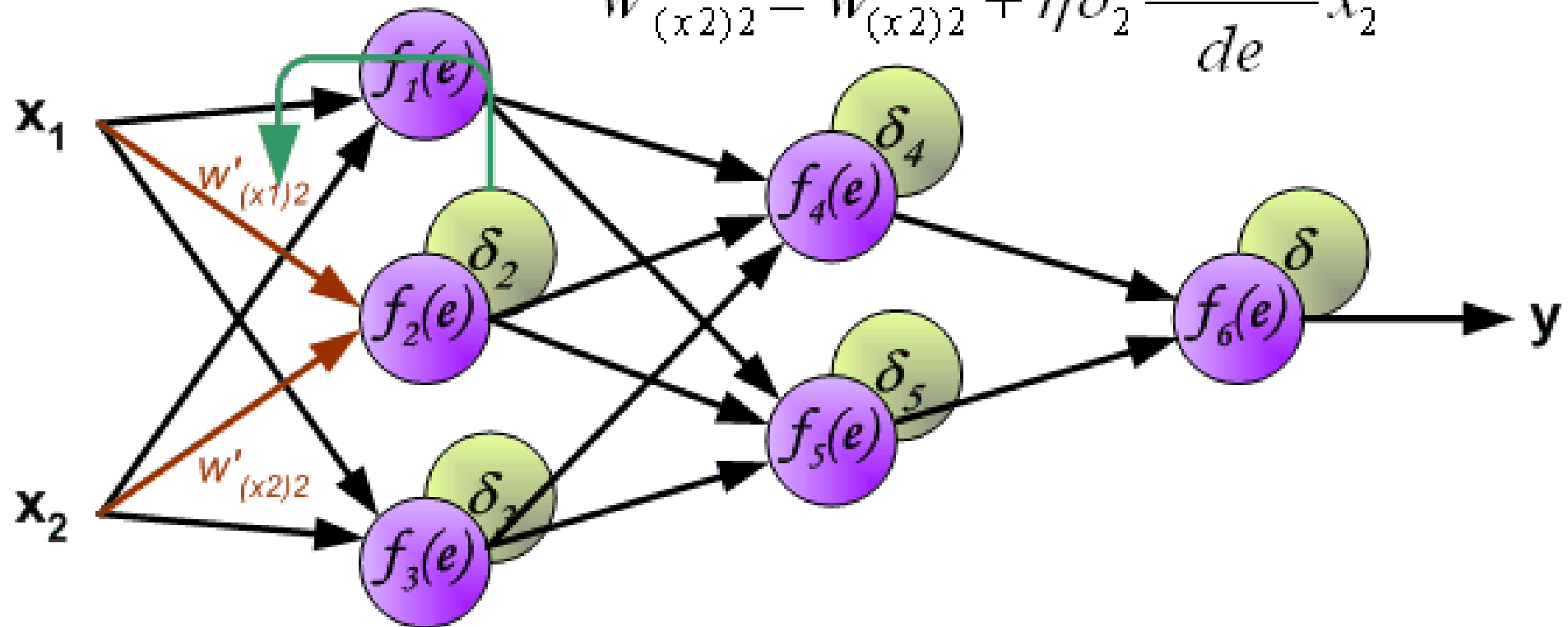
- Actualizare ponderi



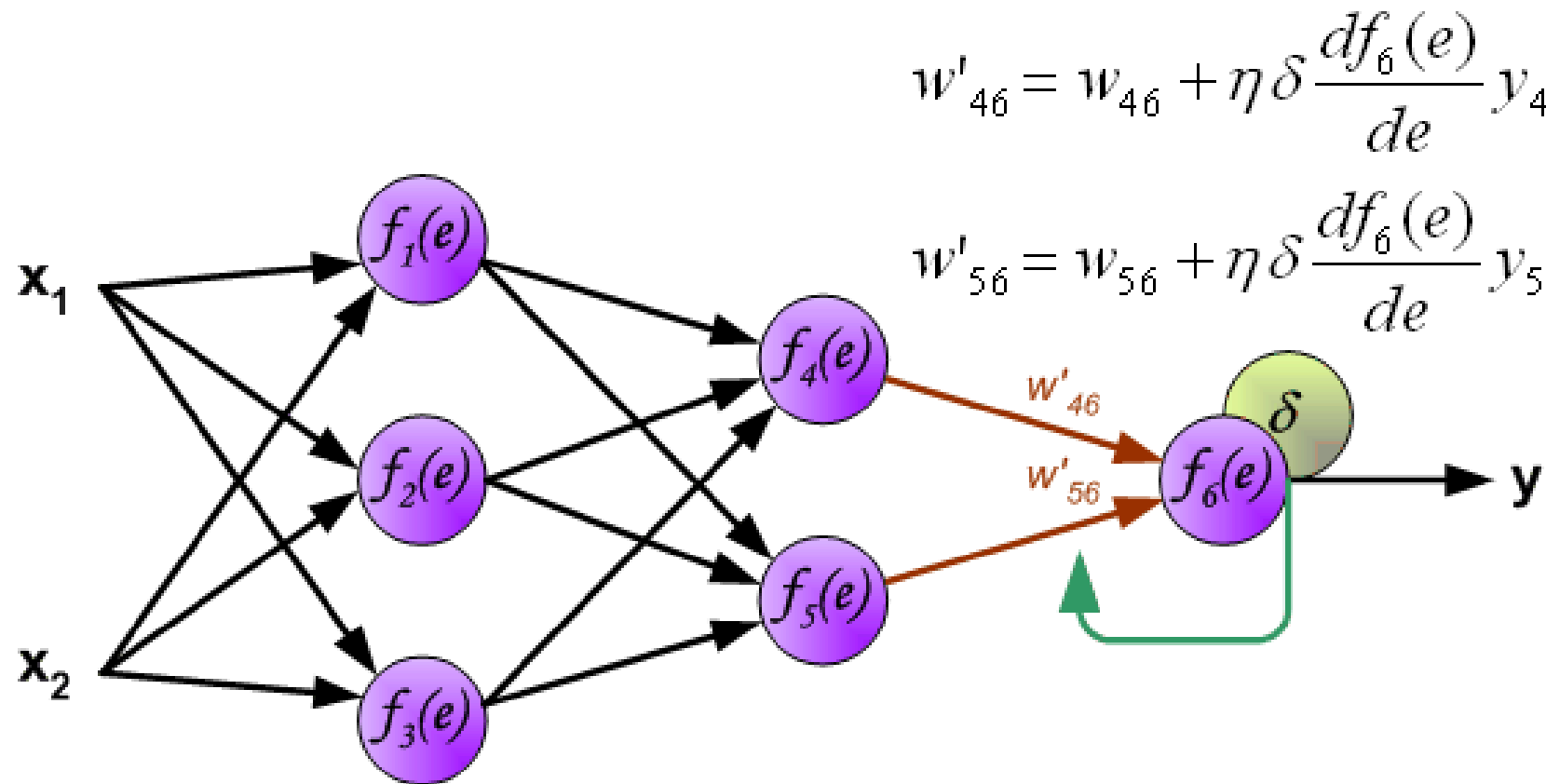
Exemplu: Backpropagation

$$w'_{(x1)2} = w_{(x1)2} + \eta \delta_2 \frac{df_2(e)}{de} x_1$$

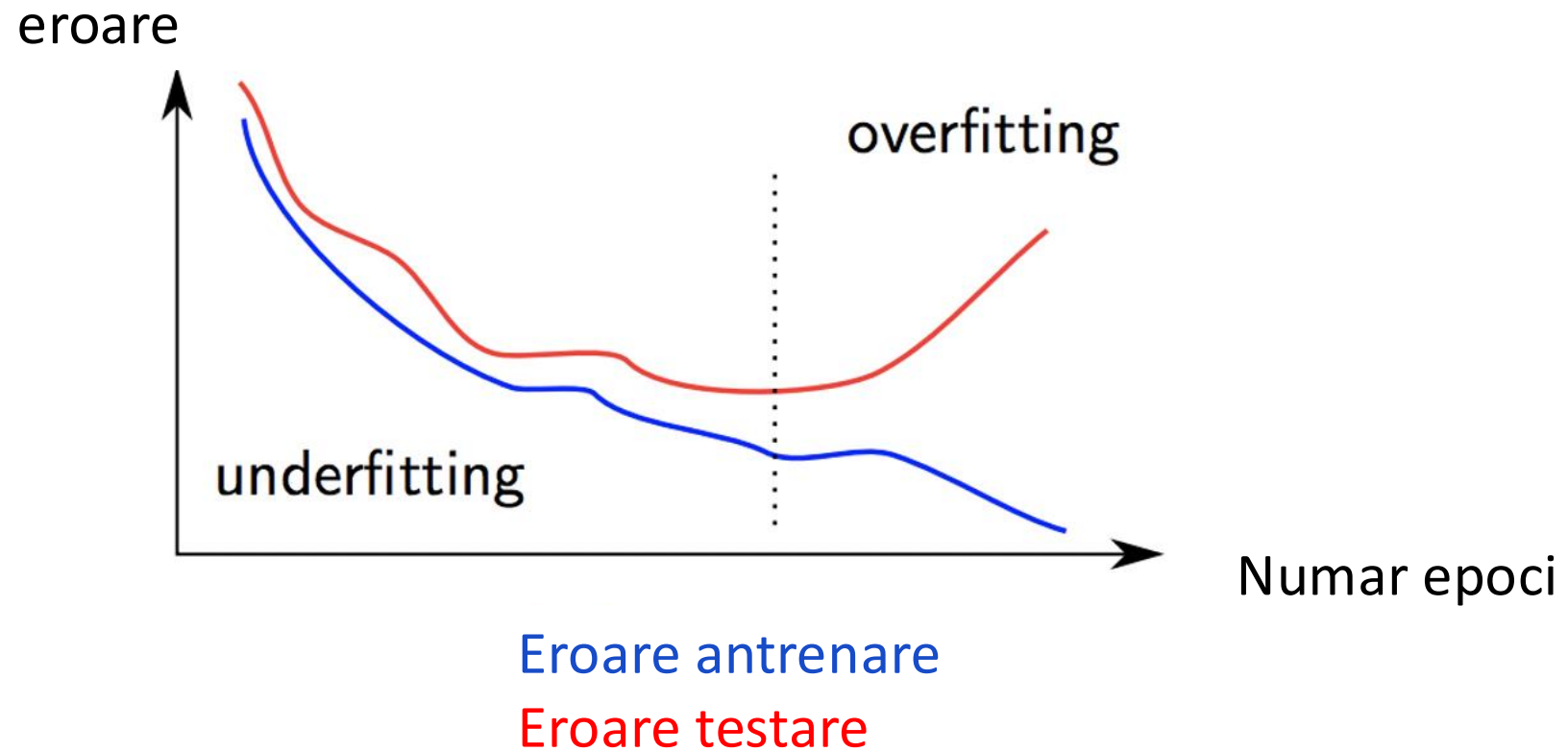
$$w'_{(x2)2} = w_{(x2)2} + \eta \delta_2 \frac{df_2(e)}{de} x_2$$



Exemplu: Backpropagation

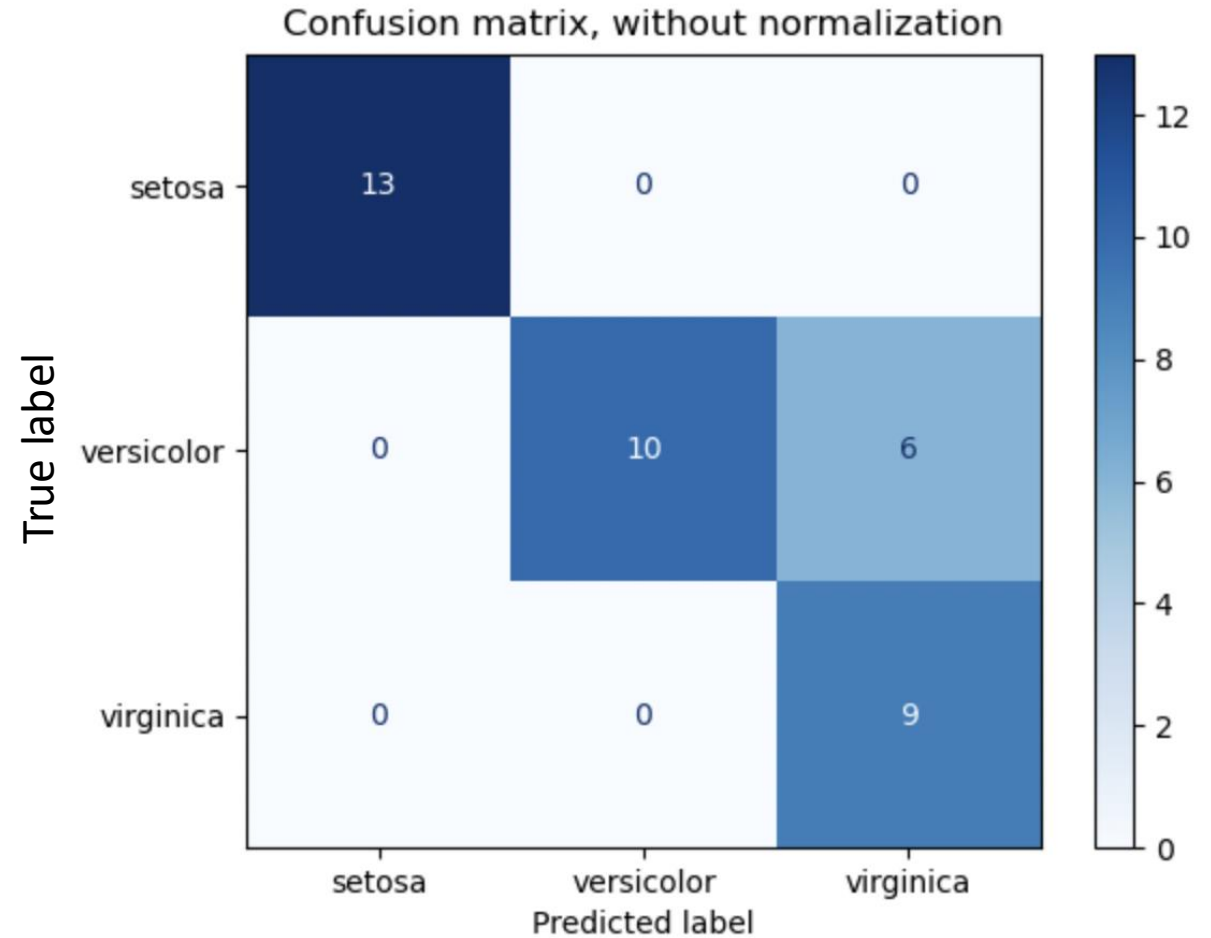


Evaluare



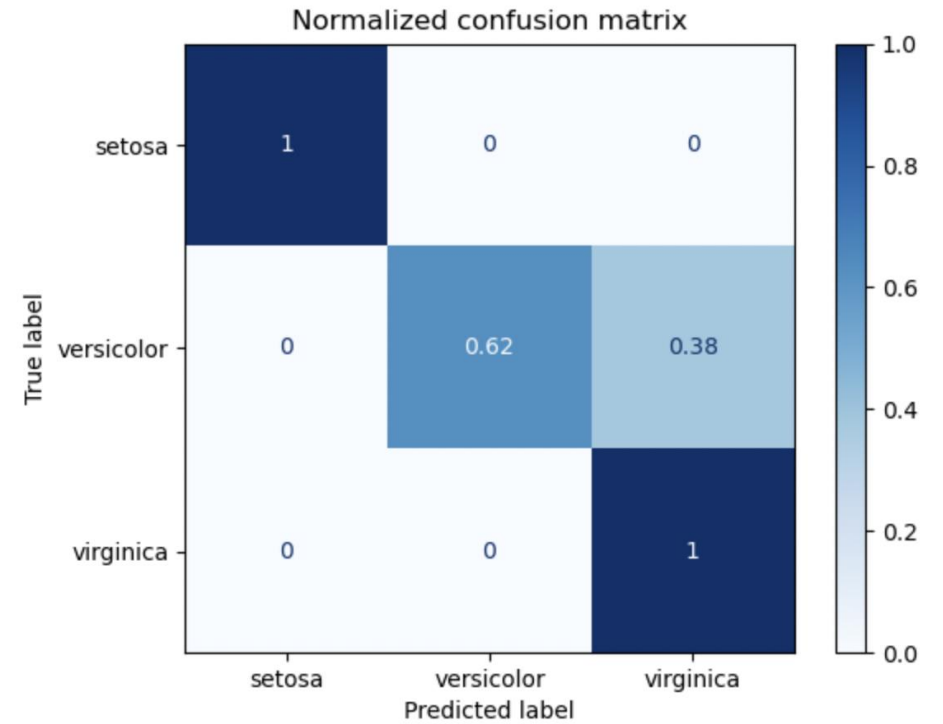
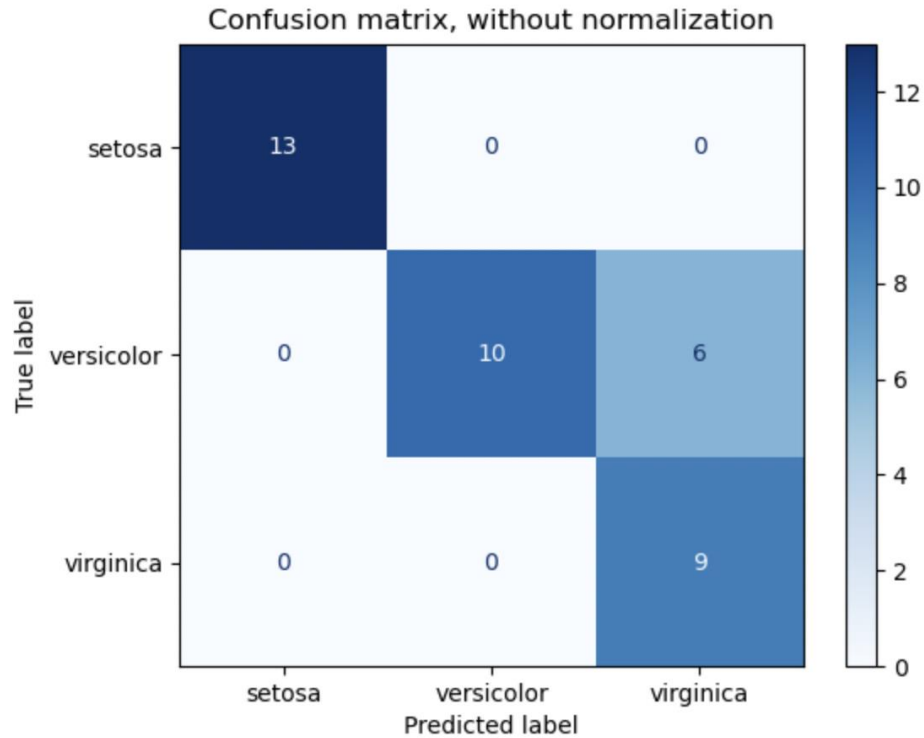
Evaluare

- Eroare antrenare
- Eroare testare
- Acuratete
- Matricea de confuzie



Evaluare

- Matricea de confuzie normalizata



Exemplu

- Set data MNIST (<http://yann.lecun.com/exdb/mnist/>)



Retea

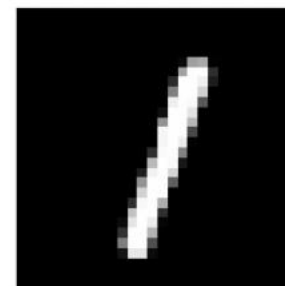
- **Recunoastere cifre**

- **set date MNIST:**

- 60,000 exemple de antrenare, 10,000 exemple de test
 - imagini: 28*28
 - imaginile sunt rescalate pentru a avea media zero și deviația standard 1.



4 (4)



1 (1)



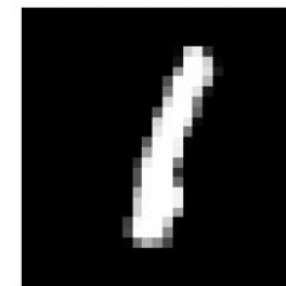
0 (0)



7 (7)



8 (8)



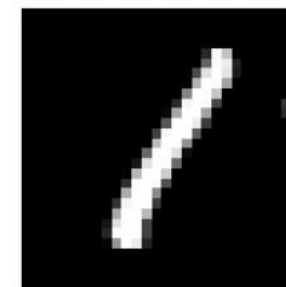
1 (1)



2 (2)

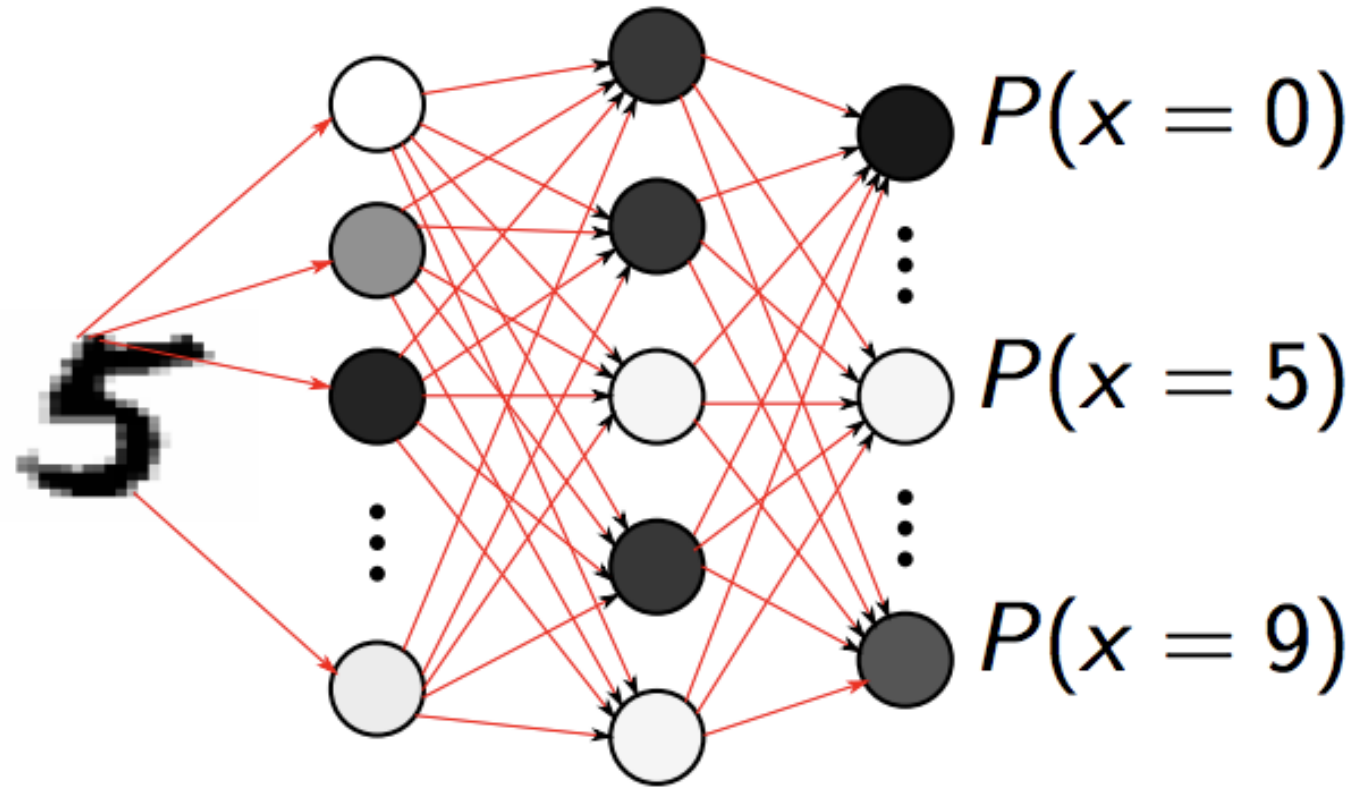


7 (7)



1 (1)

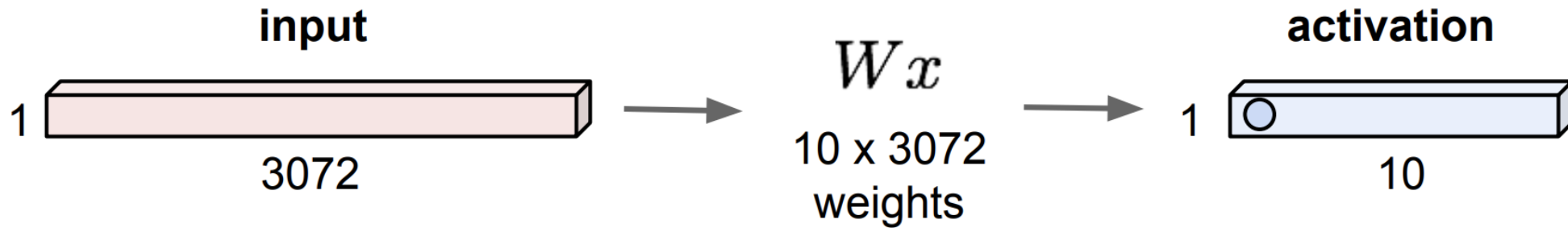
Exemplu



- Arhitectura
 - Intrari $28 \times 28 = 784$, iesiri = 10 clase

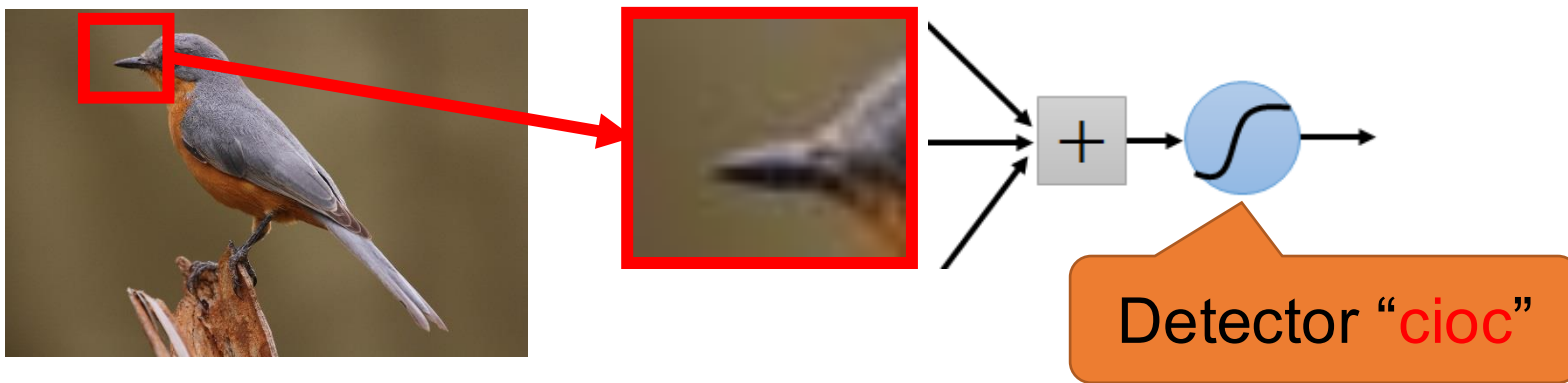
Multilayer Perceptron

32x32x3 image -> stretch to 3072 x 1

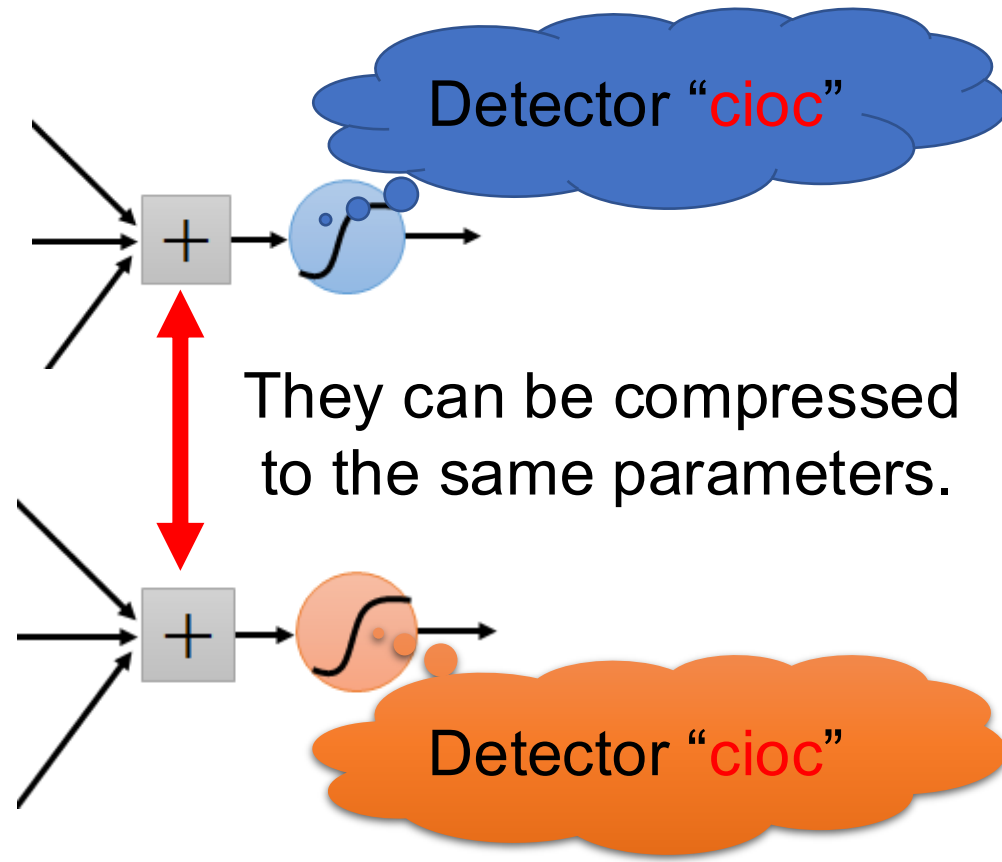
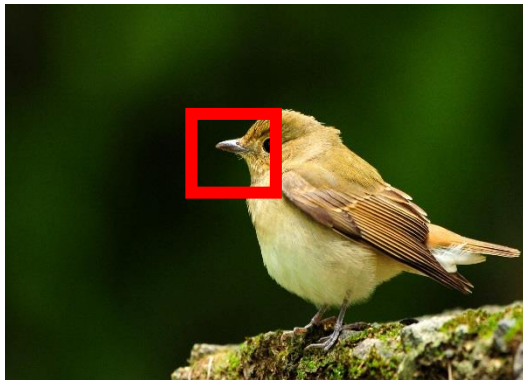


Invatarea unei imagini:

- Sabloane in interiorul imaginii

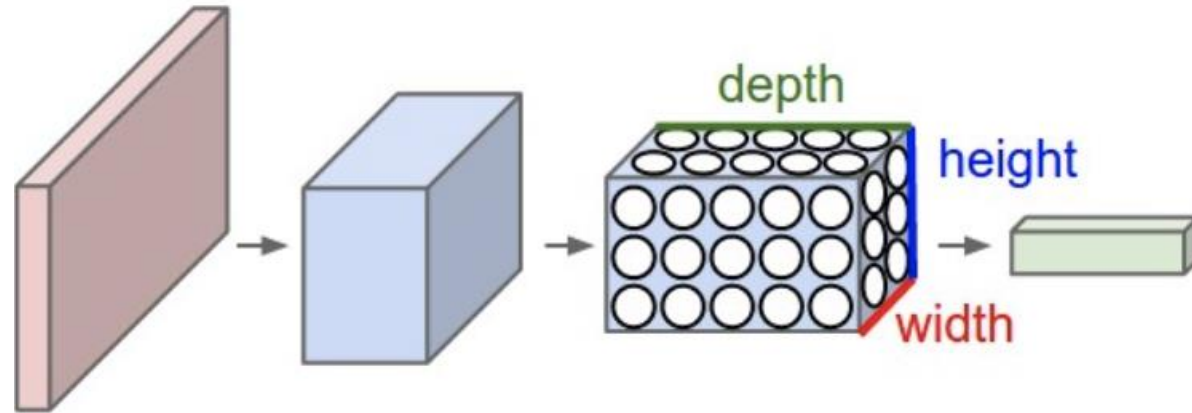


Sabloanele pot fi pozitionate diferit, avand dimensiuni diferite



Retele convolutive

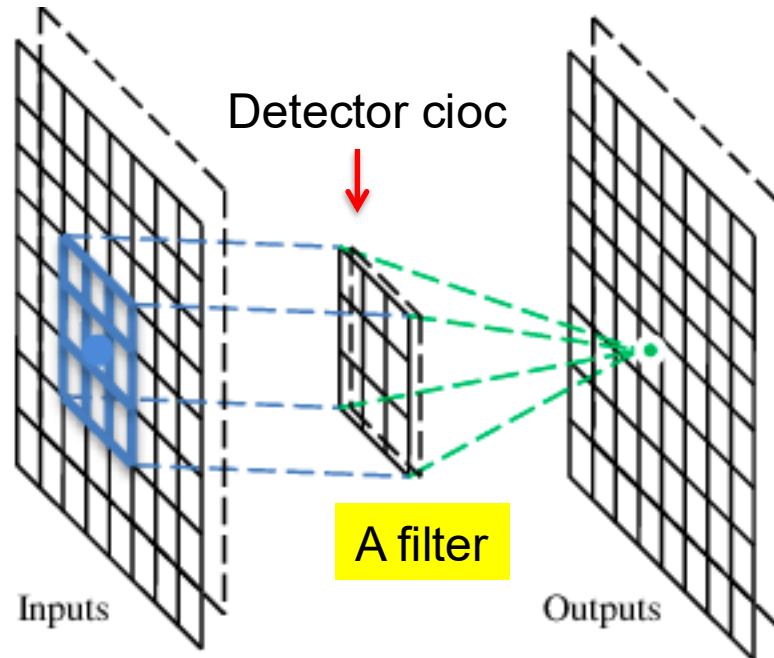
- Neuronii sunt dispusi in dimensiuni 3D (width, height, depth)
- Fiecare strat transforma volumul 3D de intrare intr-un volum 3D de iesire



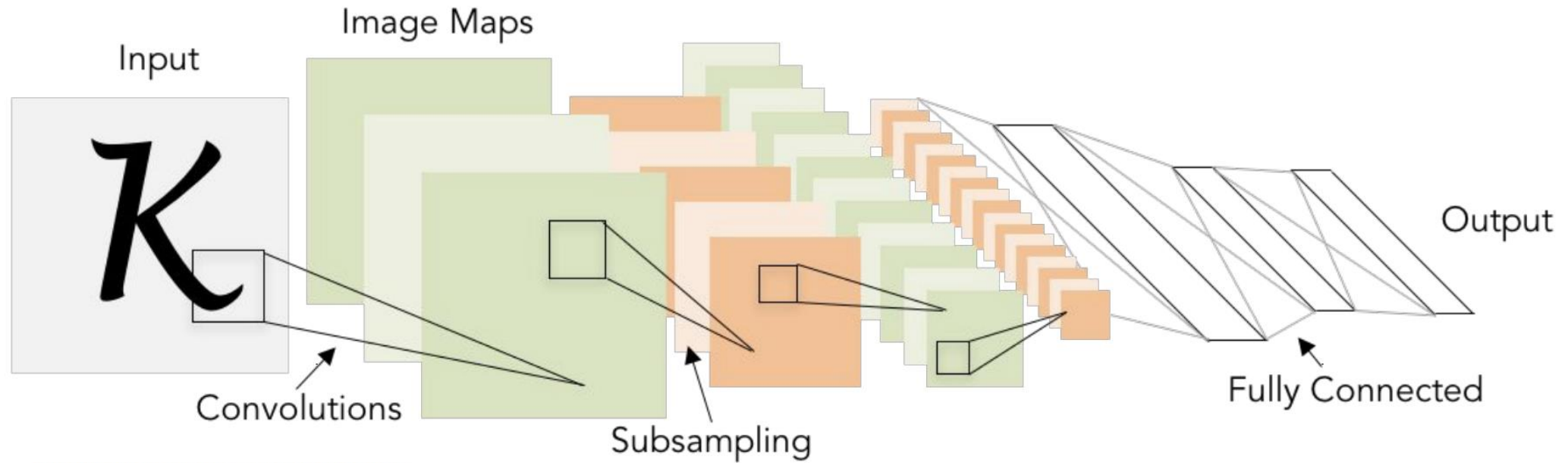
Retele convolutive (CNN)

CNN – retea neurala ce contine straturi convolutive (plus alte tipuri de straturi).

Strat convolucional – numar de filtre

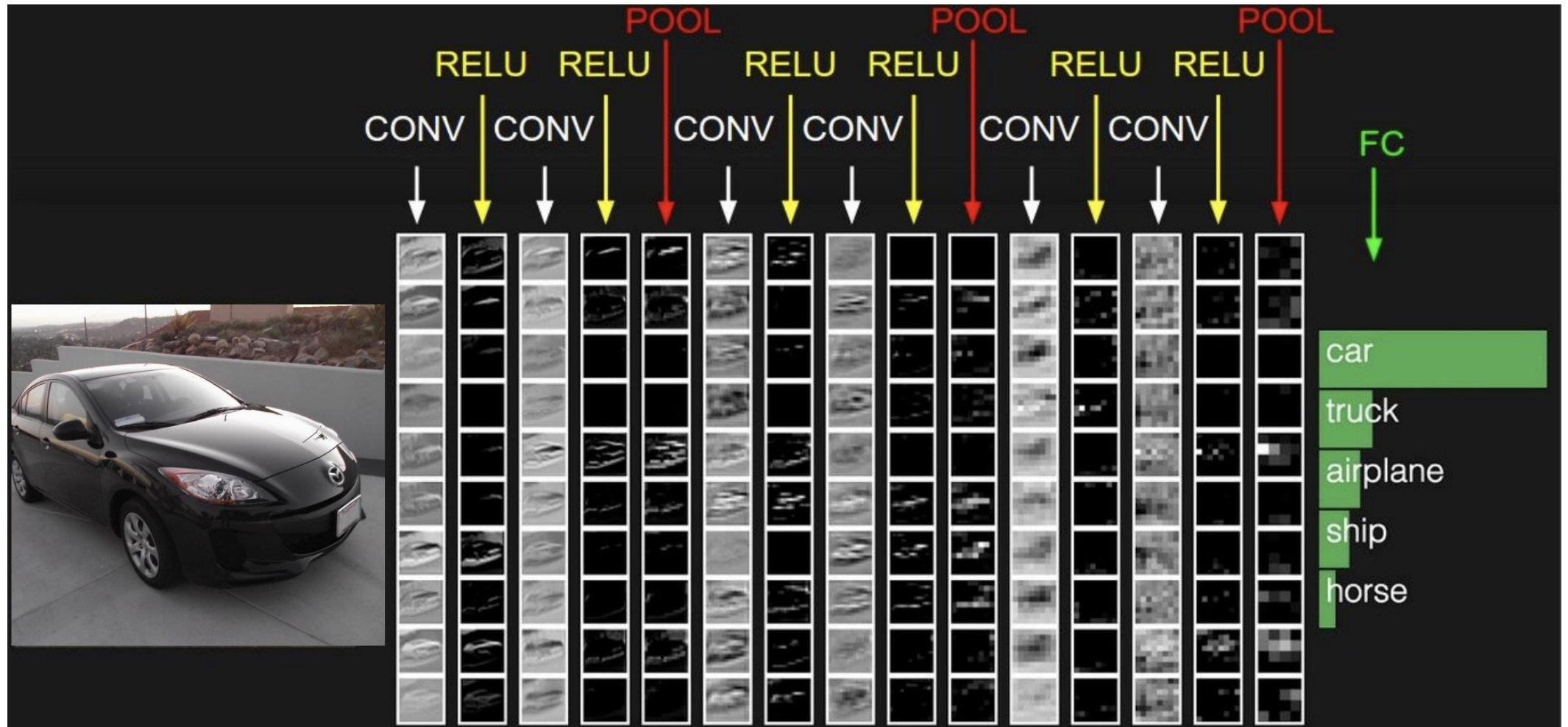


Retele convolutionale

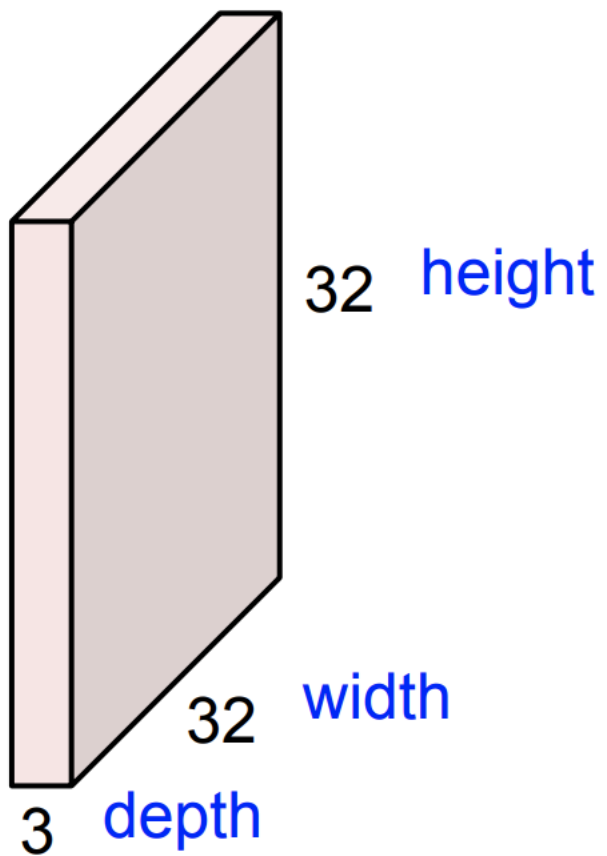


LeNet-5

Exemplu

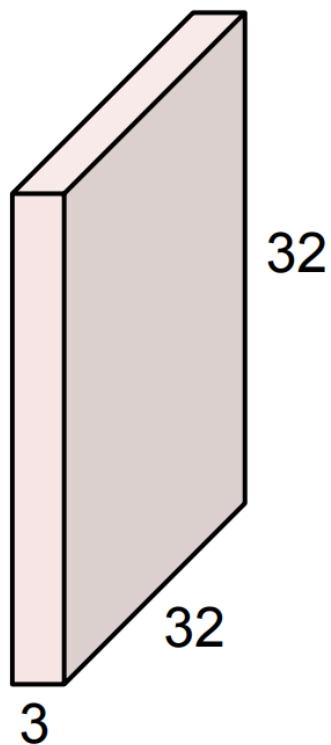


Strat convolutional



Strat convolutional

32x32x3 imagine

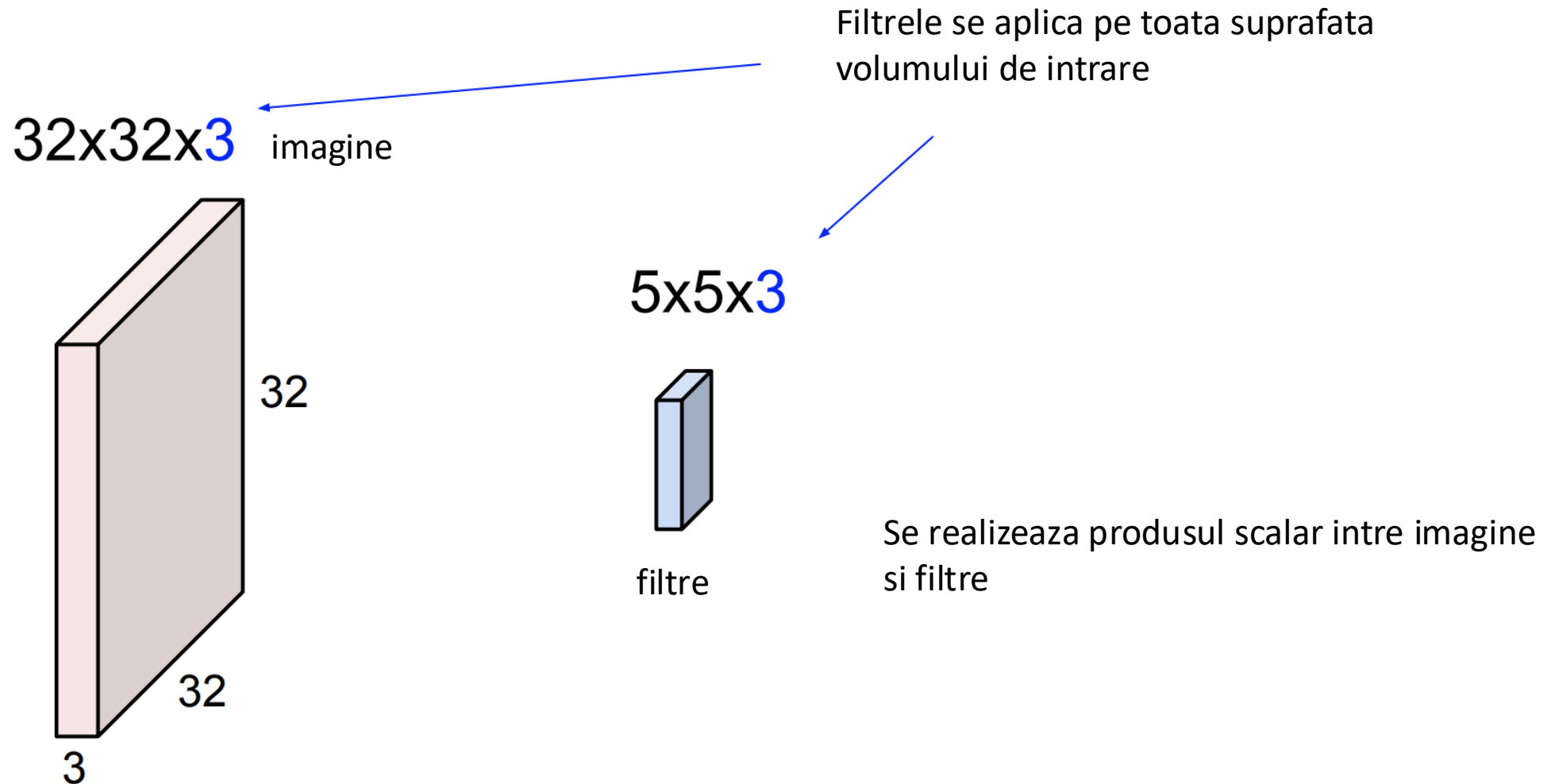


5x5x3 f

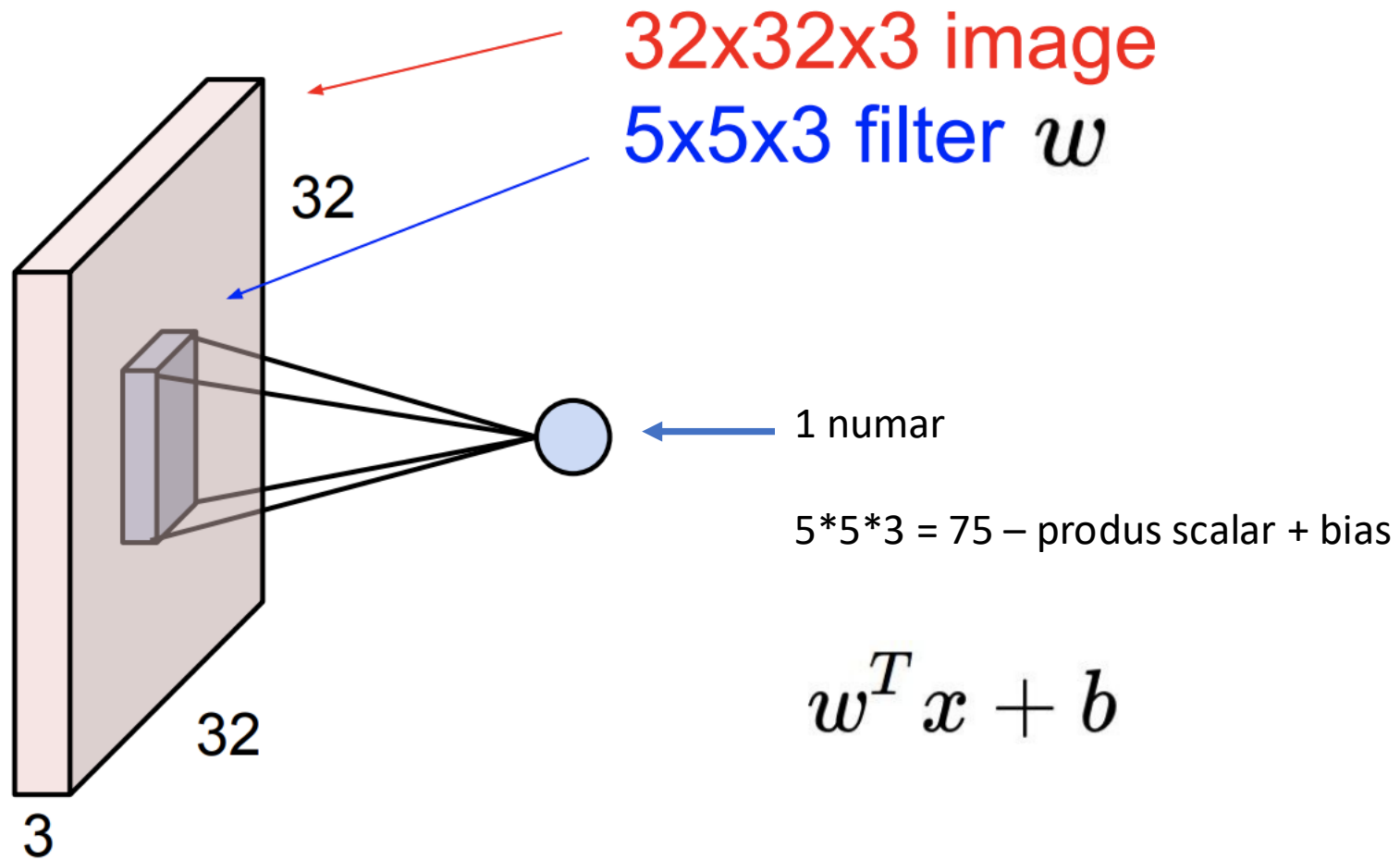


filtru

Convolution Layer

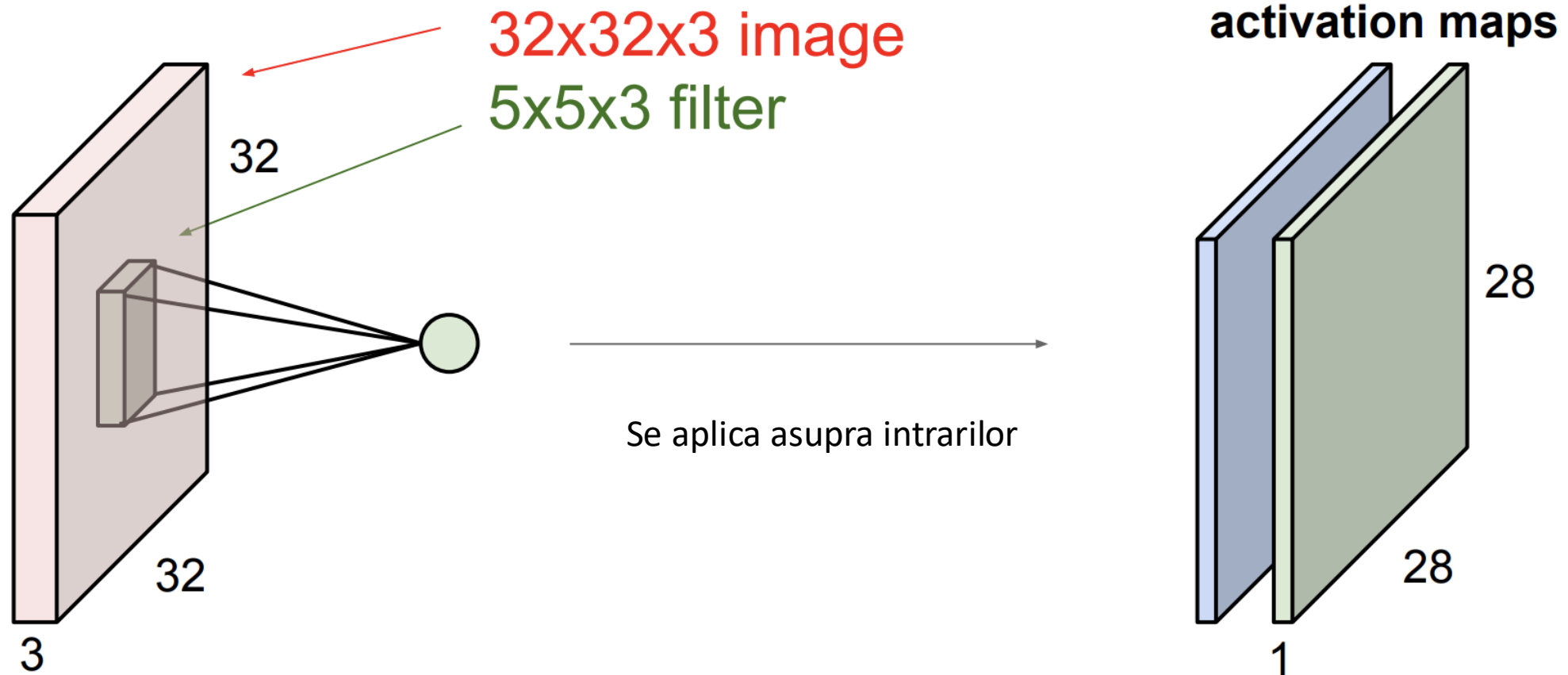


Strat convolutional

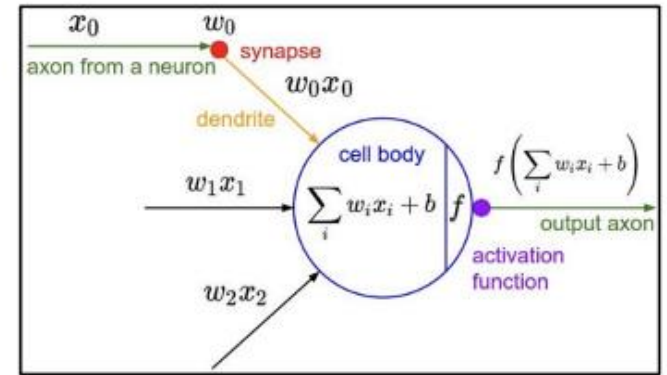
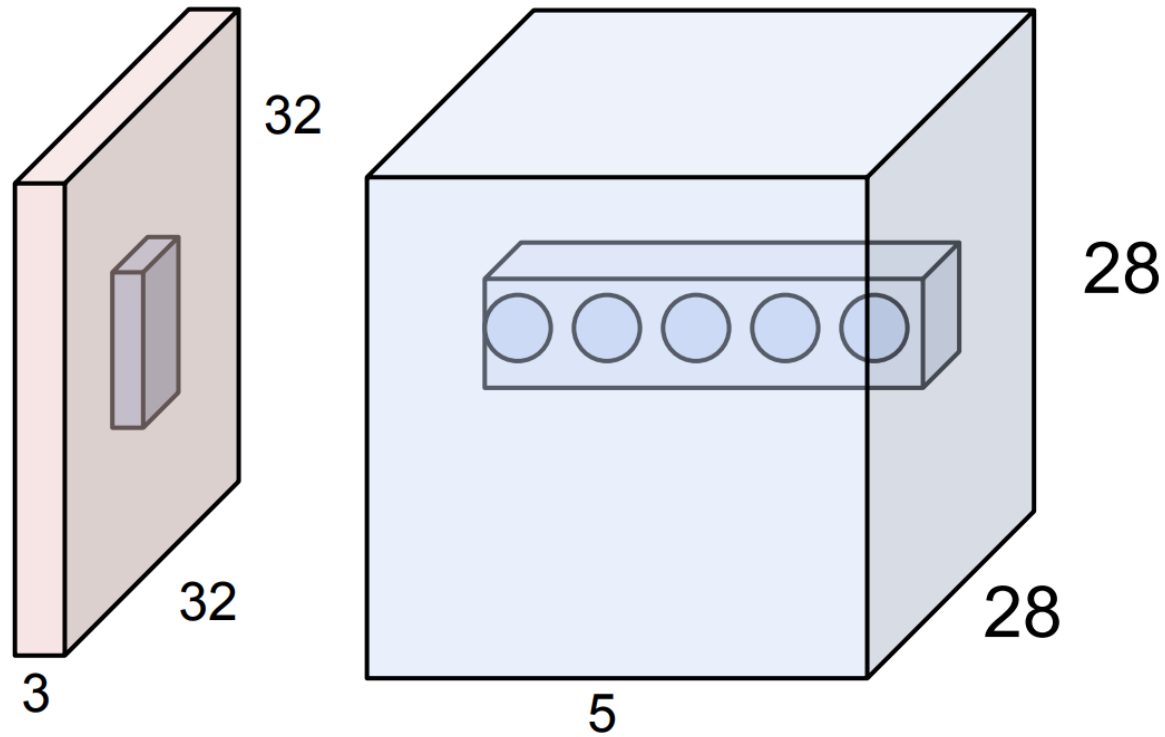


Strat convolutional

- Se considera al doilea filtru (marcat cu verde)

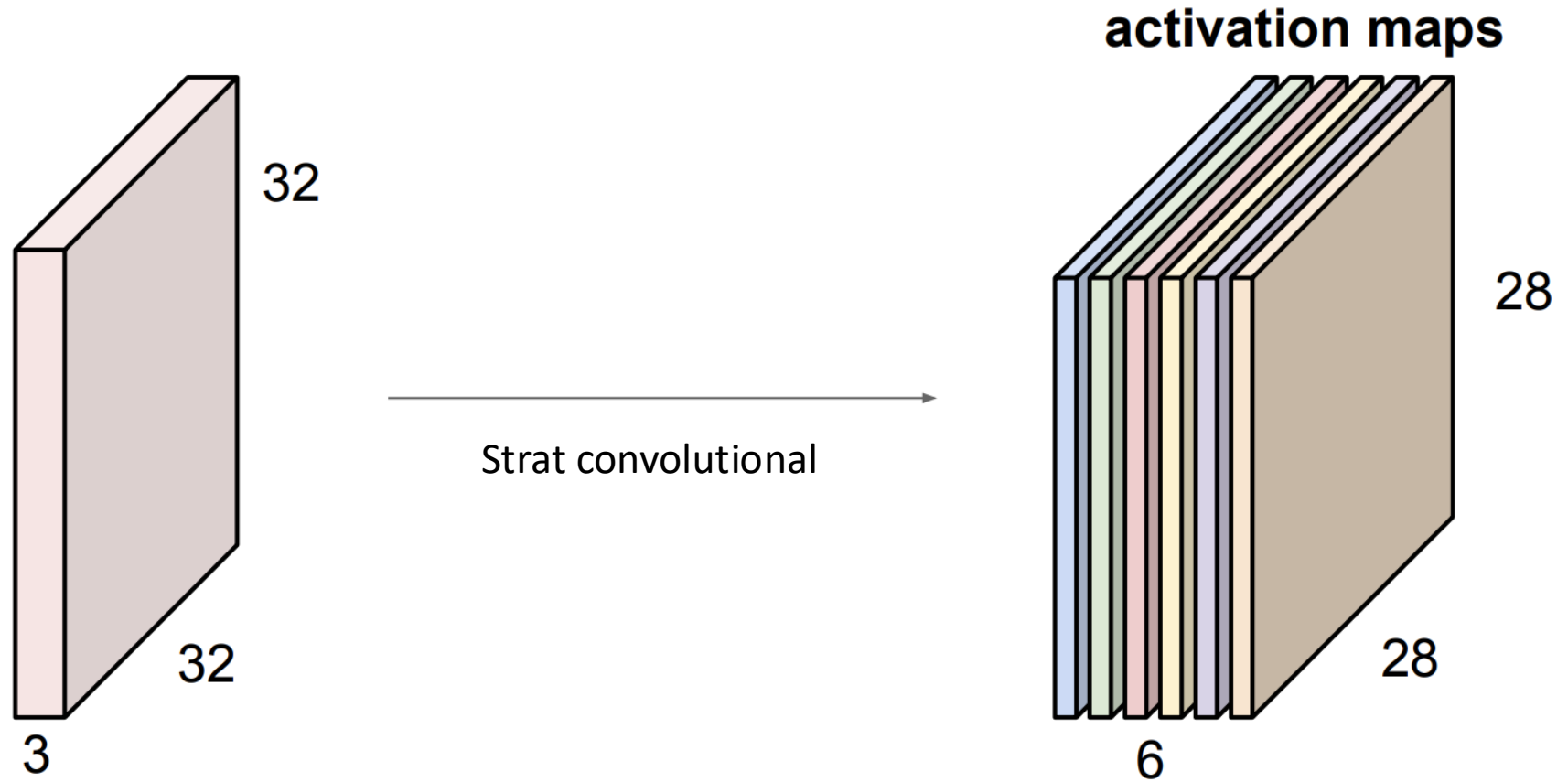


Strat convolutional



Strat convolutional

- 6 filtre de dimensiune 5x5 $\rightarrow$ 6 activation maps:

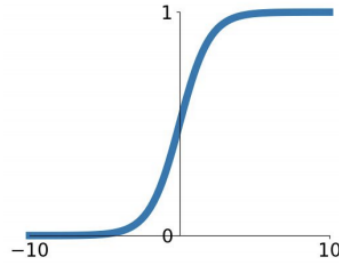


- lesirea stratului convolutional volum de dimensiune 28x28x6

Functii de activare

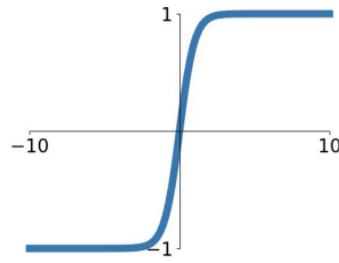
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



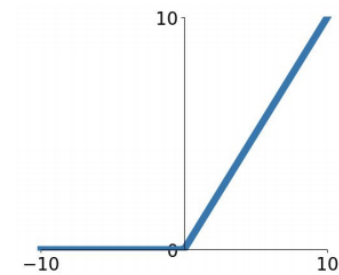
tanh

$$\tanh(x)$$



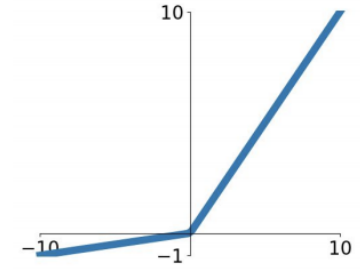
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

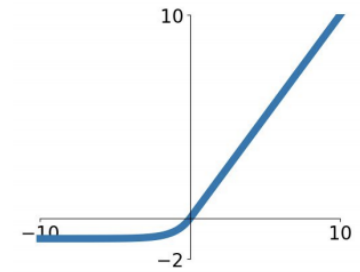


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

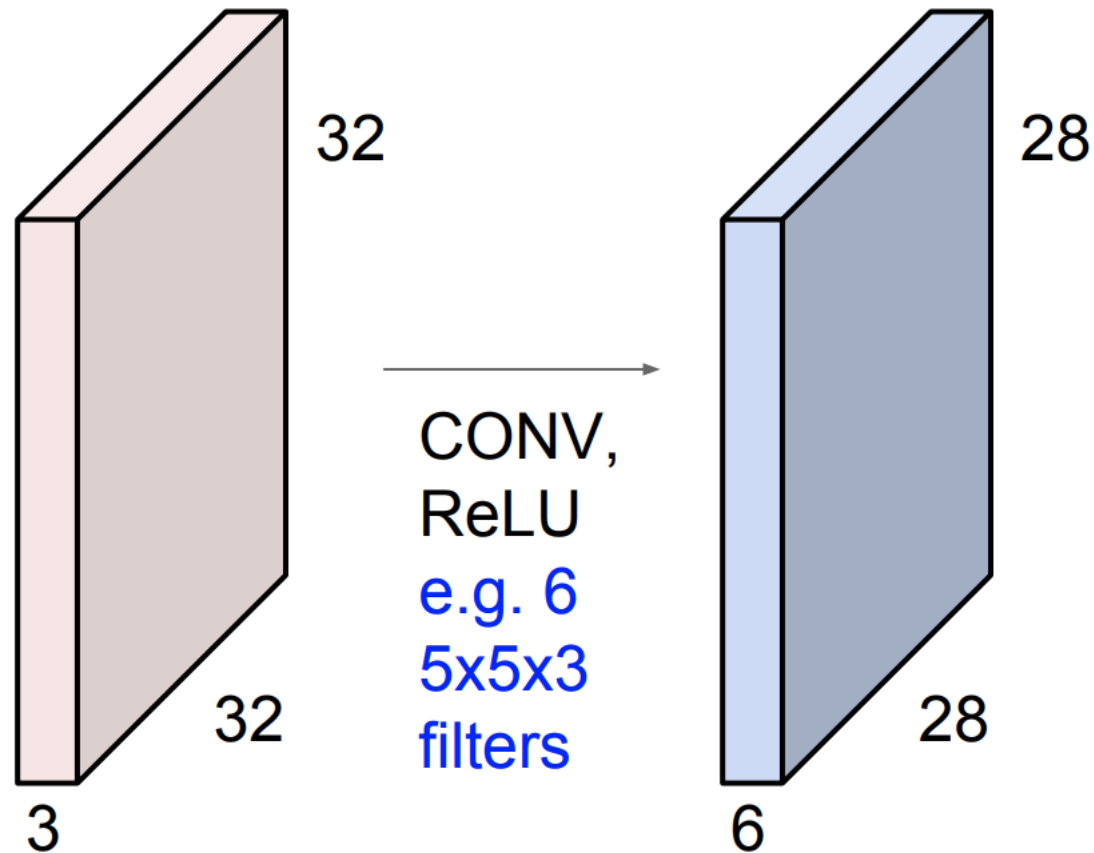
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

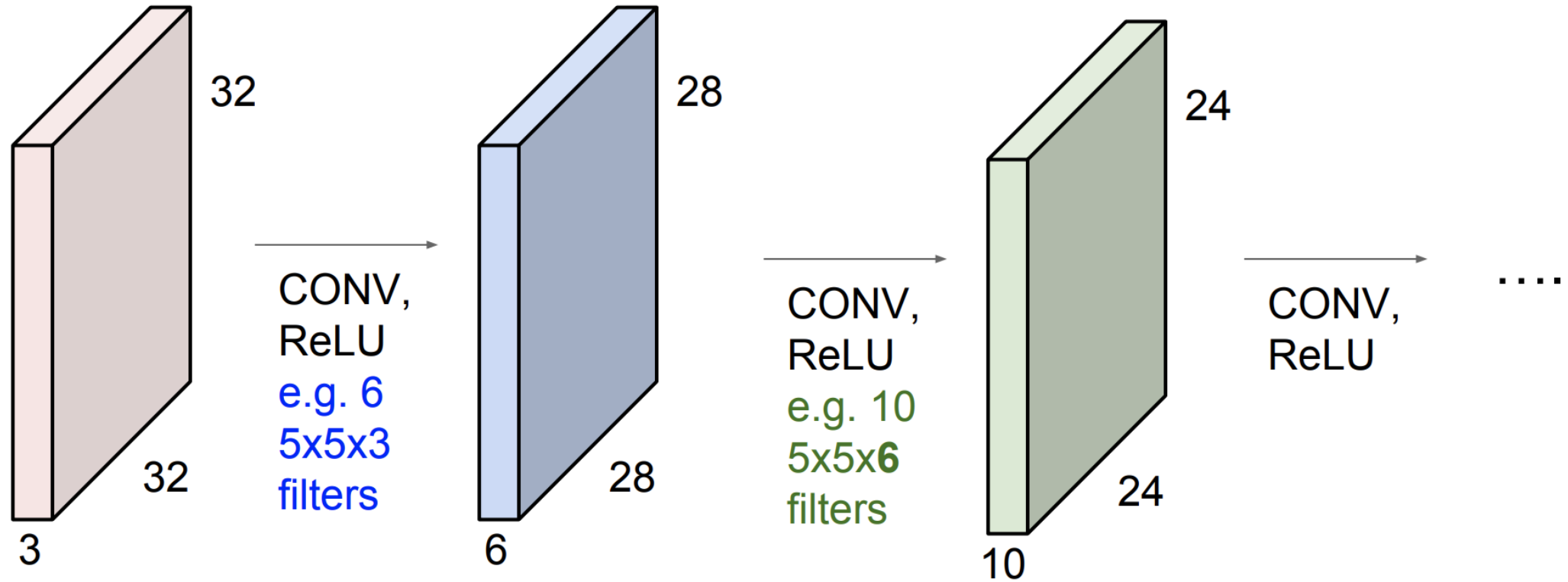


Strat convolutional

- Succesiune de straturi convolutionale + functii de activare

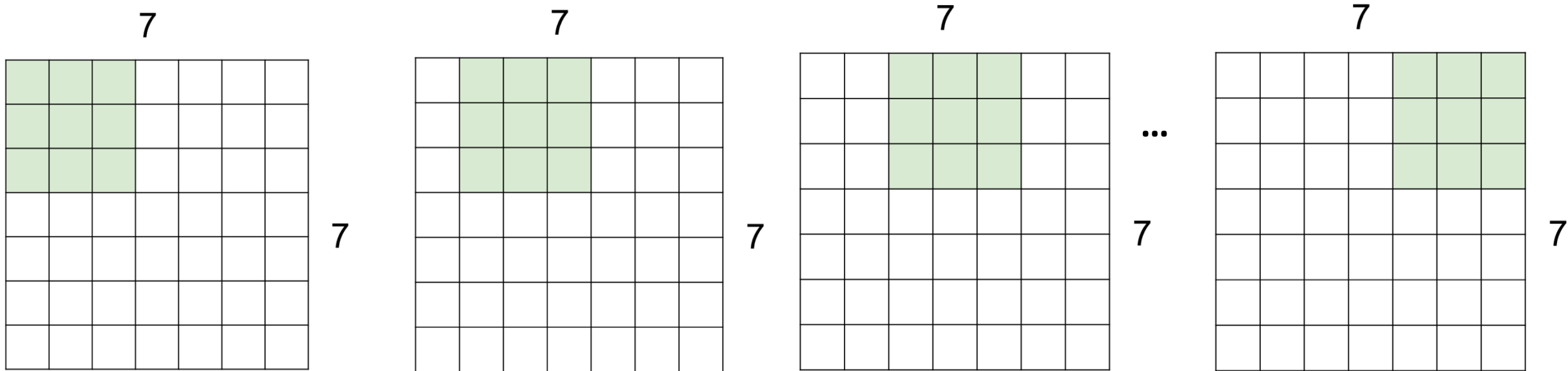


Strat convolutional



Exemplu – strat convolutional

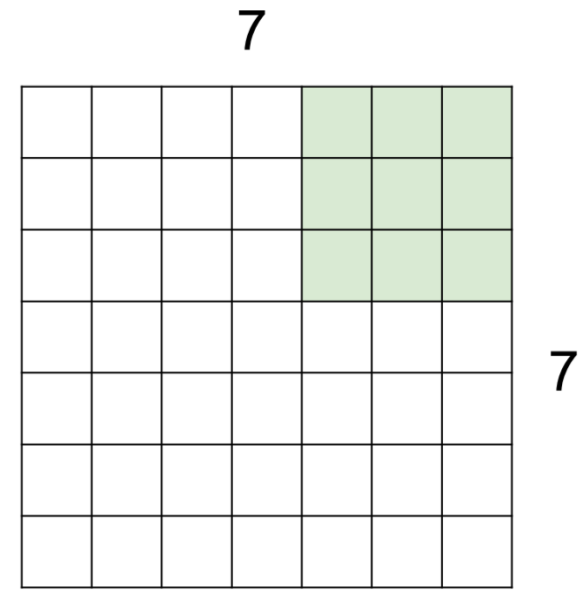
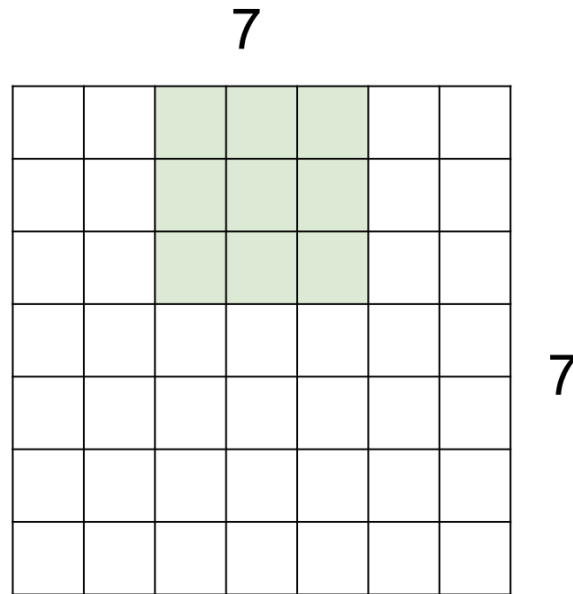
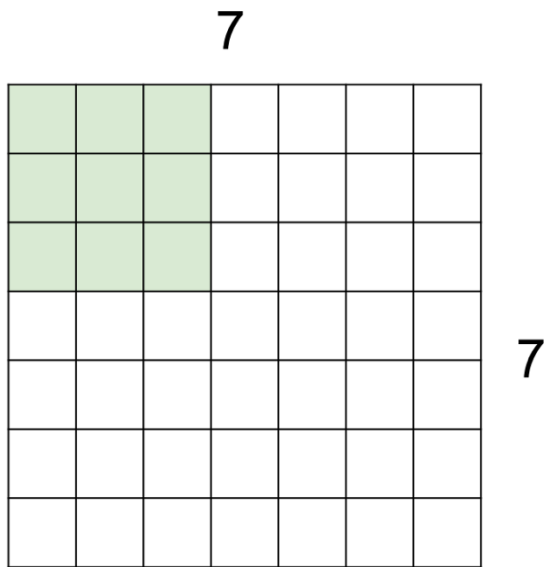
- Intrare: 7x7 input, filtru: 3x3



- => iesire: 5x5

Exemplu – strat convolutional

- Intrare: 7x7 input, filtru: 3x3, stride 2

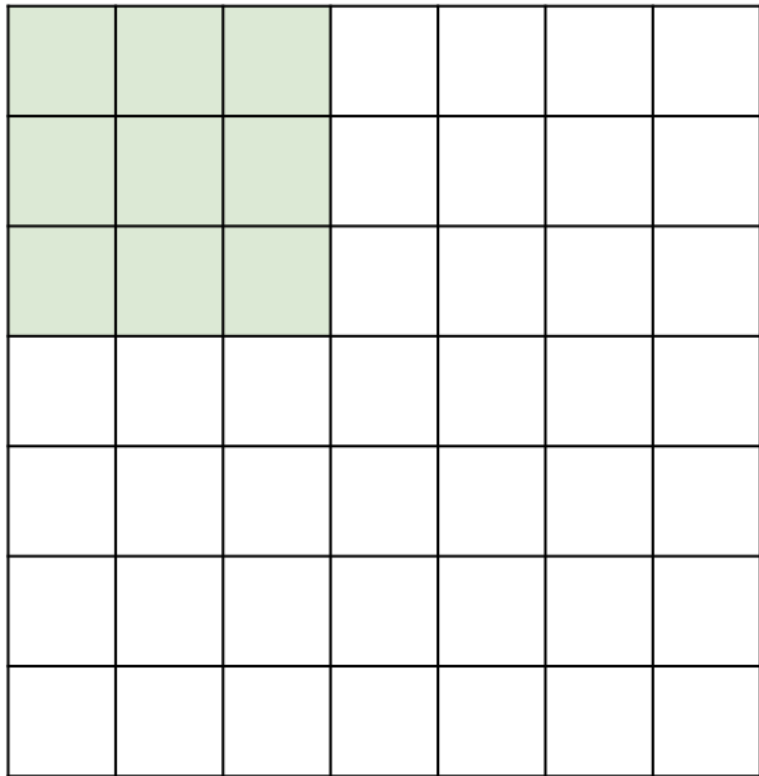


- Iesire: 3x3 output

Exemplu – strat convolutional

- Input: 7x7 input, filtru: 3x3, stride = 3?

7

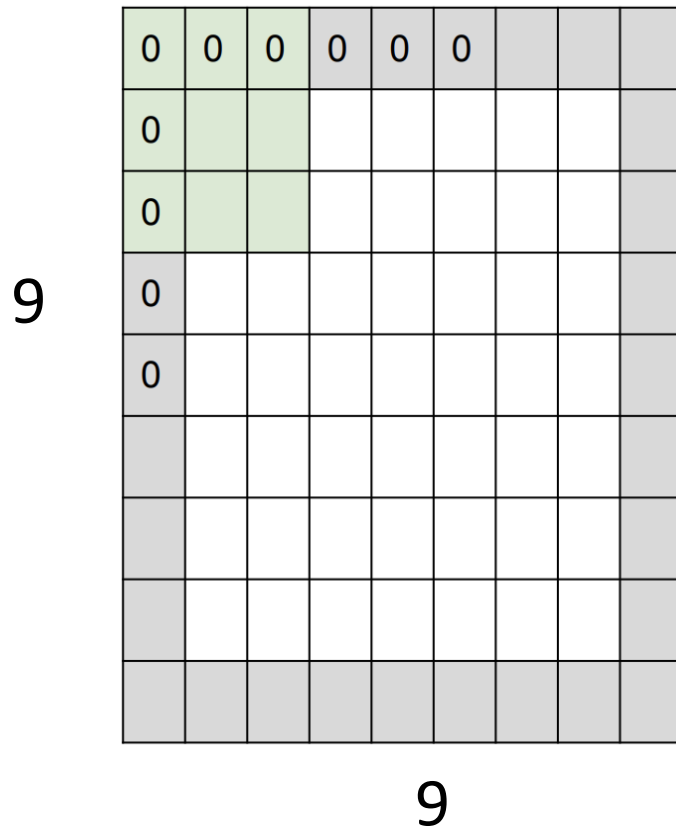


7

Nu se poate.

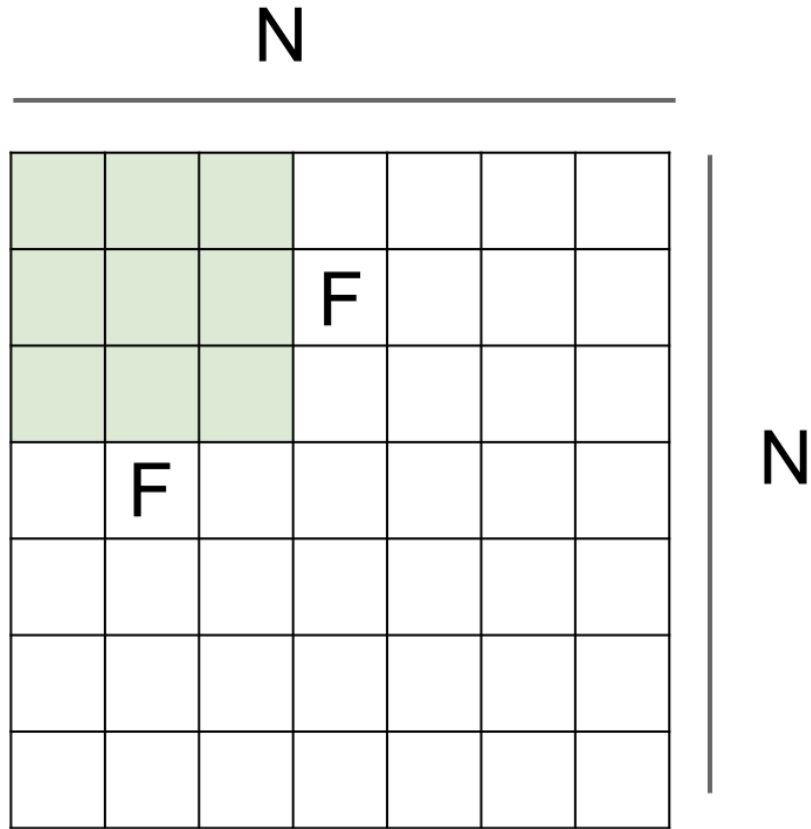
Exemplu – strat convolutional

- Input: 7x7 input, filtru: 3x3, stride = 3, padding = 1



⇒ lesire: 7x7

Exemplu – strat convolutional



Output size:

$$(N - F) / \text{stride} + 1$$

e.g. $N = 7$, $F = 3$:

$$\text{stride } 1 \Rightarrow (7 - 3) / 1 + 1 = 5$$

$$\text{stride } 2 \Rightarrow (7 - 3) / 2 + 1 = 3$$

$$\text{stride } 3 \Rightarrow (7 - 3) / 3 + 1 = 2.33$$

Strat convolutional

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 imagine

Ponderile rețelei

1	-1	-1
-1	1	-1
-1	-1	1

Filtru 1

-1	1	-1
-1	1	-1
-1	1	-1

Filtru 2

⋮ ⋮

Fiecare filtru detectează
o regiune (3 x 3).

Strat convolutional

stride=1

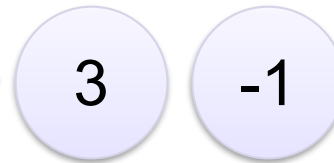
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filtru 1

Prodotto
scalare



Strat convolutional

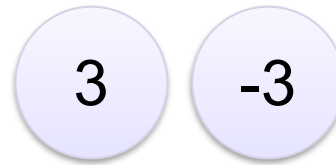
If stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

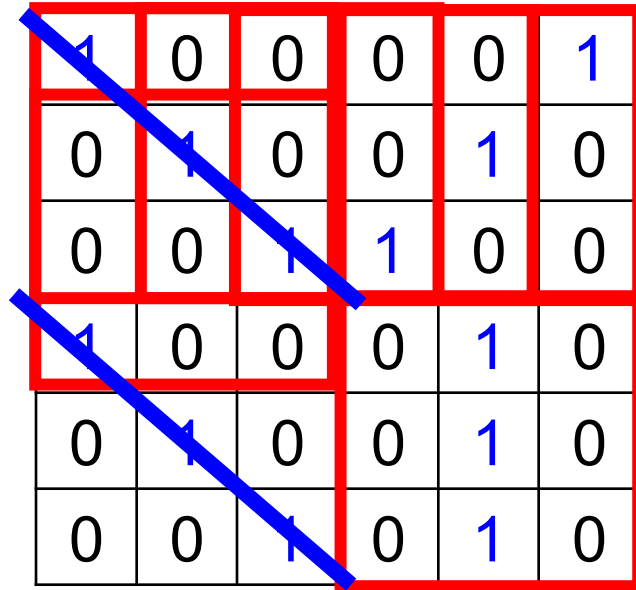
1	-1	-1
-1	1	-1
-1	-1	1

Filtru 1



Strat convolutional

stride=1

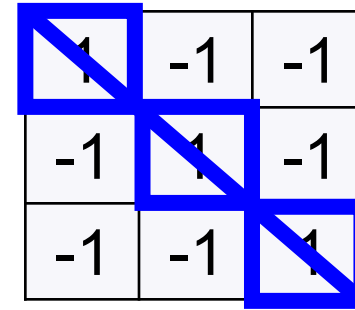


A 6x6 grid representing an input image. The grid contains the following values:

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

A 3x3 red box highlights the top-left corner of the grid, covering the first three rows and columns. A blue diagonal line runs from the top-left corner (1,1) to the bottom-right corner (6,6).

6 x 6 image

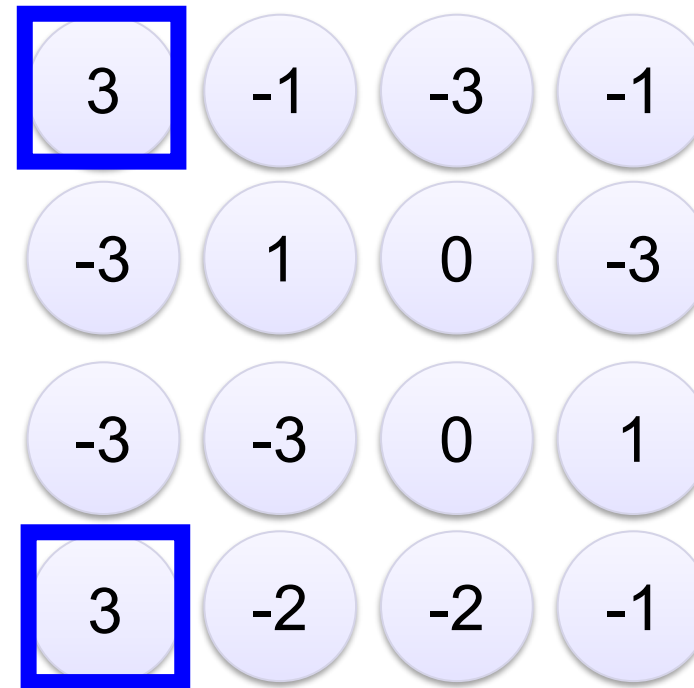


A 3x3 grid representing a filter. The grid contains the following values:

1	-1	-1
-1	1	-1
-1	-1	1

The entire 3x3 grid is enclosed in a thick blue border. A blue diagonal line runs from the top-left corner (1,1) to the bottom-right corner (3,3).

Filtru 1



A 4x4 grid representing the output of the convolution. The grid contains the following values:

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

The top-left element (3) and the bottom-left element (3) are each enclosed in a thick blue border.

Strat convolutional

stride=1

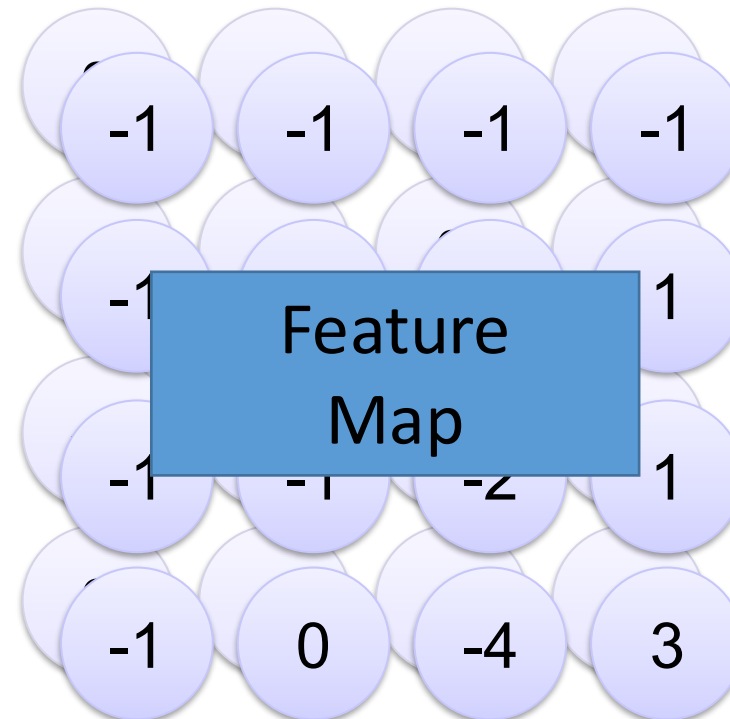
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

-1	1	-1
-1	1	-1
-1	1	-1

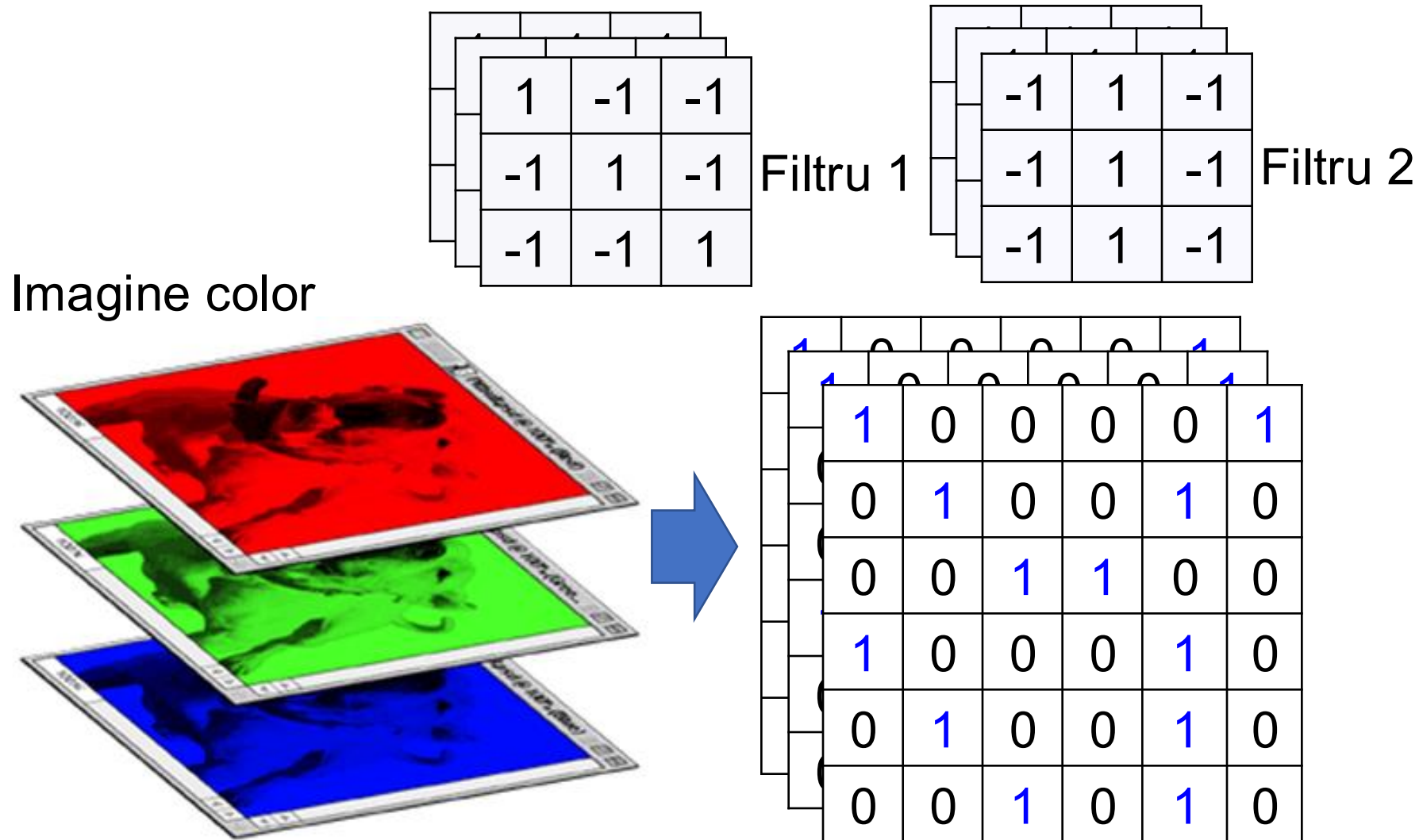
Filter 2

Repeat this for each filter

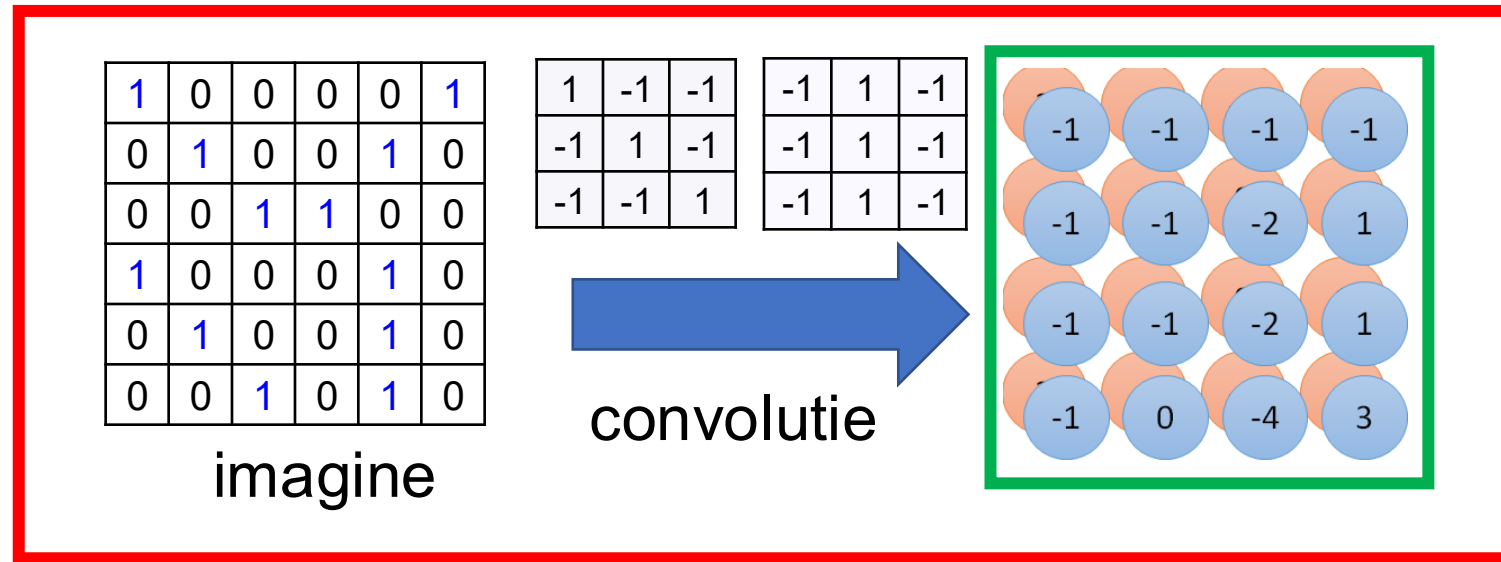


2 images 4 x 4
matrix 2 x 4 x 4

Imagine color: 3 canale RGB

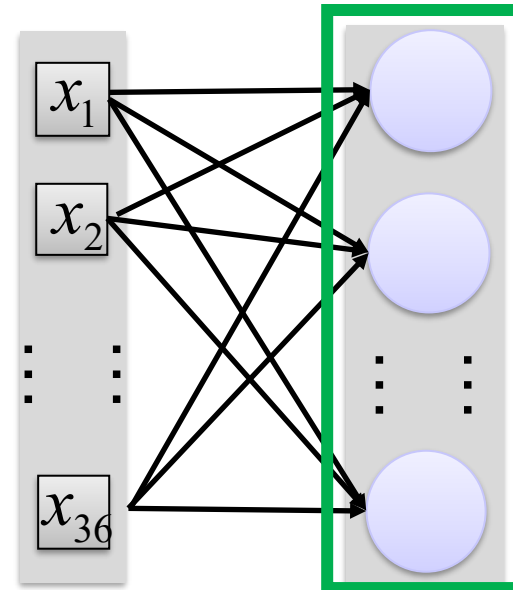


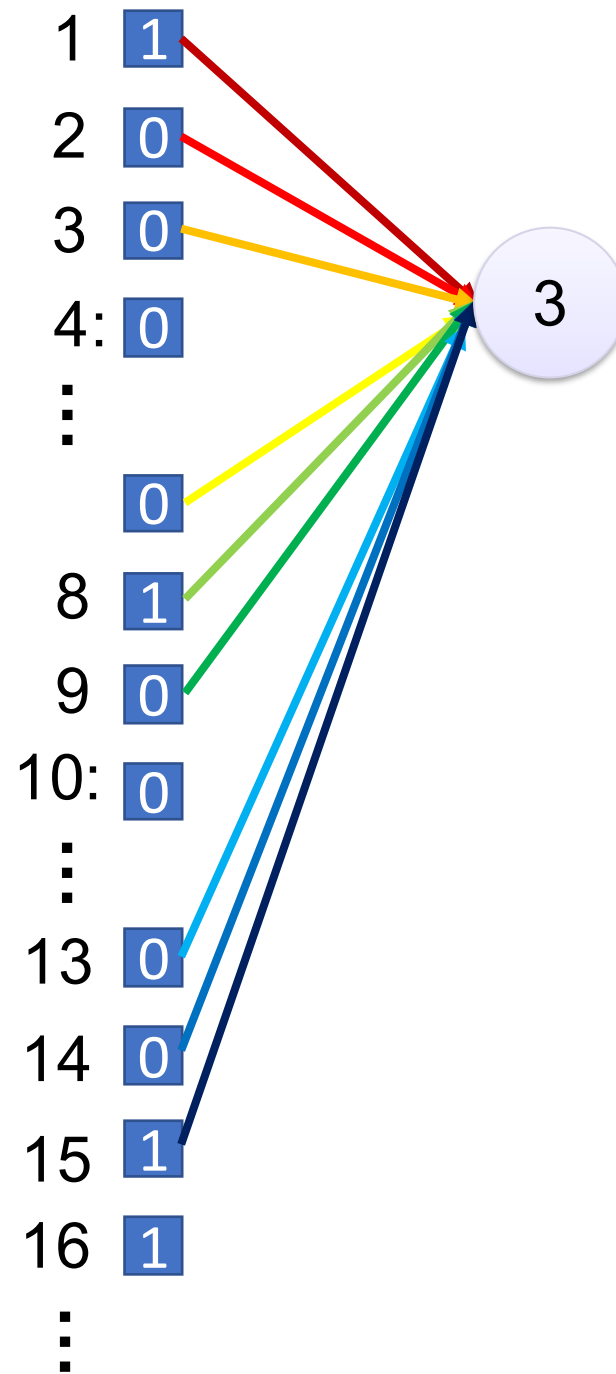
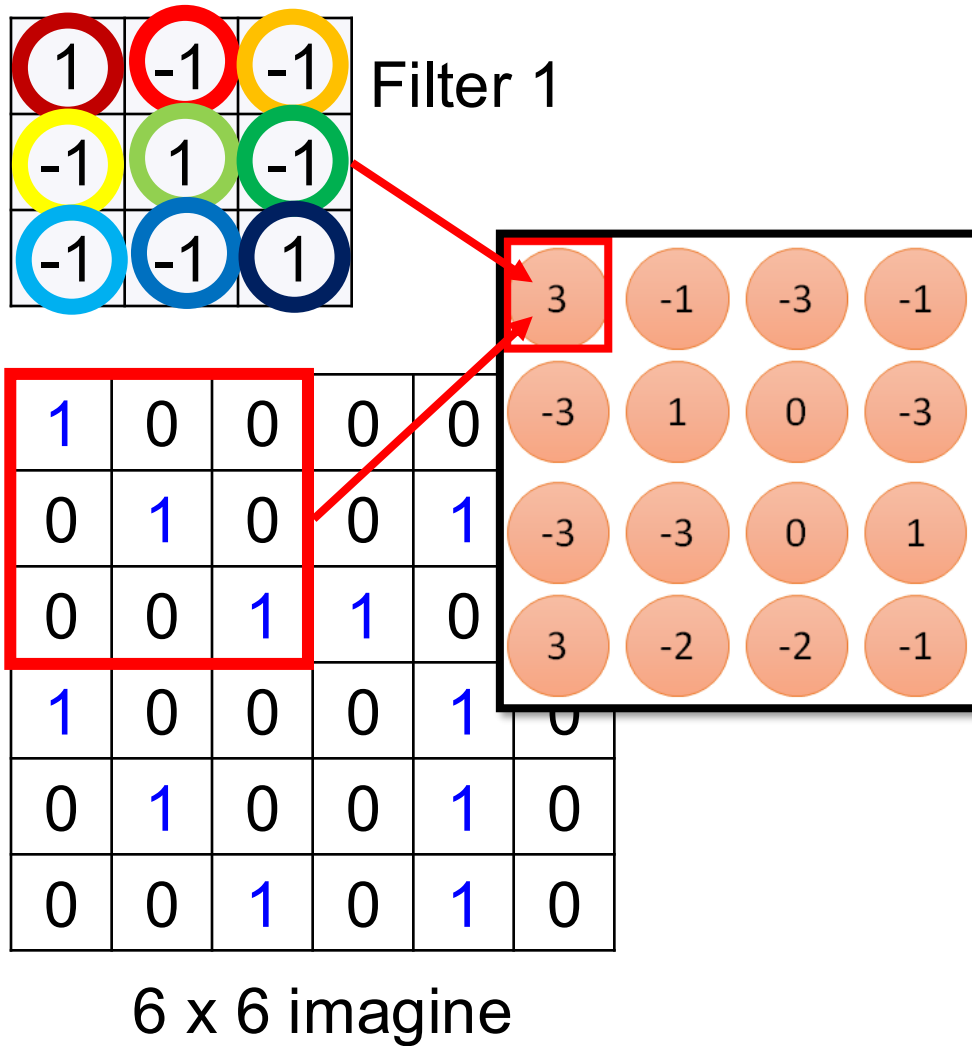
Retele convolusionale v.s. Fully Connected

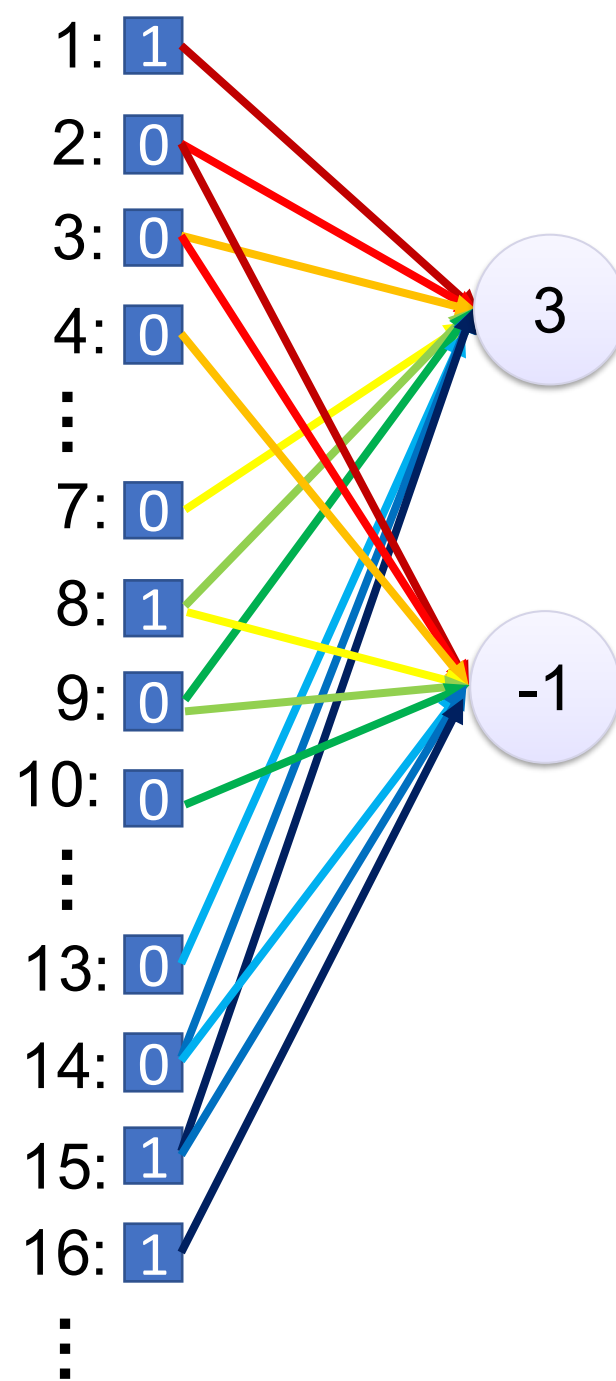
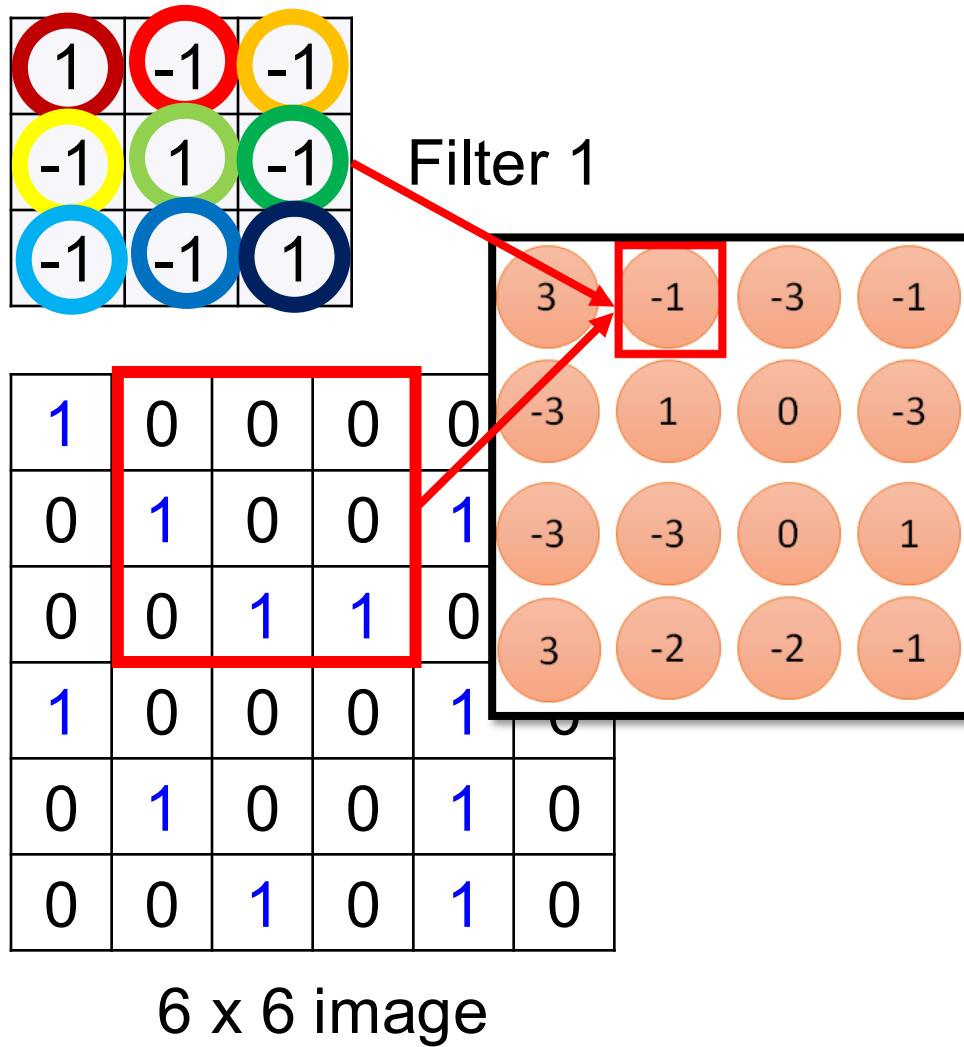


Fully-
connected

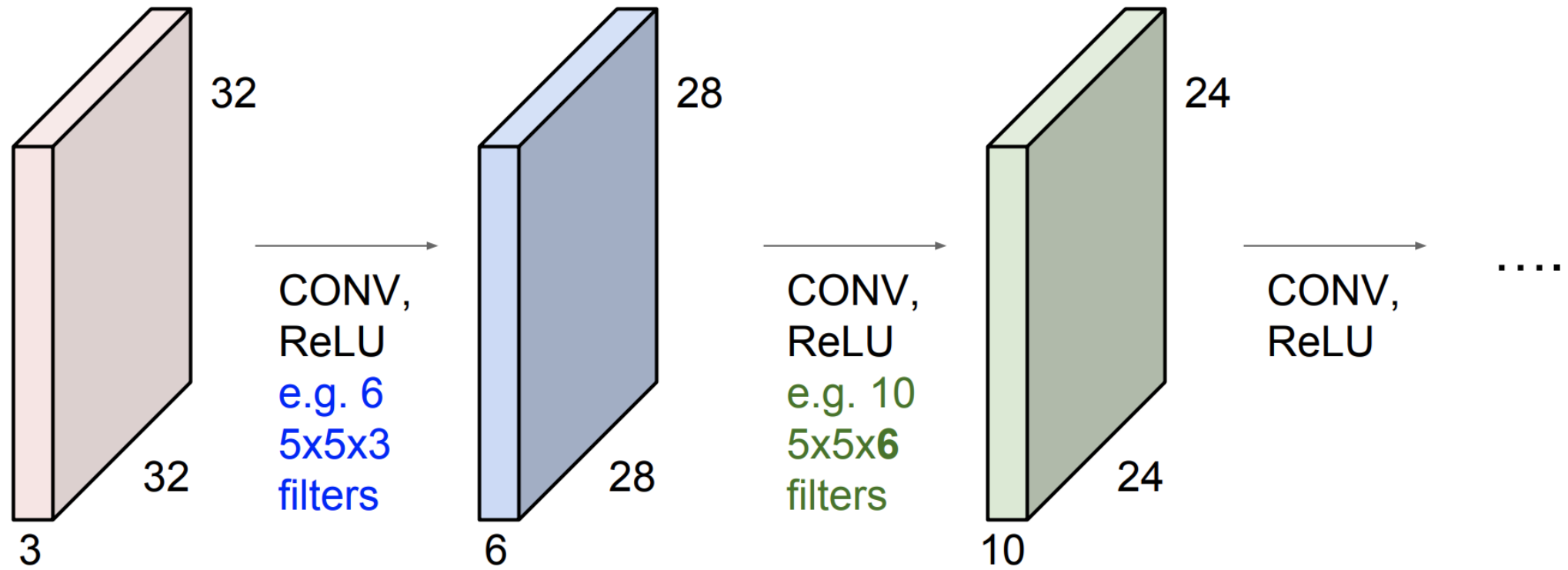
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0





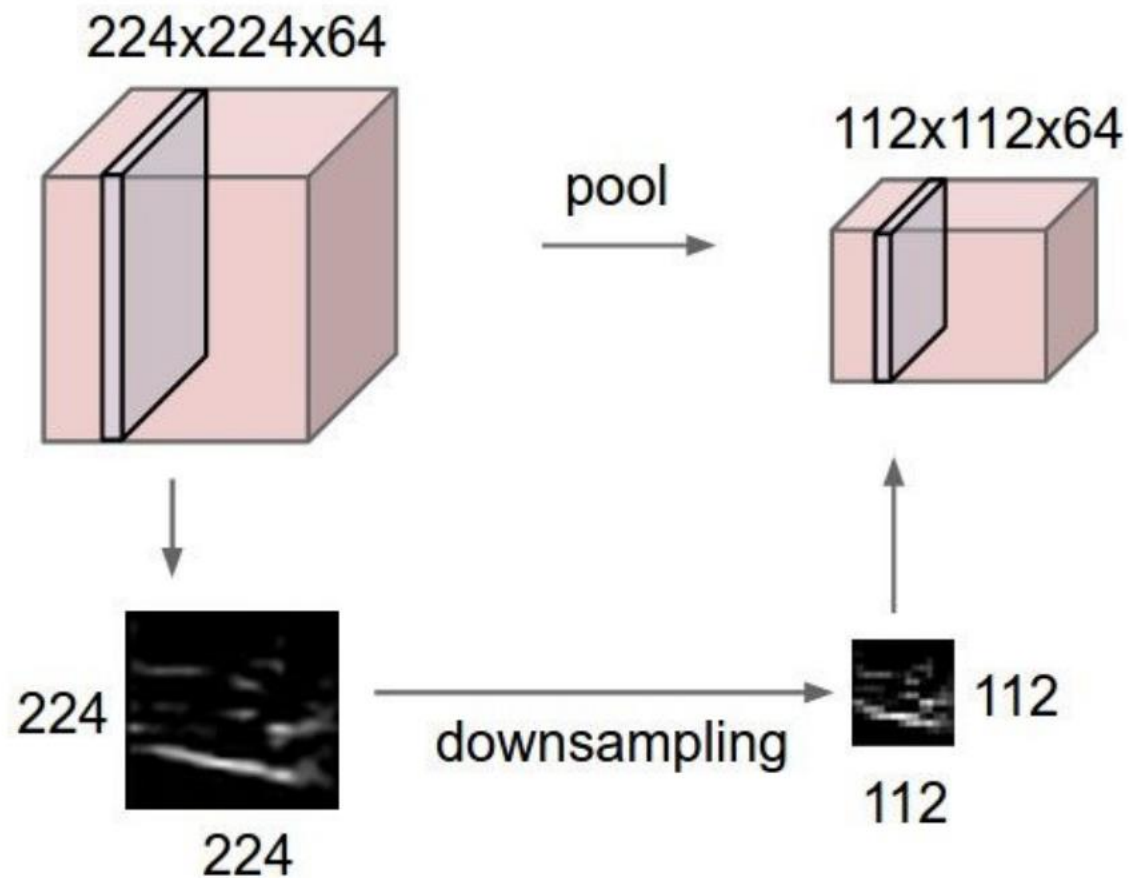


Exemplu



Stratul de pooling

- Reducerea dimensiunii



Stratul de pooling

- Nu se modifica forma obiectului

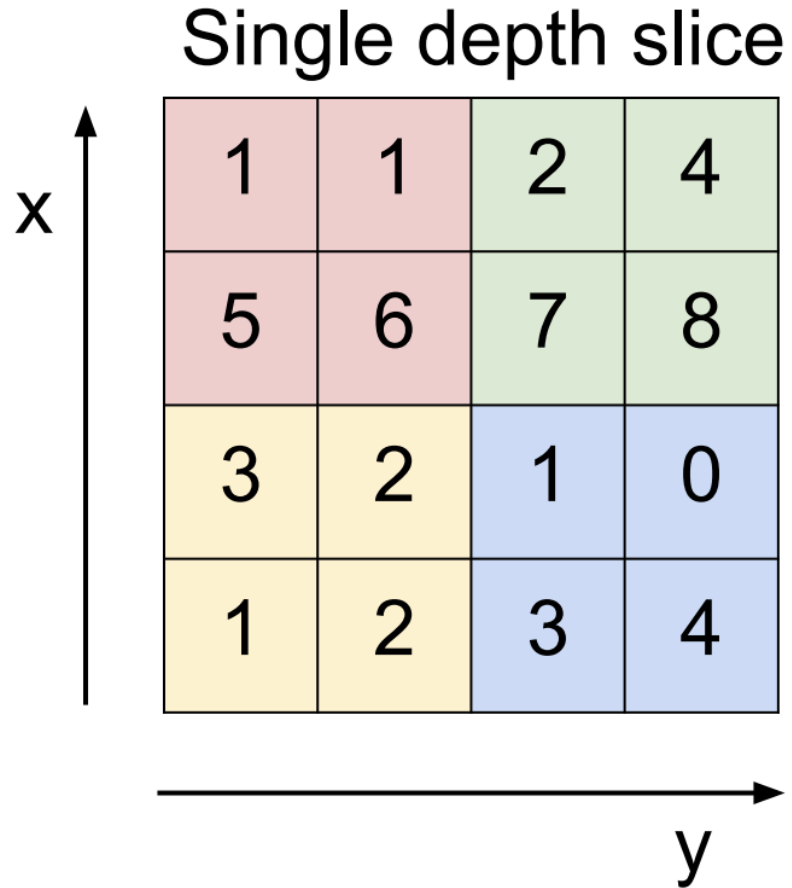


Subsampling

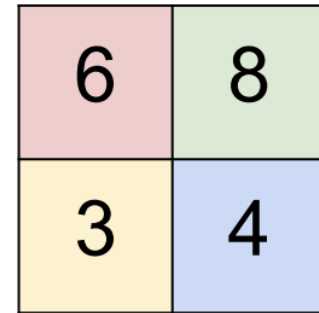


Sunt necesari mai putini parametri

Stratul de pooling



max pool with 2x2 filters
and stride 2



Max Pooling

1	-1	-1
-1	1	-1
-1	-1	1

Filtru 1

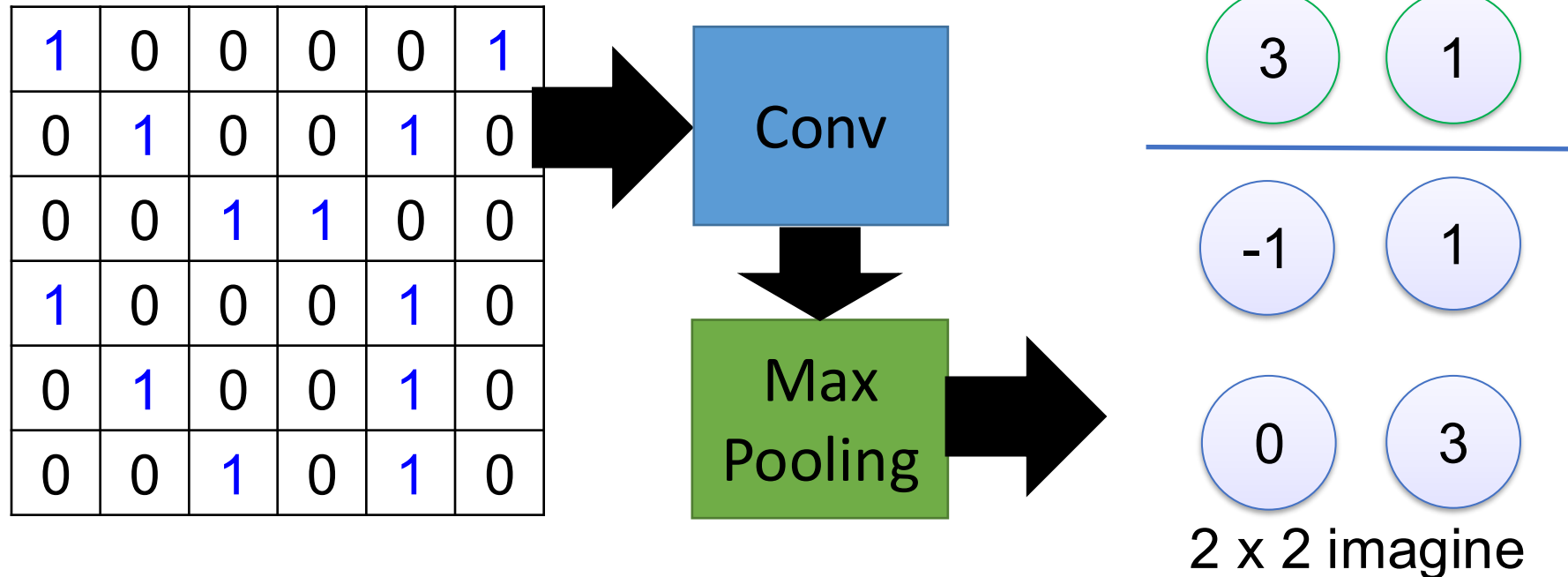
-1	1	-1
-1	1	-1
-1	1	-1

Filtru 2

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

Max Pooling



Imagine noua de
dimensiuni mai mici

3 0

3 1

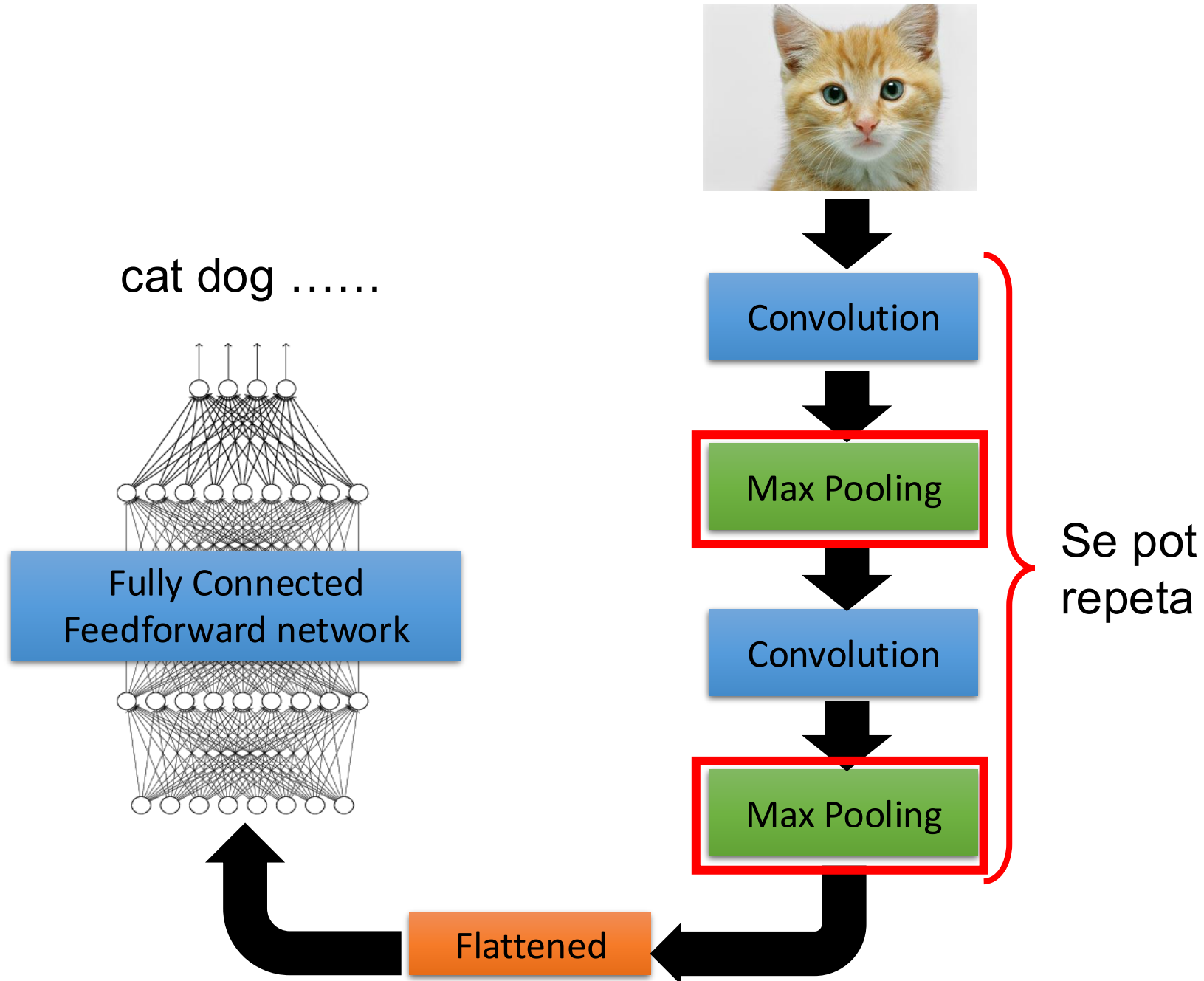
-1 1

0 3

2 x 2 imagine

fiecare filtru
reprezinta un canal

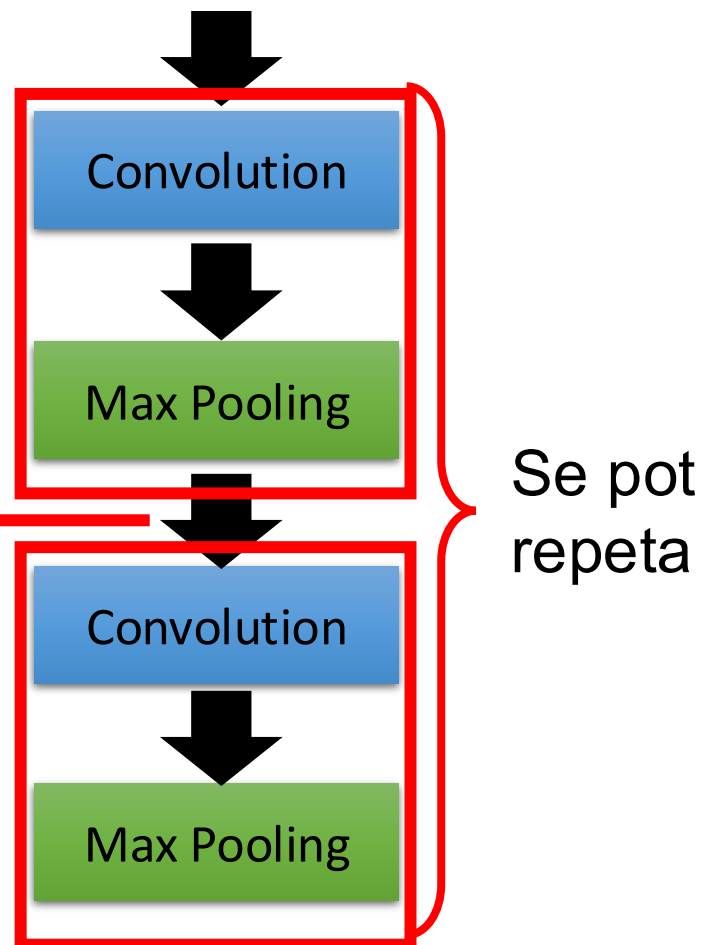
CNN



CNN

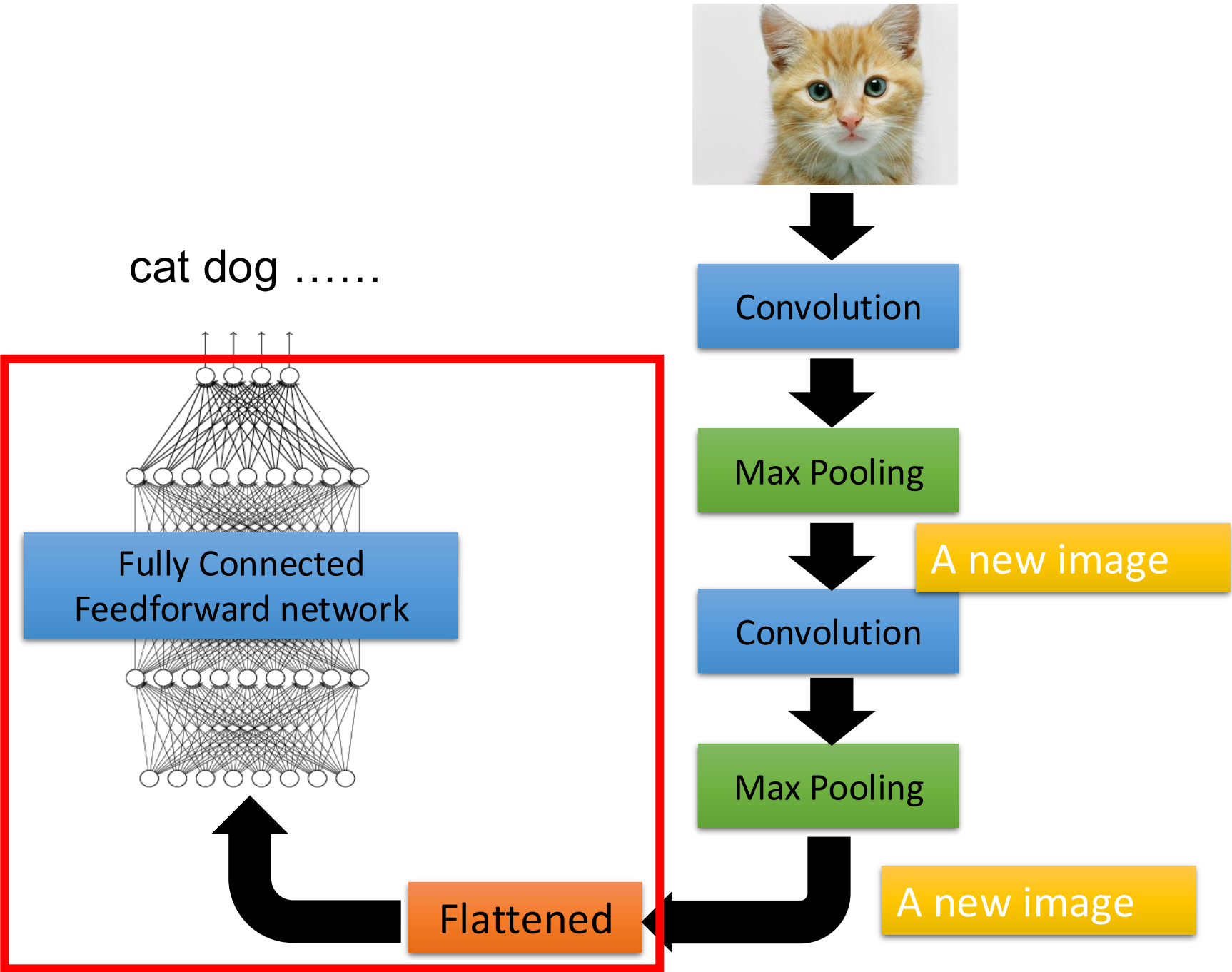


- Imagine de dimensiune mai mica decat cea originala
- Numarul de filtre reprezinta numarul de canale (adancimea)

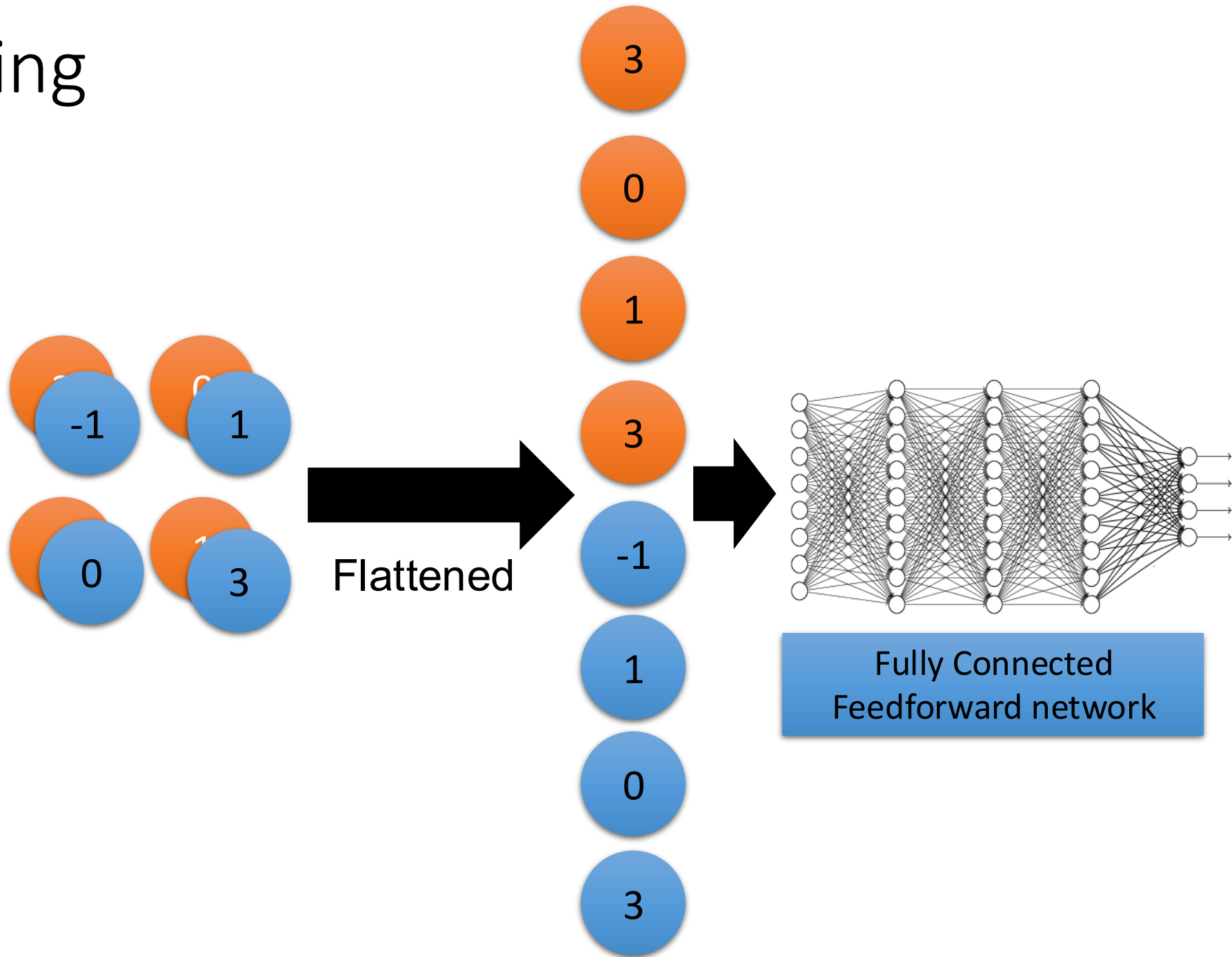


CNN

cat dog



Flattening

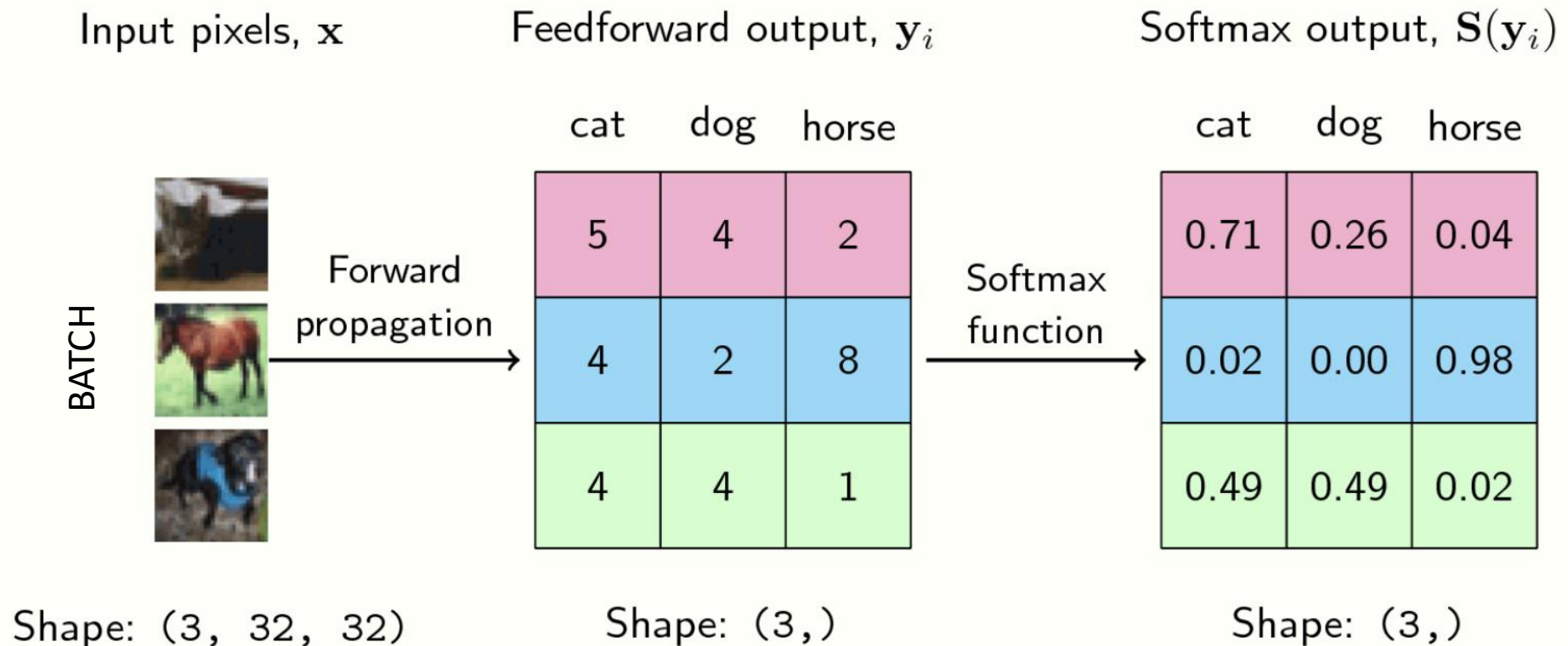


Funcția de cost

- Funcția de cost: *cross entropy* care combina un *softmax* si un cost *negative log-likelihood*.

softmax

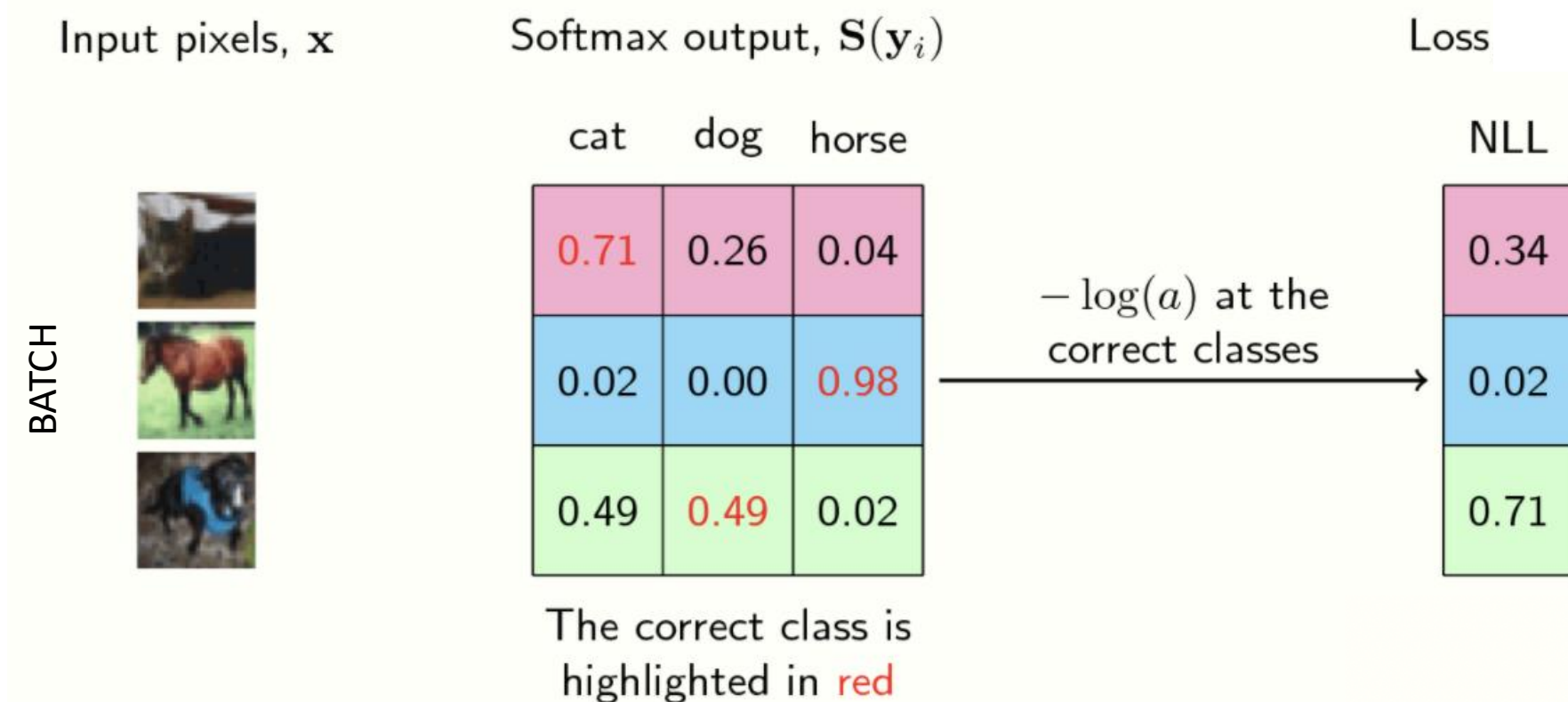
$$S(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}}$$



Negative log-likelihood

- Funcția de cost: *cross entropy* care combina un *softmax* si un cost *negative log-likelihood*.

Negative log-likelihood $L(\mathbf{y}) = -\log(\mathbf{y})$

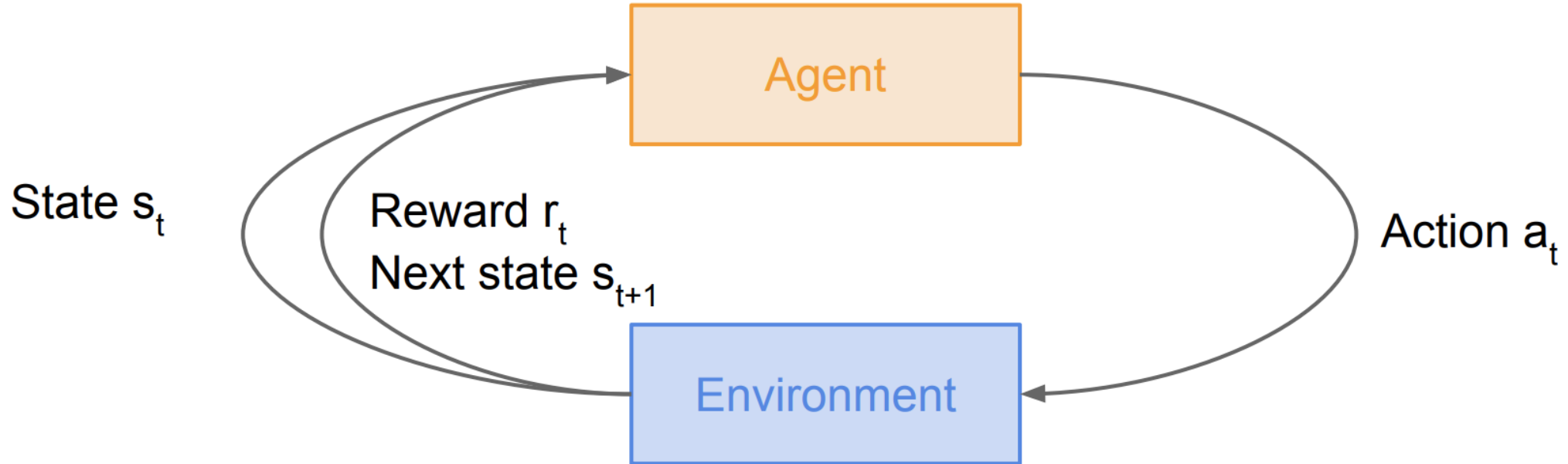


Antrenare CNNs

Repeat

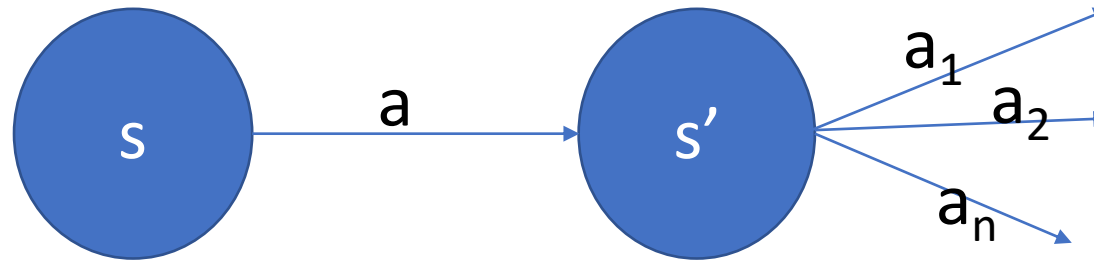
1. Pentru fiecare batch din setul de antrenare
 1. Calculeaza iesirea retelei (Forward)
 2. Backward pentru calculul gradientilor
 3. Actualizare ponderi

Q-learning



Q-learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha(R(s) + \delta \max_{a'} Q(s', a') - Q(s, a))$$



Se calculeaza pentru fiecare tranzitie din starea s in starea s'

$$Q(s, a) \leftarrow Q(s, a)(1 - \alpha) + \alpha(R(s) + \delta \max_{a'} Q(s', a'))$$

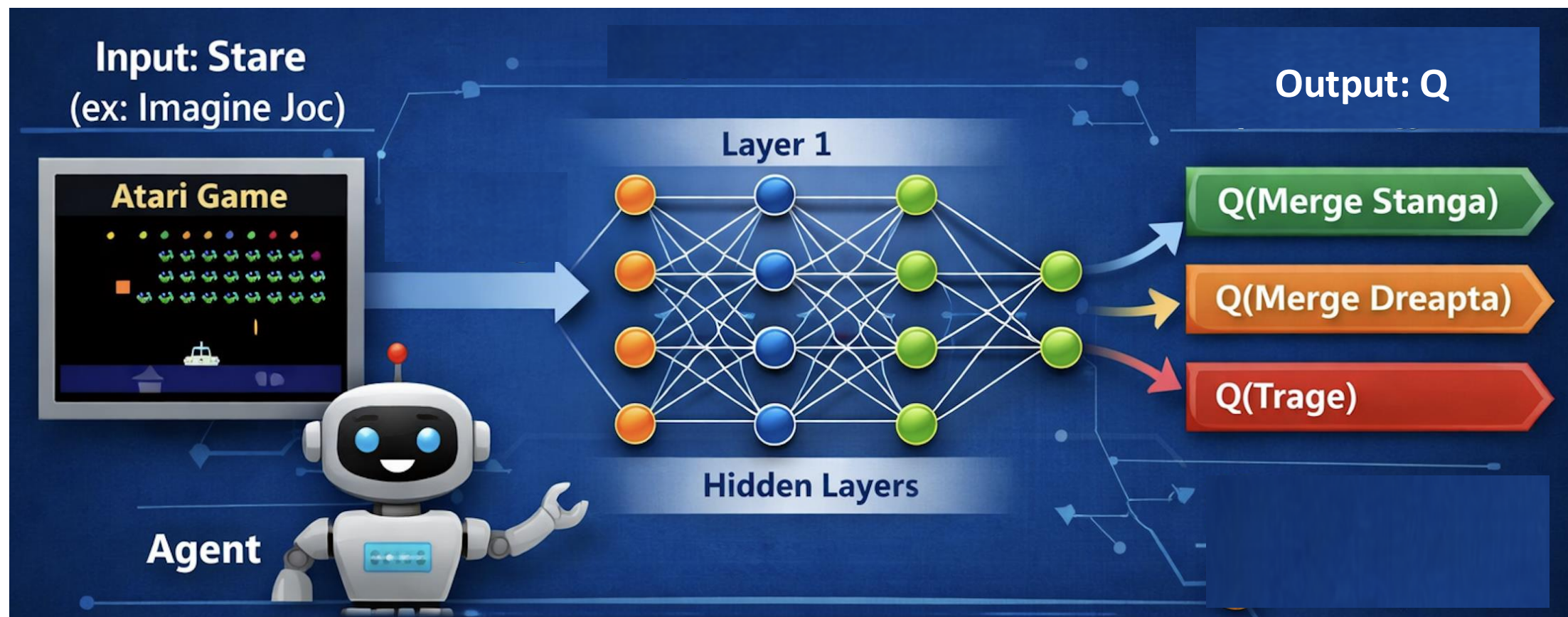
δ - factor atenuare

α – learning rate

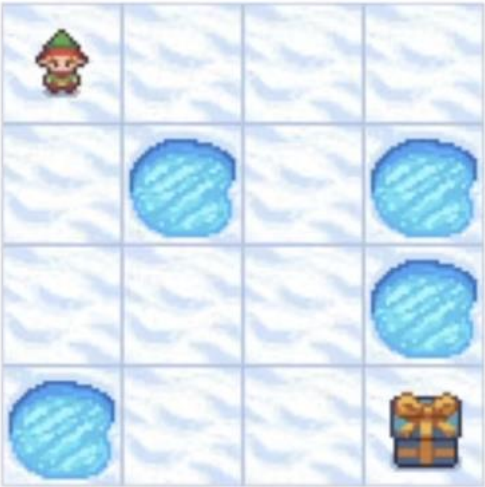
Q-learning

- Nu scaleaza:
 - stari mari
 - imagini
- Memorie multa
- Deep Q-learning: inlocuieste tabela Q cu retea neurala
- Regression problem – minimizare eroare intre predictie si target

Deep Q-Learning



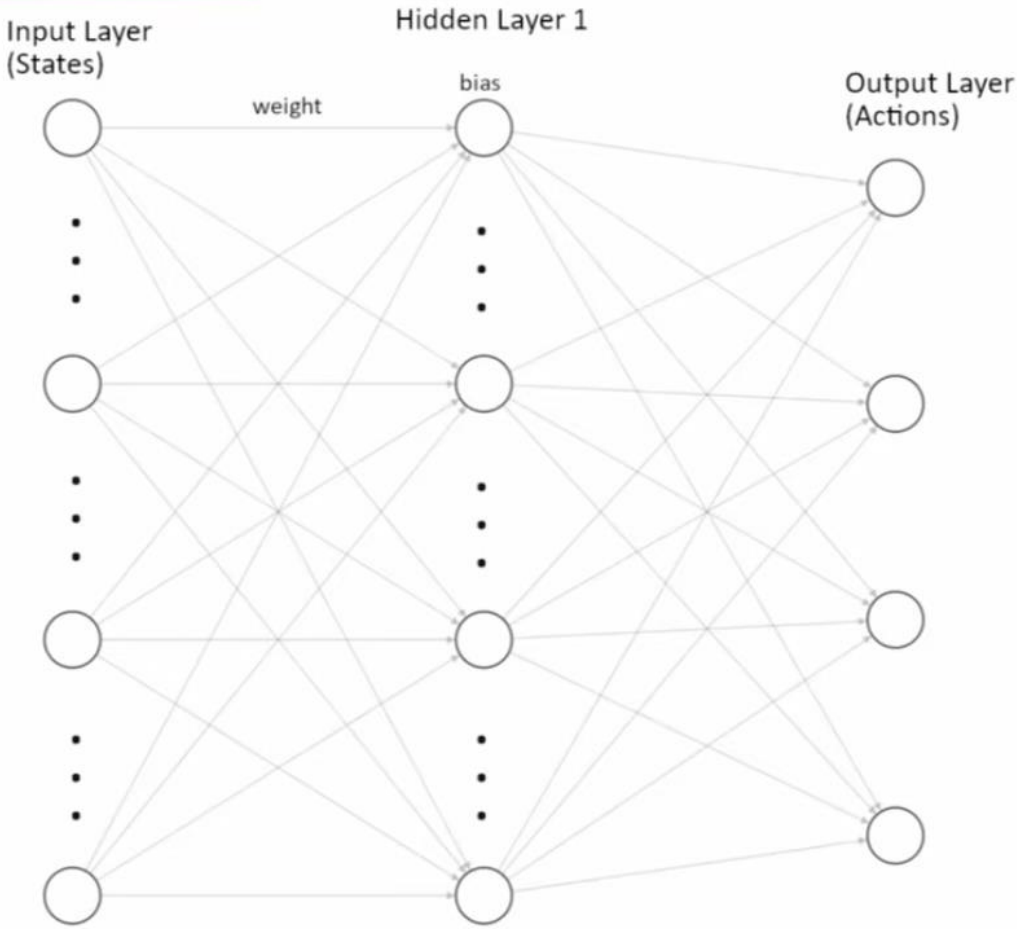
Deep Q-Learning



Q-Table

	Actions			
	← 0	↓ 1	→ 2	↑ 3
0				
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				

Deep Q-Network



one hot

Rezultat: max

Deep Q-Learning

- predicția:

$$Q_{\theta}(s, a)$$

- target-ul:

$$y = r + \gamma \max_{a'} Q_{\theta}(s', a')$$

Rețeaua:

- modifica predictia
- dar simultan se modifica si target-ul
- apare feedback instabil

Deep Q-Learning

- Instabil
- Agentul vede cadre consecutive din joc:
 - frame 1 → frame 2 → frame 3 - **foarte asemanatoare**
- Personajul merge spre dreapta:
 - toate imaginile arată aproape identic

Deep Q-learning: Experience Replay

- Nu se invata din ultima tranzitie – se alege **random un batch** din memorie
- Se antreneaza pe acel batch
- Fara replay: $s1 \rightarrow s2 \rightarrow s3 \rightarrow s4$ (ordine stricta)
- Cu replay: $[s10, s2, s7, s15]$ (ordine random)

Deep Q-learning: Experience Replay

- La fiecare pas:
 - agentul face o actiune
 - salveaza:
$$(s, a, r, s')$$
 - adauga in **replay buffer**
 - extrage batch random
 - face update pe acel batch

Deep Q-Learning: Target Network

- Train network

$$Q_{\theta}(s, a)$$

- Target network: se actualizeaza rar

$$Q_{\theta^{-}}(s', a')$$

$$y = r + \gamma \max_{a'} Q_{\theta^{-}}(s', a')$$

- Ramane corelatia datelor

Deep Q-Learning: complet

- Experience Replay & Target Network
- Pentru fiecare episod:
 - Alege starea initiala s
 - Repeta
 - Alege acțiunea a (ϵ -greedy)
 - Executa actiunea a , obtine recompensa r si noua stare s'
 - Salveaza experienta (s, a, r, s') in replay buffer
 - Alege batch replay buffer aleatory (s_i, a_i, r_i, s'_i)
 - Se calculeaza target-ul pentru fiecare element: $y_i = r_i + \gamma \max_{a'} Q_{\theta^-}(s'_i, a')$
 - Forward (train network): $Q_{\theta}(s_i, a_i)$
 - Loss: $L = (Q_{\theta}(s_i, a_i) - y_i)^2$
 - Backprop + update
 - Update target network (ocazional) $\theta^- \leftarrow \theta$
 - $s = s'$